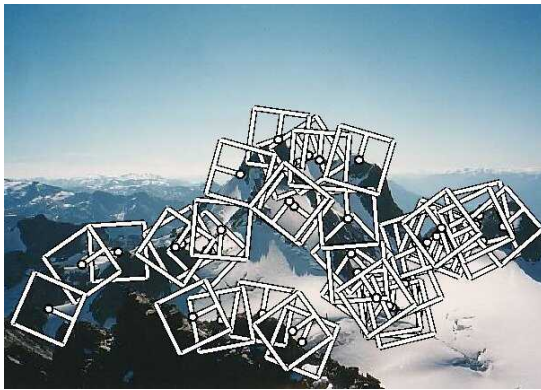


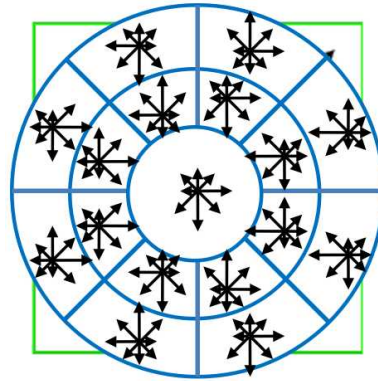
Chapter 7

Feature detection and matching

7.1	Points and patches	419
7.1.1	Feature detectors	422
7.1.2	Feature descriptors	434
7.1.3	Feature matching	441
7.1.4	Large-scale matching and retrieval	448
7.1.5	Feature tracking	452
7.1.6	<i>Application: Performance-driven animation</i>	454
7.2	Edges and contours	455
7.2.1	Edge detection	456
7.2.2	Contour detection	461
7.2.3	<i>Application: Edge editing and enhancement</i>	465
7.3	Contour tracking	466
7.3.1	Snakes and scissors	467
7.3.2	Level Sets	474
7.3.3	<i>Application: Contour tracking and rotoscoping</i>	476
7.4	Lines and vanishing points	477
7.4.1	Successive approximation	477
7.4.2	Hough transforms	477
7.4.3	Vanishing points	481
7.5	Segmentation	483
7.5.1	Graph-based segmentation	486
7.5.2	Mean shift	487
7.5.3	Normalized cuts	489
7.6	Additional reading	491
7.7	Exercises	495



(a)



(b)



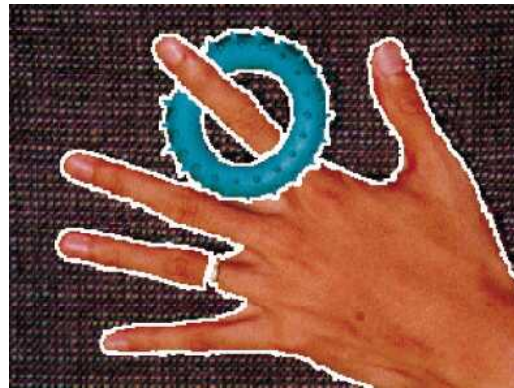
(c)



(d)



(e)



(f)

Figure 7.1 Feature detectors and descriptors can be used to analyze, describe and match images: (a) point-like interest operators (Brown, Szeliski, and Winder 2005) © 2005 IEEE; (b) GLOH descriptor (Mikolajczyk and Schmid 2005); (c) edges (Elder and Goldberg 2001) © 2001 IEEE; (d) straight lines (Sinha, Steedly et al. 2008) © 2008 ACM; (e) graph-based merging (Felzenszwalb and Huttenlocher 2004) © 2004 Springer; (f) mean shift (Comaniciu and Meer 2002) © 2002 IEEE.

Feature detection and matching are an essential component of many computer vision applications. Consider the two pairs of images shown in Figure 7.2. For the first pair, we may wish to *align* the two images so that they can be seamlessly stitched into a composite mosaic (Section 8.2). For the second pair, we may wish to establish a dense set of *correspondences* so that a 3D model can be constructed or an in-between view can be generated (Chapter 12). In either case, what kinds of *features* should you detect and then match to establish such an alignment or set of correspondences? Think about this for a few moments before reading on.

The first kind of feature that you may notice are specific locations in the images, such as mountain peaks, building corners, doorways, or interestingly shaped patches of snow. These kinds of localized features are often called *keypoint features* or *interest points* (or even *corners*) and are often described by the appearance of pixel patches surrounding the point location (Section 7.1). Another class of important features are *edges*, e.g., the profile of mountains against the sky (Section 7.2). These kinds of features can be matched based on their orientation and local appearance (edge profiles) and can also be good indicators of object boundaries and *occlusion* events in image sequences. Edges can be grouped into longer *curves* and *contours*, which can then be tracked (Section 7.3). They can also be grouped into *straight line segments*, which can be directly matched or analyzed to find *vanishing points* and hence internal and external camera parameters (Section 7.4).

In this chapter, we describe some practical approaches to detecting such features and also discuss how feature correspondences can be established across different images. Point features are now used in such a wide variety of applications that it is good practice to read and implement some of the algorithms from Section 7.1. Edges and lines provide information that is complementary to both keypoint and region-based descriptors and are well suited to describing the boundaries of manufactured objects. These alternative descriptors, while extremely useful, can be skipped in a short introductory course.

The last part of this chapter (Section 7.5) discusses bottom-up non-semantic segmentation techniques. While these were once widely used as essential components of both recognition and matching algorithms, they have mostly been supplanted by the *semantic segmentation* techniques we studied in Section 6.4. They are still used occasionally to group pixels together for faster or more reliable matching.

7.1 Points and patches

Point features can be used to find a sparse set of corresponding locations in different images, often as a precursor to computing camera pose (Chapter 11), which is a prerequisite for computing a denser set of correspondences using stereo matching (Chapter 12). Such cor-



Figure 7.2 *Two pairs of images to be matched. What kinds of features might one use to establish a set of correspondences between these images?*

respondences can also be used to align different images, e.g., when stitching image mosaics (Section 8.2) or high dynamic range images (Section 10.2), or performing video stabilization (Section 9.2.1). They are also used extensively to perform object instance recognition (Section 6.1). A key advantage of keypoints is that they permit matching even in the presence of clutter (occlusion) and large scale and orientation changes.

Feature-based correspondence techniques have been used since the early days of stereo matching (Hannah 1974; Moravec 1983; Hannah 1988) and subsequently gained popularity for image-stitching applications (Zoghlami, Faugeras, and Deriche 1997; Brown and Lowe 2007) as well as fully automated 3D modeling (Beardsley, Torr, and Zisserman 1996; Schaffalitzky and Zisserman 2002; Brown and Lowe 2005; Snavely, Seitz, and Szeliski 2006).

There are two main approaches to finding feature points and their correspondences. The first is to find features in one image that can be accurately *tracked* using a local search technique, such as correlation or least squares (Section 7.1.5). The second is to independently detect features in all the images under consideration and then *match* features based on their local appearance (Section 7.1.3). The former approach is more suitable when images are taken from nearby viewpoints or in rapid succession (e.g., video sequences), while the latter is more suitable when a large amount of motion or appearance change is expected, e.g., in stitching together panoramas (Brown and Lowe 2007), establishing correspondences in *wide baseline stereo* (Schaffalitzky and Zisserman 2002), or performing object recognition

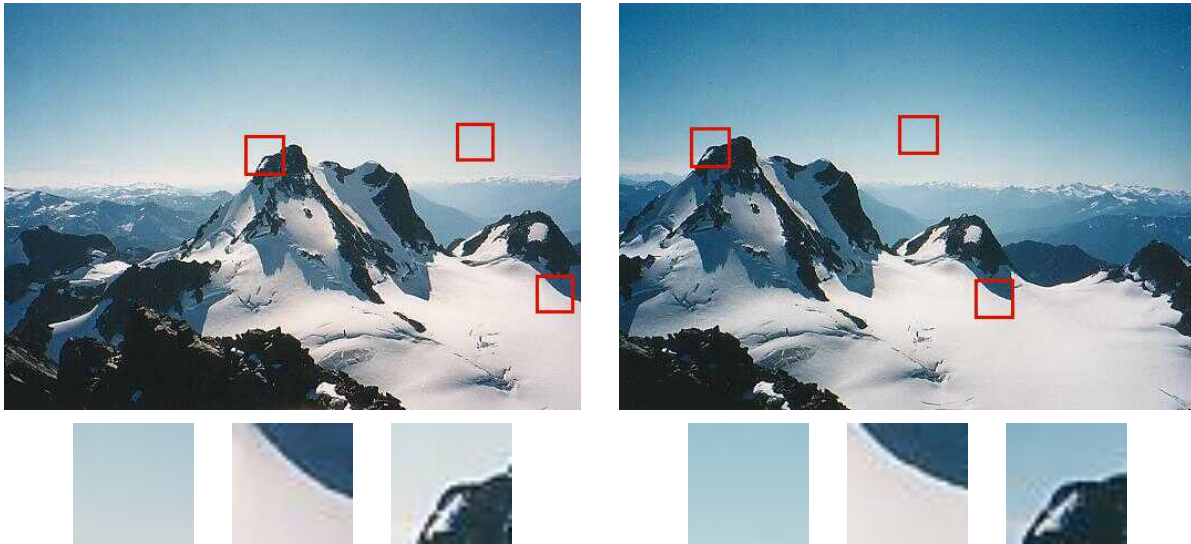


Figure 7.3 Image pairs with extracted patches below. Notice how some patches can be localized or matched with higher accuracy than others.

(Fergus, Perona, and Zisserman 2007).

In this section, we split the keypoint detection and matching pipeline into four separate stages. During the *feature detection* (extraction) stage (Section 7.1.1), each image is searched for locations that are likely to match well in other images. In the *feature description* stage (Section 7.1.2), each region around detected keypoint locations is converted into a more compact and stable (invariant) *descriptor* that can be matched against other descriptors. The *feature matching* stage (Sections 7.1.3 and 7.1.4) efficiently searches for likely matching candidates in other images. The *feature tracking* stage (Section 7.1.5) is an alternative to the third stage that only searches a small neighborhood around each detected feature and is therefore more suitable for video processing.

A wonderful example of all of these stages can be found in David Lowe’s (2004) paper, which describes the development and refinement of his *Scale Invariant Feature Transform* (SIFT). Comprehensive descriptions of alternative techniques can be found in a series of survey and evaluation papers covering both feature detection (Schmid, Mohr, and Bauckhage 2000; Mikolajczyk, Tuytelaars *et al.* 2005; Tuytelaars and Mikolajczyk 2008) and feature descriptors (Mikolajczyk and Schmid 2005; Balntas, Lenc *et al.* 2020). Shi and Tomasi (1994) and Triggs (2004) also provide nice reviews of classic (pre-neural network) feature detection techniques.

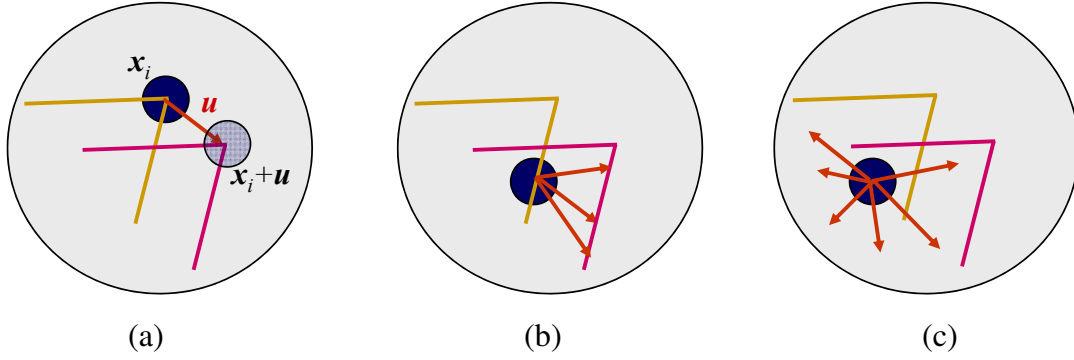


Figure 7.4 Aperture problems for different image patches: (a) stable (“corner-like”) flow; (b) classic aperture problem (barber-pole illusion); (c) textureless region. The two images I_0 (yellow) and I_1 (red) are overlaid. The red vector $\mathbf{u}$ indicates the displacement between the patch centers and the $w(\mathbf{x}_i)$ weighting function (patch window) is shown as a dark circle.

7.1.1 Feature detectors

How can we find image locations where we can reliably find correspondences with other images, i.e., what are good features to track (Shi and Tomasi 1994; Triggs 2004)? Look again at the image pair shown in Figure 7.3 and at the three sample *patches* to see how well they might be matched or tracked. As you may notice, textureless patches are nearly impossible to localize. Patches with large contrast changes (gradients) are easier to localize, although straight line segments at a single orientation suffer from the *aperture problem* (Horn and Schunck 1981; Lucas and Kanade 1981; Anandan 1989), i.e., it is only possible to align the patches along the direction *normal* to the edge direction (Figure 7.4b). Patches with gradients in at least two (significantly) different orientations are the easiest to localize, as shown schematically in Figure 7.4a.

These intuitions can be formalized by looking at the simplest possible matching criterion for comparing two image patches, i.e., their (weighted) summed square difference,

$$E_{\text{WSSD}}(\mathbf{u}) = \sum_i w(\mathbf{x}_i) [I_1(\mathbf{x}_i + \mathbf{u}) - I_0(\mathbf{x}_i)]^2, \quad (7.1)$$

where I_0 and I_1 are the two images being compared, $\mathbf{u} = (u, v)$ is the *displacement* vector, $w(\mathbf{x})$ is a spatially varying weighting (or window) function, and the summation i is over all the pixels in the patch. Note that this is the same formulation we later use to estimate motion between complete images (Section 9.1).

When performing feature detection, we do not know which other image locations the feature will end up being matched against. Therefore, we can only compute how stable this metric is with respect to small variations in position $\Delta\mathbf{u}$ by comparing an image patch against

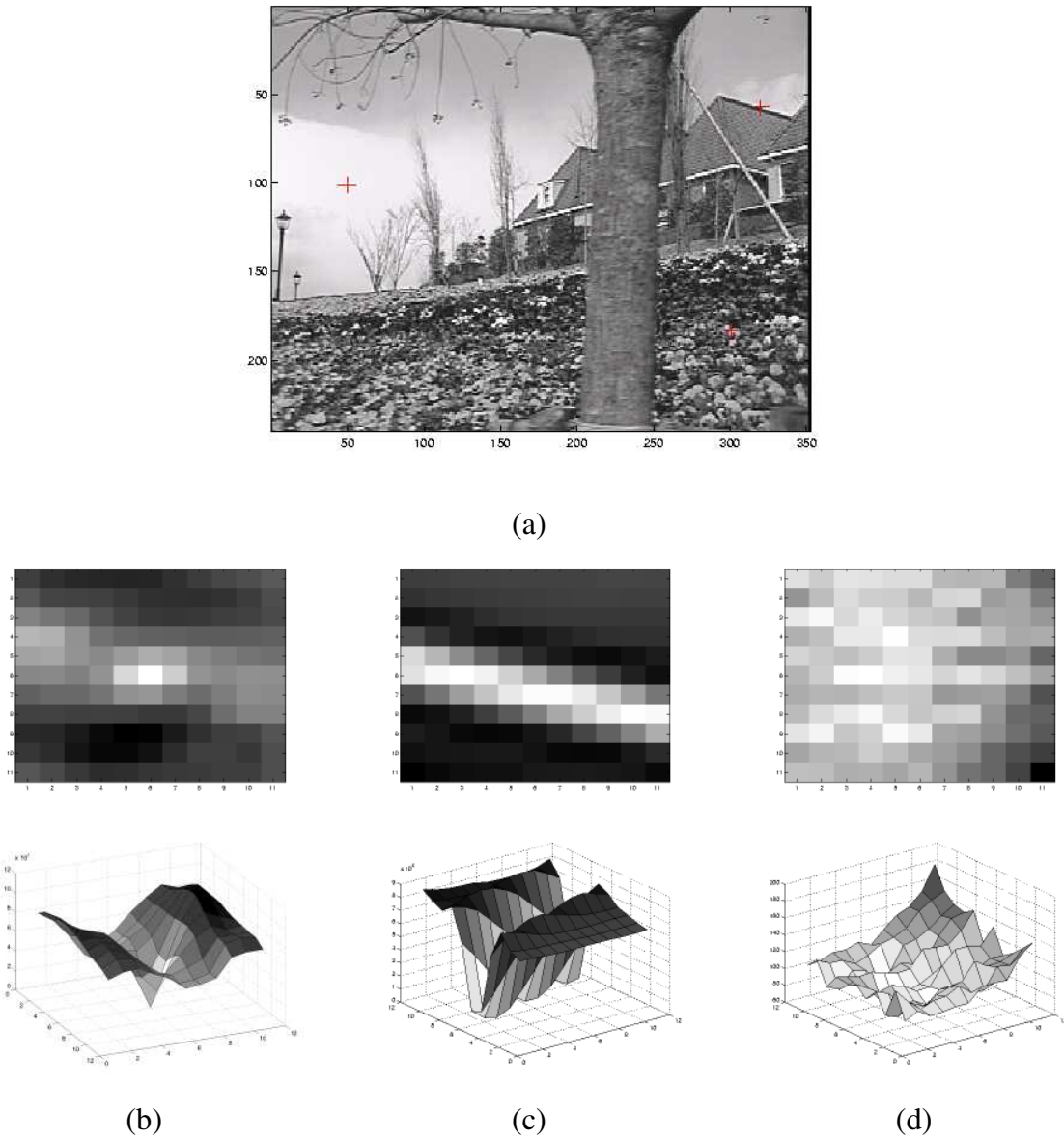


Figure 7.5 Three auto-correlation surfaces $E_{AC}(\Delta \mathbf{u})$ shown as both grayscale images and surface plots: (a) The original image is marked with three red crosses to denote where the auto-correlation surfaces were computed; (b) this patch is from the flower bed (good unique minimum); (c) this patch is from the roof edge (one-dimensional aperture problem); and (d) this patch is from the cloud (no good peak). Each grid point in figures b–d is one value of $\Delta \mathbf{u}$.

itself, which is known as an *auto-correlation function* or *surface*

$$E_{AC}(\Delta \mathbf{u}) = \sum_i w(\mathbf{x}_i) [I_0(\mathbf{x}_i + \Delta \mathbf{u}) - I_0(\mathbf{x}_i)]^2 \quad (7.2)$$

(Figure 7.5).¹ Note how the auto-correlation surface for the textured flower bed (Figure 7.5b) and the red cross in the lower right quadrant of Figure 7.5a) exhibits a strong minimum, indicating that it can be well localized. The correlation surface corresponding to the roof edge (Figure 7.5c) has a strong ambiguity along one direction, while the correlation surface corresponding to the cloud region (Figure 7.5d) has no stable minimum.

Using a Taylor Series expansion of the image function $I_0(\mathbf{x}_i + \Delta \mathbf{u}) \approx I_0(\mathbf{x}_i) + \nabla I_0(\mathbf{x}_i) \cdot \Delta \mathbf{u}$ (Lucas and Kanade 1981; Shi and Tomasi 1994), we can approximate the auto-correlation surface as

$$E_{AC}(\Delta \mathbf{u}) = \sum_i w(\mathbf{x}_i) [I_0(\mathbf{x}_i + \Delta \mathbf{u}) - I_0(\mathbf{x}_i)]^2 \quad (7.3)$$

$$\approx \sum_i w(\mathbf{x}_i) [I_0(\mathbf{x}_i) + \nabla I_0(\mathbf{x}_i) \cdot \Delta \mathbf{u} - I_0(\mathbf{x}_i)]^2 \quad (7.4)$$

$$= \sum_i w(\mathbf{x}_i) [\nabla I_0(\mathbf{x}_i) \cdot \Delta \mathbf{u}]^2 \quad (7.5)$$

$$= \Delta \mathbf{u}^T \mathbf{A} \Delta \mathbf{u}, \quad (7.6)$$

where

$$\nabla I_0(\mathbf{x}_i) = \left(\frac{\partial I_0}{\partial x}, \frac{\partial I_0}{\partial y} \right)(\mathbf{x}_i) \quad (7.7)$$

is the *image gradient* at $\mathbf{x}_i$. This gradient can be computed using a variety of techniques (Schmid, Mohr, and Bauckhage 2000). The classic “Harris” detector (Harris and Stephens 1988) uses a $[-2 \ -1 \ 0 \ 1 \ 2]$ filter, but more modern variants (Schmid, Mohr, and Bauckhage 2000; Triggs 2004) convolve the image with horizontal and vertical derivatives of a Gaussian (typically with $\sigma = 1$).

The auto-correlation matrix $\mathbf{A}$ can be written as

$$\mathbf{A} = w * \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}, \quad (7.8)$$

where we have replaced the weighted summations with discrete convolutions with the weighting kernel w . This matrix can be interpreted as a tensor (multiband) image, where the outer products of the gradients ∇I are convolved with a weighting function w to provide a per-pixel estimate of the local (quadratic) shape of the auto-correlation function.

¹Strictly speaking, a correlation is the *product* of two patches (3.12); I’m using the term here in a more qualitative sense. The weighted sum of squared differences is often called an *SSD surface* (Section 9.1).

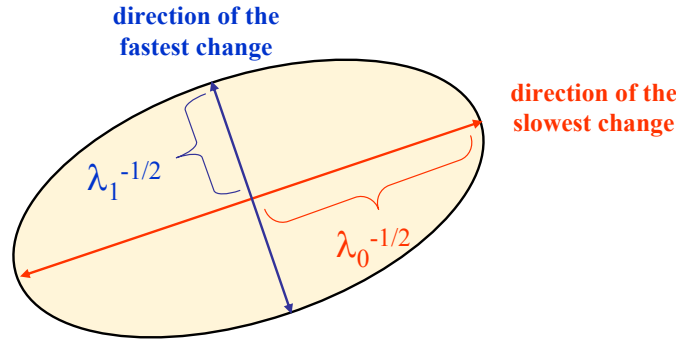


Figure 7.6 *Uncertainty ellipse corresponding to an eigenvalue analysis of the auto-correlation matrix $\mathbf{A}$.*

As first shown by Anandan (1984; 1989) and further discussed in Section 9.1.3 and Equation (9.37), the inverse of the matrix $\mathbf{A}$ provides a lower bound on the uncertainty in the location of a matching patch. It is therefore a useful indicator of which patches can be reliably matched. The easiest way to visualize and reason about this uncertainty is to perform an eigenvalue analysis of the auto-correlation matrix $\mathbf{A}$, which produces two eigenvalues (λ_0, λ_1) and two eigenvector directions (Figure 7.6). Since the larger uncertainty depends on the smaller eigenvalue, i.e., $\lambda_0^{-1/2}$, it makes sense to find maxima in the smaller eigenvalue to locate good features to track (Shi and Tomasi 1994).

Förstner–Harris. While Anandan (1984) and Lucas and Kanade (1981) were the first to analyze the uncertainty structure of the auto-correlation matrix, they did so in the context of associating certainties with optical flow measurements. Förstner (1986) and Harris and Stephens (1988) were the first to propose using local maxima in rotationally invariant scalar measures derived from the auto-correlation matrix to locate keypoints for the purpose of sparse feature matching.² Both of these techniques also proposed using a Gaussian weighting window instead of the previously used square patches, which makes the detector response insensitive to in-plane image rotations.

The minimum eigenvalue λ_0 (Shi and Tomasi 1994) is not the only quantity that can be used to find keypoints. A simpler quantity, proposed by Harris and Stephens (1988), is

$$\det(\mathbf{A}) - \alpha \operatorname{trace}(\mathbf{A})^2 = \lambda_0 \lambda_1 - \alpha (\lambda_0 + \lambda_1)^2 \quad (7.9)$$

with $\alpha = 0.06$. Unlike eigenvalue analysis, this quantity does not require the use of square roots and yet is still rotationally invariant and also downweights edge-like features where

²Schmid, Mohr, and Bauckhage (2000) and Triggs (2004) give more detailed historical reviews of feature detection algorithms.

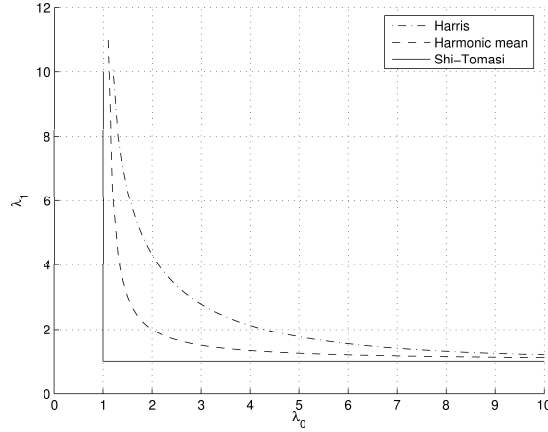


Figure 7.7 *Isocontours of popular keypoint detection functions (Brown, Szeliski, and Winder 2004). Each detector looks for points where the eigenvalues λ_0, λ_1 of $\mathbf{A} = w * \nabla I \nabla I^T$ are both large.*

$\lambda_1 \gg \lambda_0$. Triggs (2004) suggests using the quantity

$$\lambda_0 - \alpha \lambda_1 \quad (7.10)$$

(say, with $\alpha = 0.05$), which also reduces the response at 1D edges, where aliasing errors sometimes inflate the smaller eigenvalue. He also shows how the basic 2×2 Hessian can be extended to parametric motions to detect points that are also accurately localizable in scale and rotation. Brown, Szeliski, and Winder (2005), on the other hand, use the harmonic mean,

$$\frac{\det \mathbf{A}}{\text{tr } \mathbf{A}} = \frac{\lambda_0 \lambda_1}{\lambda_0 + \lambda_1}, \quad (7.11)$$

which is a smoother function in the region where $\lambda_0 \approx \lambda_1$. Figure 7.7 shows isocontours of the various interest point operators, from which we can see how the two eigenvalues are blended to determine the final interest value. Figure 7.8 shows the resulting interest operator responses for the classic Harris detector as well as the difference of Gaussian (DoG) detector discussed below.

Adaptive non-maximal suppression (ANMS). While most feature detectors simply look for local maxima in the interest function, this can lead to an uneven distribution of feature points across the image, e.g., points will be denser in regions of higher contrast. To mitigate this problem, Brown, Szeliski, and Winder (2005) only detect features that are both local maxima and whose response value is significantly (10%) greater than that of all of its neighbors within a radius r (Figure 7.9c–d). They devise an efficient way to associate suppression radii with all local maxima by first sorting them by their response strength and then creating

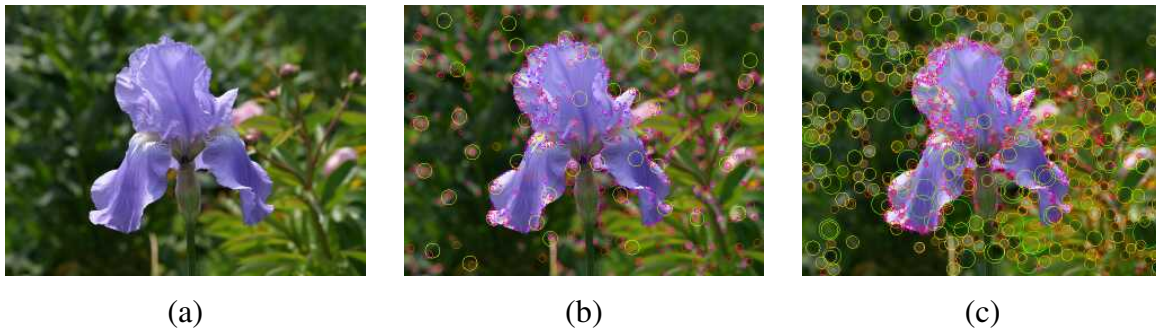


Figure 7.8 Interest operator responses: (a) Sample image, (b) Harris response, and (c) DoG response. The circle sizes and colors indicate the scale at which each interest point was detected. Notice how the two detectors tend to respond at complementary locations.

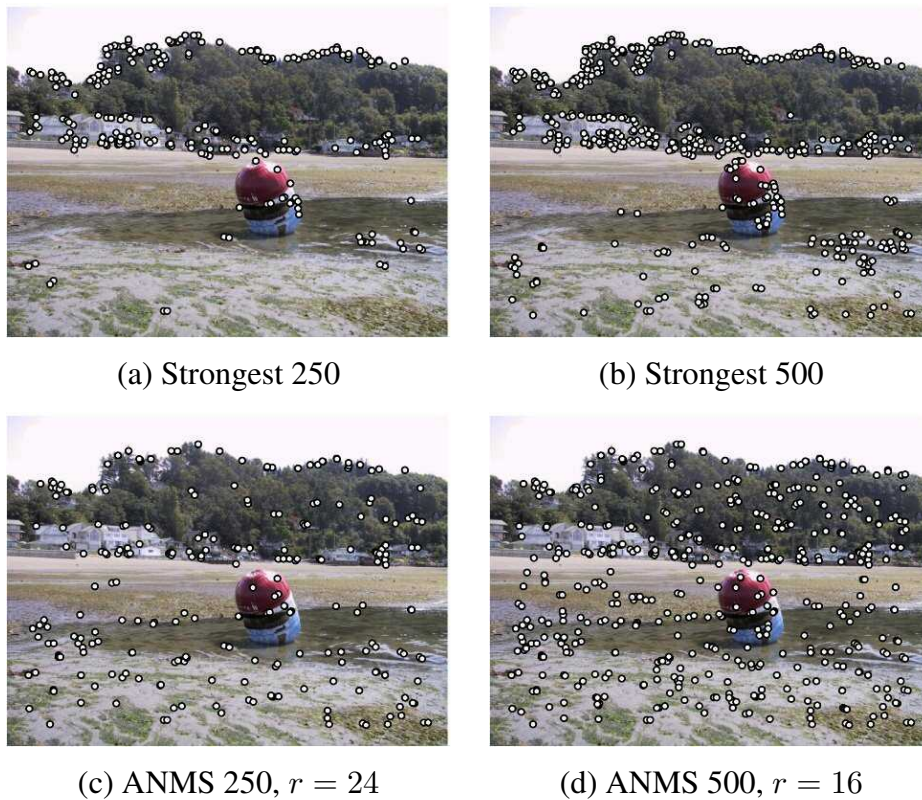


Figure 7.9 Adaptive non-maximal suppression (ANMS) (Brown, Szeliski, and Winder 2005) © 2005 IEEE: The upper two images show the strongest 250 and 500 interest points, while the lower two images show the interest points selected with adaptive non-maximal suppression, along with the corresponding suppression radius r . Note how the latter features have a much more uniform spatial distribution across the image.

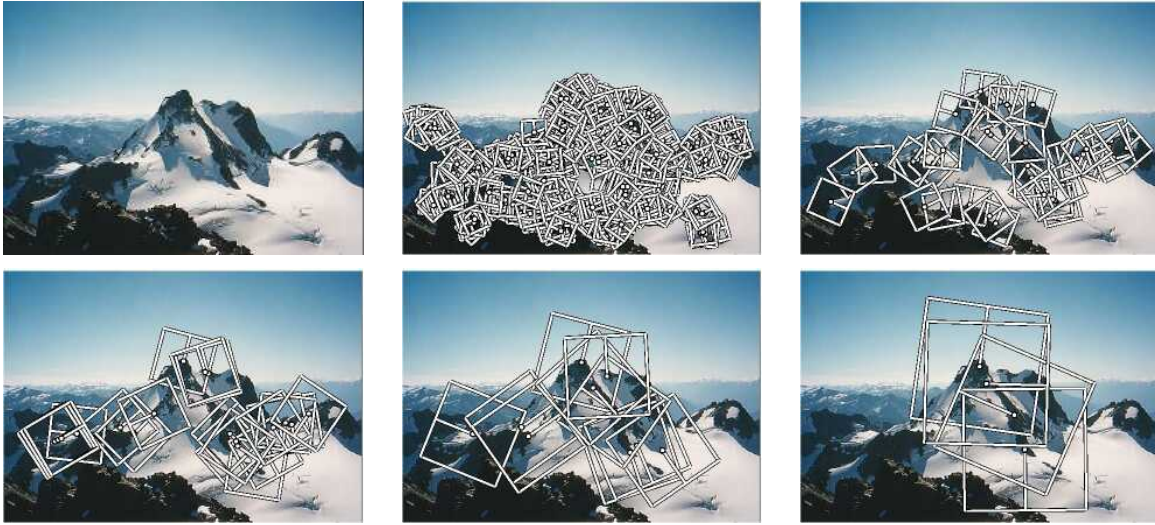


Figure 7.10 *Multi-scale oriented patches (MOPS) extracted at five pyramid levels (Brown, Szeliski, and Winder 2005) © 2005 IEEE. The boxes show the feature orientation and the region from which the descriptor vectors are sampled.*

a second list sorted by decreasing suppression radius (Brown, Szeliski, and Winder 2005). Figure 7.9 shows a qualitative comparison of selecting the top n features and using ANMS. Note that non-maximal suppression is now also an essential component of DNN-based object detectors, as discussed in Section 6.3.3.

Measuring repeatability. Given the large number of feature detectors that have been developed in computer vision, how can we decide which ones to use? Schmid, Mohr, and Bauckhage (2000) were the first to propose measuring the *repeatability* of feature detectors, which they define as the frequency with which keypoints detected in one image are found within ϵ (say, $\epsilon = 1.5$) pixels of the corresponding location in a transformed image. In their paper, they transform their planar images by applying rotations, scale changes, illumination changes, viewpoint changes, and adding noise. They also measure the *information content* available at each detected feature point, which they define as the entropy of a set of rotationally invariant local grayscale descriptors. Among the techniques they survey, they find that the improved (Gaussian derivative) version of the Harris operator with $\sigma_d = 1$ (scale of the derivative Gaussian) and $\sigma_i = 2$ (scale of the integration Gaussian) works best.

Scale invariance

In many situations, detecting features at the finest stable scale possible may not be appropriate. For example, when matching images with little high-frequency detail (e.g., clouds),

fine-scale features may not exist.

One solution to the problem is to extract features at a variety of scales, e.g., by performing the same operations at multiple resolutions in a pyramid and then matching features at the same level. This kind of approach is suitable when the images being matched do not undergo large scale changes, e.g., when matching successive aerial images taken from an airplane or stitching panoramas taken with a fixed-focal-length camera. Figure 7.10 shows the output of one such approach: the multi-scale oriented patch detector of Brown, Szeliski, and Winder (2005), for which responses at five different scales are shown.

However, for most object recognition applications, the scale of the object in the image is unknown. Instead of extracting features at many different scales and then matching all of them, it is more efficient to extract features that are stable in both location *and* scale (Lowe 2004; Mikolajczyk and Schmid 2004).

Early investigations into scale selection were performed by Lindeberg (1993; 1998b), who first proposed using extrema in the Laplacian of Gaussian (LoG) function as interest point locations. Based on this work, Lowe (2004) proposed computing a set of sub-octave Difference of Gaussian filters (Figure 7.11a), looking for 3D (space+scale) maxima in the resulting structure (Figure 7.11b), and then computing a sub-pixel space+scale location using a quadratic fit (Brown and Lowe 2002). The number of sub-octave levels was determined, after careful empirical investigation, to be three, which corresponds to a quarter-octave pyramid, which is the same as used by Triggs (2004).

As with the Harris operator, pixels where there is strong asymmetry in the local curvature of the indicator function (in this case, the DoG) are rejected. This is implemented by first computing the local Hessian of the difference image D ,

$$\mathbf{H} = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}, \quad (7.12)$$

and then rejecting keypoints for which

$$\frac{\text{Tr}(\mathbf{H})^2}{\text{Det}(\mathbf{H})} > 10. \quad (7.13)$$

While Lowe's Scale Invariant Feature Transform (SIFT) performs well in practice, it is not based on the same theoretical foundation of maximum spatial stability as the auto-correlation-based detectors. (In fact, its detection locations are often complementary to those produced by such techniques and can therefore be used in conjunction with these other approaches.) In order to add a scale selection mechanism to the Harris corner detector, Mikolajczyk and Schmid (2004) evaluate the Laplacian of Gaussian function at each detected Harris point (in a multi-scale pyramid) and keep only those points for which the Laplacian is extremal (larger

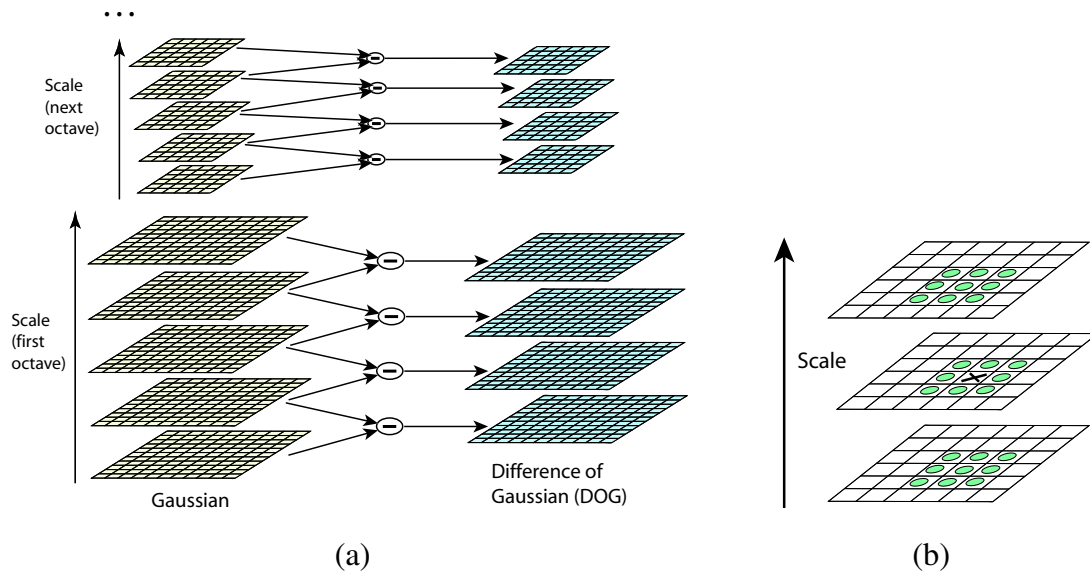


Figure 7.11 *Scale-space feature detection using a sub-octave Difference of Gaussian pyramid (Lowe 2004) © 2004 Springer: (a) Adjacent levels of a sub-octave Gaussian pyramid are subtracted to produce Difference of Gaussian images; (b) extrema (maxima and minima) in the resulting 3D volume are detected by comparing a pixel to its 26 neighbors.*

or smaller than both its coarser and finer-level values). An optional iterative refinement for both scale and position is also proposed and evaluated. Additional examples of scale-invariant region detectors are discussed by Mikolajczyk, Tuytelaars *et al.* (2005) and Tuytelaars and Mikolajczyk (2008).

Rotational invariance and orientation estimation

In addition to dealing with scale changes, most image matching and object recognition algorithms need to deal with (at least) in-plane image rotation. One way to deal with this problem is to design descriptors that are rotationally invariant (Schmid and Mohr 1997), but such descriptors have poor discriminability, i.e. they map different looking patches to the same descriptor.

A better method is to estimate a *dominant orientation* at each detected keypoint. Once the local orientation and scale of a keypoint have been estimated, a scaled and oriented patch around the detected point can be extracted and used to form a feature descriptor (Figures 7.10 and 7.15).

The simplest possible orientation estimate is the average gradient within a region around the keypoint. If a Gaussian weighting function is used (Brown, Szeliski, and Winder 2005),

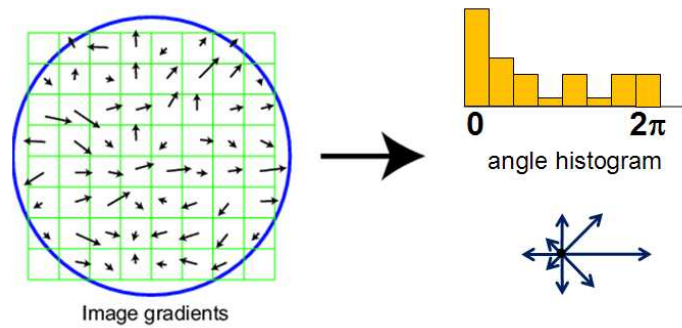


Figure 7.12 A dominant orientation estimate can be computed by creating a histogram of all the gradient orientations (weighted by their magnitudes or after thresholding out small gradients) and then finding the significant peaks in this distribution (Lowe 2004) © 2004 Springer.

this average gradient is equivalent to a first-order steerable filter (Section 3.2.3), i.e., it can be computed using an image convolution with the horizontal and vertical derivatives of Gaussian filter (Freeman and Adelson 1991). To make this estimate more reliable, it is usually preferable to use a larger aggregation window (Gaussian kernel size) than detection window (Brown, Szeliski, and Winder 2005). The orientations of the square boxes shown in Figure 7.10 were computed using this technique.

Sometimes, however, the averaged (signed) gradient in a region can be small and therefore an unreliable indicator of orientation. A more reliable technique is to look at the *histogram* of orientations computed around the keypoint. Lowe (2004) computes a 36-bin histogram of edge orientations weighted by both gradient magnitude and Gaussian distance to the center, finds all peaks within 80% of the global maximum, and then computes a more accurate orientation estimate using a three-bin parabolic fit (Figure 7.12).

Affine invariance

While scale and rotation invariance are highly desirable, for many applications such as *wide baseline stereo matching* (Pritchett and Zisserman 1998; Schaffalitzky and Zisserman 2002) or location recognition (Chum, Philbin *et al.* 2007), full affine invariance is preferred. Affine-invariant detectors not only respond at consistent locations after scale and orientation changes, they also respond consistently across affine deformations such as (local) perspective foreshortening (Figure 7.13). In fact, for a small enough patch, any continuous image warping can be well approximated by an affine deformation.

To introduce affine invariance, several authors have proposed fitting an ellipse to the auto-

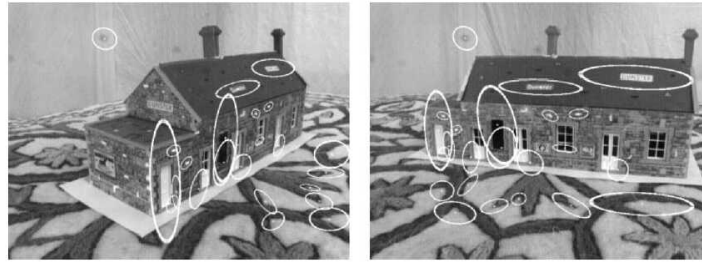


Figure 7.13 *Affine region detectors used to match two images taken from dramatically different viewpoints (Mikolajczyk and Schmid 2004) © 2004 Springer.*



Figure 7.14 *Maximally stable extremal regions (MSERs) extracted and matched from a number of images (Matas, Chum et al. 2004) © 2004 Elsevier.*

correlation or Hessian matrix (using eigenvalue analysis) and then using the principal axes and ratios of this fit as the affine coordinate frame (Lindeberg and Gårding 1997; Baumberg 2000; Mikolajczyk and Schmid 2004; Mikolajczyk, Tuytelaars *et al.* 2005; Tuytelaars and Mikolajczyk 2008).

Another important affine invariant region detector is the maximally stable extremal region (MSER) detector developed by Matas, Chum *et al.* (2004). To detect MSERs, binary regions are computed by thresholding the image at all possible gray levels (the technique therefore only works for grayscale images). This operation can be performed efficiently by first sorting all pixels by gray value and then incrementally adding pixels to each connected component as the threshold is changed (Nistér and Stewénus 2008). As the threshold is changed, the area of each component (region) is monitored; regions whose rate of change of area with respect to the threshold is minimal are defined as *maximally stable*. Such regions are therefore invariant to both affine geometric and photometric (linear bias-gain or smooth monotonic) transformations (Figure 7.14). If desired, an affine coordinate frame can be fit to each detected region using its moment matrix.

The area of feature point detection continues to be very active, with papers appearing every year at major computer vision conferences. Mikolajczyk, Tuytelaars *et al.* (2005) and Tuytelaars and Mikolajczyk (2008) survey a number of popular (pre-DNN) affine region detectors and provide experimental comparisons of their invariance to common image transfor-

mations such as scaling, rotations, noise, and blur.

More recent papers published in the last decade include:

- SURF (Bay, Ess *et al.* 2008), which uses integral images for faster convolutions;
- FAST and FASTER (Rosten, Porter, and Drummond 2010), one of the first learned detectors;
- BRISK (Leutenegger, Chli, and Siegwart 2011), which uses a scale-space FAST detector together with a bit-string descriptor;
- ORB (Rublee, Rabaud *et al.* 2011), which adds orientation to FAST; and
- KAZE (Alcantarilla, Bartoli, and Davison 2012) and Accelerated-KAZE (Alcantarilla, Nuevo, and Bartoli 2013), which use non-linear diffusion to select the scale for feature detection.

While FAST introduced the idea of machine learning for feature detectors, more recent papers use convolutional neural networks to perform the detection. These include:

- Learning covariant feature detectors (Lenc and Vedaldi 2016);
- Learning to assign orientations to feature points (Yi, Verdie *et al.* 2016);
- LIFT, learned invariant feature transforms (Yi, Trulls *et al.* 2016), SuperPoint, self-supervised interest point detection and description (DeTone, Malisiewicz, and Rabinovich 2018), and LF-Net, learning local features from images (Ono, Trulls *et al.* 2018), all three of which jointly optimize the detectors and descriptors in a single (multi-head) pipeline;
- AffNet (Mishkin, Radenovic, and Matas 2018), which detects matchable affine-covariant regions;
- Key.Net (Barroso-Laguna, Riba *et al.* 2019), which uses a combination of handcrafted and learned CNN features; and
- D2-Net (Dusmanu, Rocco *et al.* 2019), R2D2 (Revaud, Weinzaepfel *et al.* 2019), and D2D (Tian, Balntas *et al.* 2020), which all extract dense local feature descriptors and then keeps the ones that have high saliency or repeatability.

These last two papers also contains a nice review of other recent feature detectors, as does the paper by Balntas, Lenc *et al.* (2020).

Of course, keypoints are not the only features that can be used for registering images. Zoghلامي, Faugeras, and Deriche (1997) use line segments as well as point-like features to estimate homographies between pairs of images, whereas Bartoli, Coquerelle, and Sturm (2004) use line segments with local correspondences along the edges to extract 3D structure and motion. Tuytelaars and Van Gool (2004) use affine invariant regions to detect correspondences for wide baseline stereo matching, whereas Kadir, Zisserman, and Brady (2004) detect salient regions where patch entropy and its rate of change with scale are locally maximal. Corso and Hager (2005) use a related technique to fit 2D oriented Gaussian kernels to homogeneous regions. More details on techniques for finding and matching curves, lines, and regions can be found later in this chapter.

7.1.2 Feature descriptors

After detecting keypoint features, we must *match* them, i.e., we must determine which features come from corresponding locations in different images. In some situations, e.g., for video sequences (Shi and Tomasi 1994) or for stereo pairs that have been *rectified* (Zhang, Deriche *et al.* 1995; Loop and Zhang 1999; Scharstein and Szeliski 2002), the local motion around each feature point may be mostly translational. In this case, simple error metrics, such as the *sum of squared differences* or *normalized cross-correlation*, described in Section 9.1, can be used to directly compare the intensities in small patches around each feature point. (The comparative study by Mikolajczyk and Schmid (2005), discussed below, uses cross-correlation.) Because feature points may not be exactly located, a more accurate matching score can be computed by performing incremental motion refinement as described in Section 9.1.3, but this can be time-consuming and can sometimes even decrease performance (Brown, Szeliski, and Winder 2005).

In most cases, however, the local appearance of features will change in orientation and scale, and sometimes even undergo affine deformations. Extracting a local scale, orientation, or affine frame estimate and then using this to resample the patch before forming the feature descriptor is thus usually preferable (Figure 7.15).

Even after compensating for these changes, the local appearance of image patches will usually still vary from image to image. How can we make image descriptors more invariant to such changes, while still preserving discriminability between different (non-corresponding) patches? Mikolajczyk and Schmid (2005) review a number of view-invariant local image descriptors and experimentally compare their performance. More recently, Balntas, Lenc *et al.* (2020) and Jin, Mishkin *et al.* (2021) compare the large number of learned feature descriptors developed in the prior decade.³ Below, we describe a few of these descriptors in

³Many recent publications such as Tian, Yu *et al.* (2019) use their HPatches dataset to compare their performance

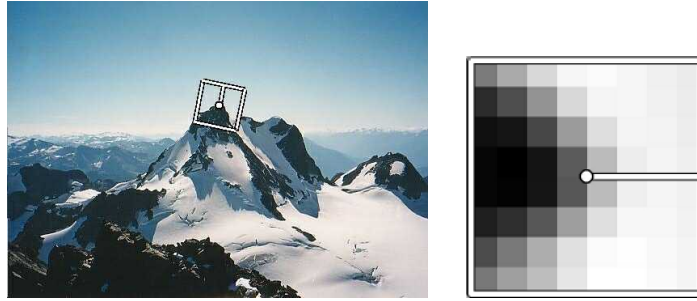


Figure 7.15 Once a local scale and orientation estimate has been determined, MOPS descriptors are formed using an 8×8 sampling of bias and gain normalized intensity values, with a sample spacing of five pixels relative to the detection scale (Brown, Szeliski, and Winder 2005) © 2005 IEEE. This low frequency sampling gives the features some robustness to interest point location error and is achieved by sampling at a higher pyramid level than the detection scale.

more detail.

Bias and gain normalization (MOPS). For tasks that do not exhibit large amounts of foreshortening, such as image stitching, simple normalized intensity patches perform reasonably well and are simple to implement (Brown, Szeliski, and Winder 2005) (Figure 7.15). To compensate for slight inaccuracies in the feature point detector (location, orientation, and scale), multi-scale oriented patches (MOPS) are sampled at a spacing of five pixels relative to the detection scale, using a coarser level of the image pyramid to avoid aliasing. To compensate for affine photometric variations (linear exposure changes or bias and gain, (3.3)), patch intensities are re-scaled so that their mean is zero and their variance is one.

Scale invariant feature transform (SIFT). SIFT features (Lowe 2004) are formed by computing the gradient at each pixel in a 16×16 window around the detected keypoint, using the appropriate level of the Gaussian pyramid at which the keypoint was detected. The gradient magnitudes are downweighted by a Gaussian fall-off function (shown as a blue circle in Figure 7.16a) to reduce the influence of gradients far from the center, as these are more affected by small misregistrations.

In each 4×4 quadrant, a gradient orientation histogram is formed by (conceptually) adding the gradient values weighted by the Gaussian fall-off function to one of eight orientation histogram bins. To reduce the effects of location and dominant orientation misestimation, each of the original 256 weighted gradient magnitudes is softly added to $2 \times 2 \times 2$ adjacent

against previous approaches.

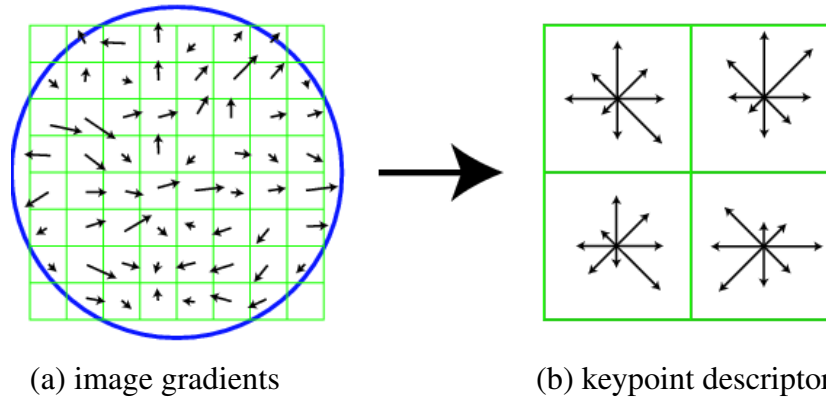


Figure 7.16 A schematic representation of Lowe’s (2004) scale invariant feature transform (SIFT): (a) Gradient orientations and magnitudes are computed at each pixel and weighted by a Gaussian fall-off function (blue circle). (b) A weighted gradient orientation histogram is then computed in each subregion, using trilinear interpolation. While this figure shows an 8×8 pixel patch and a 2×2 descriptor array, Lowe’s actual implementation uses 16×16 patches and a 4×4 array of eight-bin histograms.

histogram bins in the (x, y, θ) space using trilinear interpolation. Softly distributing values to adjacent histogram bins is generally a good idea in any application where histograms are being computed, e.g., for Hough transforms (Section 7.4.2) or local histogram equalization (Section 3.1.4).

The 4×4 array of eight-bin histogram yields 128 non-negative values form a raw version of the SIFT descriptor vector. To reduce the effects of contrast or gain (additive variations are already removed by the gradient), the 128-D vector is normalized to unit length. To further make the descriptor robust to other photometric variations, values are clipped to 0.2 and the resulting vector is once again renormalized to unit length.

PCA-SIFT. Ke and Sukthankar (2004) propose a simpler way to compute descriptors inspired by SIFT; it computes the x and y (gradient) derivatives over a 39×39 patch and then reduces the resulting 3042-dimensional vector to 36 using principal component analysis (PCA) (Section 5.2.3 and Appendix A.1.2). Another popular variant of SIFT is SURF (Bay, Ess *et al.* 2008), which uses box filters to approximate the derivatives and integrals used in SIFT.

RootSIFT. Arandjelović and Zisserman (2012) observe that by simply re-normalizing SIFT descriptors using an L_1 measure and then taking the square root of each component, a dramatic increase in performance (discriminability) can be obtained.

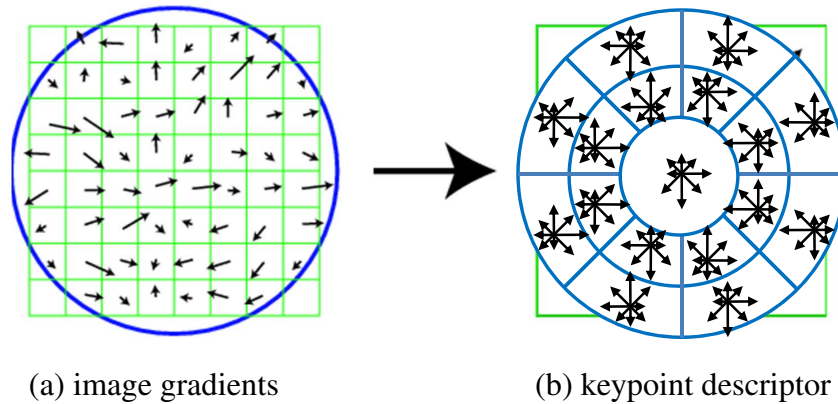


Figure 7.17 The gradient location-orientation histogram (GLOH) descriptor uses log-polar bins instead of square bins to compute orientation histograms (Mikolajczyk and Schmid 2005). GLOH uses 16 gradient orientations inside each bin, although this figure only shows 8 to appear less cluttered.

Gradient location-orientation histogram (GLOH). This descriptor, developed by Mikolajczyk and Schmid (2005), is a variant of SIFT that uses a log-polar binning structure instead of the four quadrants used by Lowe (2004) (Figure 7.17). The spatial bins extend over the radii $0 \dots 6$, $6 \dots 11$, and $11 \dots 15$, with eight angular bins (except for the single central region), for a total of 17 spatial bins and GLOH uses 16 orientation bins instead of the 8 used in SIFT. The 272-dimensional histogram is then projected onto a 128-dimensional descriptor using PCA trained on a large database. In their evaluation, Mikolajczyk and Schmid (2005) found that GLOH, which has the best performance overall, outperforms SIFT by a small margin.

Steerable filters. Steerable filters (Section 3.2.3) are combinations of derivative of Gaussian filters that permit the rapid computation of even and odd (symmetric and anti-symmetric) edge-like and corner-like features at all possible orientations (Freeman and Adelson 1991). Because they use reasonably broad Gaussians, they too are somewhat insensitive to localization and orientation errors.

Performance of local descriptors. Among the local descriptors that Mikolajczyk and Schmid (2005) compared, they found that GLOH performed best, followed closely by SIFT. They also present results for many other descriptors not covered in this book.

The field of feature descriptors continued to advance rapidly, with some techniques looking at local color information (van de Weijer and Schmid 2006; Abdel-Hakim and Farag 2006). Winder and Brown (2007) develop a multi-stage framework for feature descriptor computation that subsumes both SIFT and GLOH (Figure 7.18a) and also allows them to

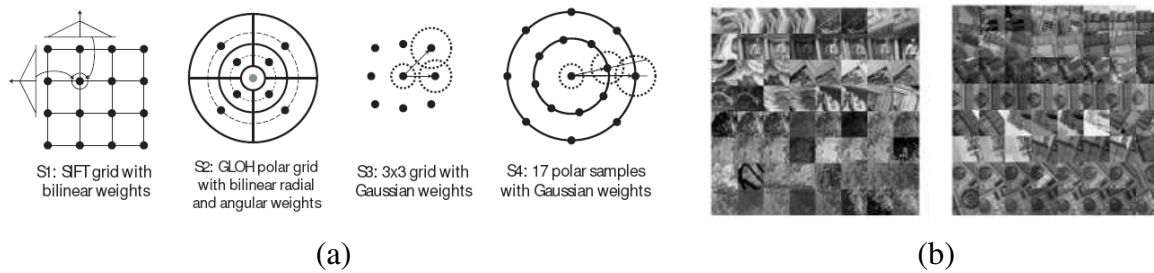


Figure 7.18 *Spatial summation blocks for SIFT, GLOH, and some related feature descriptors (Winder and Brown 2007) © 2007 IEEE: (a) The parameters for the features, e.g., their Gaussian weights, are learned from a training database of (b) matched real-world image patches obtained from robust structure from motion applied to internet photo collections (Hua, Brown, and Winder 2007).*

learn optimal parameters for newer descriptors that outperform previous hand-tuned descriptors. Hua, Brown, and Winder (2007) extend this work by learning lower-dimensional projections of higher-dimensional descriptors that have the best discriminative power, and Brown, Hua, and Winder (2011) further extend it by learning the optimal placement of the pooling regions. All of these papers use a database of real-world image patches (Figure 7.18b) obtained by sampling images at locations that were reliably matched using a robust structure-from-motion algorithm applied to internet photo collections (Snavely, Seitz, and Szeliski 2006; Goesele, Snavely *et al.* 2007). In concurrent work, Tola, Lepetit, and Fua (2010) developed a similar DAISY descriptor for dense stereo matching and optimized its parameters based on ground truth stereo data.

While these techniques construct feature detectors that optimize for repeatability across *all* object classes, it is also possible to develop class- or instance-specific feature detectors that maximize *discriminability* from other classes (Ferencz, Learned-Miller, and Malik 2008). If planar surface orientations can be determined in the images being matched, it is also possible to extract viewpoint-invariant patches (Wu, Clipp *et al.* 2008).

A more recent trend has been the development of binary bit-string feature descriptors, which can take advantage of fast Hamming distance operators in modern computer architectures. The BRIEF descriptor (Calonder, Lepetit *et al.* 2010) compares 128 different pairs of pixel values (denoted as line segments in Figure 7.19a) scattered around the keypoint location to obtain a 128-bit vector. ORB (Rublee, Rabaud *et al.* 2011) adds an orientation component to the FAST detector before computing oriented BRIEF descriptors. BRISK (Leutenegger, Chli, and Siewgwart 2011) adds scale-space analysis to the FAST detector and a radially symmetric sampling pattern (Figure 7.19b) to produce the binary descriptor. FREAK (Alahi,

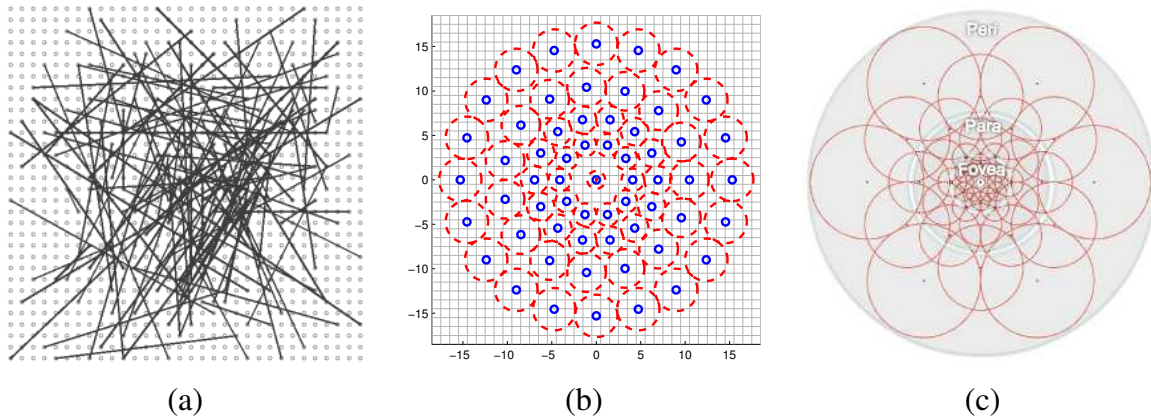


Figure 7.19 Binary bit-string feature descriptors: (a) the BRIEF descriptor compares 128 pairs of pixel values (denoted by line segments) and stores the comparison results in a 128-bit vector (Calonder, Lepetit et al. 2010) © 2010 Springer; (b) BRISK sampling pattern and Gaussian blur radii; (Leutenegger, Chli, and Siegwart 2011) © 2011 IEEE; (c) FREAK retinal sampling pattern (Alahi, Ortiz, and Vanderghenst 2012) © 2012 IEEE.

Ortiz, and Vanderghenst 2012) uses a more pronounced “retinal” (log-polar) sampling pattern paired with a cascade of bit comparisons for even greater speed and efficiency. The survey and evaluation by Mukherjee, Wu, and Wang (2015) compares all of these “classic” feature detectors and descriptors.

Since 2015 or so, most of the new feature descriptors are constructed using deep learning techniques, as surveyed in Balntas, Lenc et al. (2020) and Jin, Mishkin et al. (2021). Some of these descriptors, such as LIFT (Yi, Trulls et al. 2016), TFeat (Balntas, Riba et al. 2016), HPatches (Balntas, Lenc et al. 2020), L2-Net (Tian, Fan, and Wu 2017), HardNet (Mishchuk, Mishkin et al. 2017), Geodesc (Luo, Shen et al. 2018), LF-Net (Ono, Trulls et al. 2018), SOSNet (Tian, Yu et al. 2019), and Key.Net (Barroso-Laguna, Riba et al. 2019) operate on patches, much like the classical SIFT approach. They hence require an initial local feature detector to determine the center of the patch and use a predetermined patch size when constructing the input to the network.

In contrast, approaches such as DELF (Noh, Araujo et al. 2017), SuperPoint (DeTone, Malisiewicz, and Rabinovich 2018), D2-Net (Dusmanu, Rocco et al. 2019), ContextDesc (Luo, Shen et al. 2019), R2D2 (Revaud, Weinzaepfel et al. 2019), ASLFeat (Luo, Zhou et al. 2020), and CAPS (Wang, Zhou et al. 2020) use the entire image as the input to the descriptor computation. This has the added benefit that the receptive field used to compute the descriptor can be learned from the data and does not require specifying a patch size. Theoretically, these CNN models can learn receptive fields that use all of the pixels in the image, although in

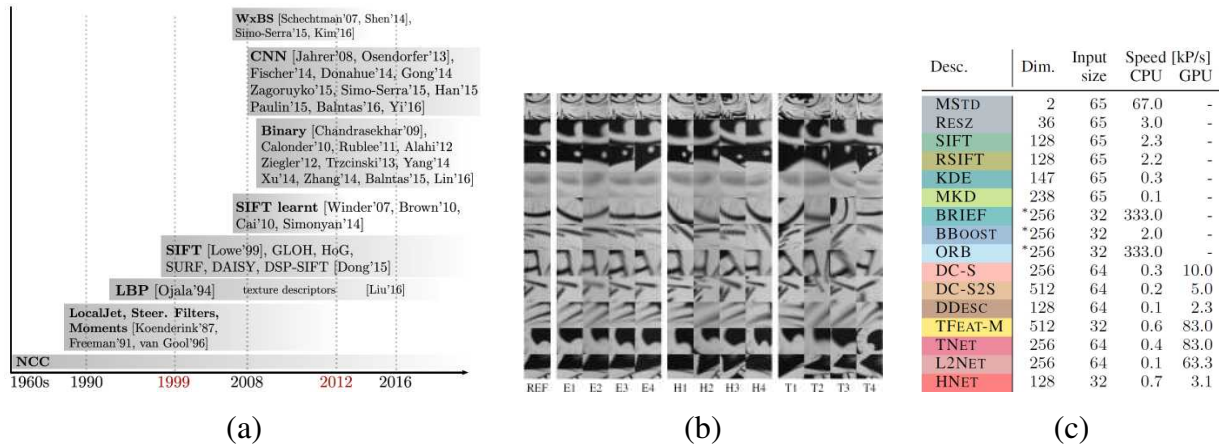


Figure 7.20 *HPatches local descriptors benchmark (Balntas, Lenc et al. 2020) © 2019 IEEE: (a) chronology of feature descriptors; (b) typical patches in the dataset (grouped by Easy, Hard, and Tough); (c) size and speed of different descriptors.*

practice they tend to use Gaussian-like receptive fields (Zhou, Khosla *et al.* 2015; Luo, Li *et al.* 2016; Selvaraju, Cogswell *et al.* 2017).

In the HPatches benchmark (Figure 7.20) for evaluating patch matching by Balntas, Lenc *et al.* (2020), HardNet and L2-net performed the best on average. Another paper (Wang, Zhou *et al.* 2020) shows CAPS and R2D2 as the best performers, while S2DNet (Germain, Bourmaud, and Lepetit 2020) and LISRD (Pautrat, Larsson *et al.* 2020) also claim state-of-the-art performance, while the WISW benchmark (Bellavia and Colombo 2020) shows that traditional descriptors such as SIFT enhanced with more recent ideas do the best. On the wide baseline image matching benchmark by Jin, Mishkin *et al.* (2021),⁴ HardNet, Key.Net, and D2-Net were top performers (e.g., D2-Net had the highest number of landmarks), although the results were quite task-dependent and the Difference of Gaussian detector was still the best. The performance of these descriptors on matching features across large illumination differences (day-night) has also been studied (Radenović, Schönberger *et al.* 2016; Zhou, Sattler, and Jacobs 2016; Mishkin 2021).

The most recent trend in wide-baseline matching has been to densely extract features without a detector stage and to then match and refine the set of correspondences (Jiang, Trulls *et al.* 2021; Sarlin, Unagar *et al.* 2021; Sun, Shen *et al.* 2021; Truong, Danelljan *et al.* 2021; Zhou, Sattler, and Leal-Taixé 2021). Some of these more recent techniques have been evaluated by Mishkin (2021).

⁴The benchmark is associated with the CVPR Workshop on Image Matching: Local Features & Beyond: <https://image-matching-workshop.github.io>.

7.1.3 Feature matching

Once we have extracted features and their descriptors from two or more images, the next step is to establish some preliminary feature matches between these images. The approach we take depends partially on the application, e.g., different strategies may be preferable for matching images that are known to overlap (e.g., in image stitching) vs. images that may have no correspondence whatsoever (e.g., when trying to recognize objects from a database).

In this section, we divide this problem into two separate components. The first is to select a *matching strategy*, which determines which correspondences are passed on to the next stage for further processing. The second is to devise efficient *data structures* and *algorithms* to perform this matching as quickly as possible, which we expand on in Section 7.1.4.

Matching strategy and error rates

Determining which feature matches are reasonable to process further depends on the context in which the matching is being performed. Say we are given two images that overlap to a fair amount (e.g., for image stitching or for tracking objects in a video). We know that most features in one image are likely to match the other image, although some may not match because they are occluded or their appearance has changed too much.

On the other hand, if we are trying to recognize how many known objects appear in a cluttered scene (Figure 6.2), most of the features may not match. Furthermore, a large number of potentially matching objects must be searched, which requires more efficient strategies, as described below.

To begin with, we assume that the feature descriptors have been designed so that Euclidean (vector magnitude) distances in feature space can be directly used for ranking potential matches. If it turns out that certain parameters (axes) in a descriptor are more reliable than others, it is usually preferable to re-scale these axes ahead of time, e.g., by determining how much they vary when compared against other known good matches (Hua, Brown, and Winder 2007). A more general process, which involves transforming feature vectors into a new scaled basis, is called *whitening* and is discussed in more detail in the context of eigenface-based face recognition (Section 5.2.3).

Given a Euclidean distance metric, the simplest matching strategy is to set a threshold (maximum distance) and to return all matches from other images within this threshold. Setting the threshold too high results in too many *false positives*, i.e., incorrect matches being returned. Setting the threshold too low results in too many *false negatives*, i.e., too many correct matches being missed (Figure 7.21).

We can quantify the performance of a matching algorithm at a particular threshold by

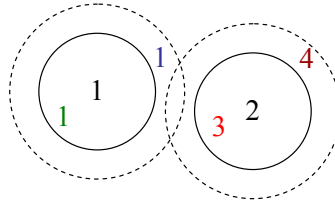


Figure 7.21 *False positives and negatives: The black digits 1 and 2 are features being matched against a database of features in other images. At the current threshold setting (the solid circles), the green 1 is a true positive (good match), the blue 1 is a false negative (failure to match), and the red 3 is a false positive (incorrect match). If we set the threshold higher (the dashed circles), the blue 1 becomes a true positive but the brown 4 becomes an additional false positive.*

	True matches	True non-matches	
Predicted matches	TP = 18	FP = 4	P' = 22
Predicted non-matches	FN = 2	TN = 76	N' = 78
	P = 20	N = 80	Total = 100

TPR = 0.90	FPR = 0.05	ACC = 0.94
------------	------------	------------

Table 7.1 *The number of matches correctly and incorrectly estimated by a feature matching algorithm, showing the number of true positives (TP), false positives (FP), false negatives (FN), and true negatives (TN). The columns sum up to the actual number of positives (P) and negatives (N), while the rows sum up to the predicted number of positives (P') and negatives (N'). The formulas for the true positive rate (TPR), the false positive rate (FPR), the positive predictive value (PPV), and the accuracy (ACC) are given in the text.*

first counting the number of true and false matches and match failures, using the following definitions (Fawcett 2006), which we already discussed in Section 6.3.3:

- **TP**: true positives, i.e., number of correct matches;
- **FN**: false negatives, matches that were not correctly detected;
- **FP**: false positives, proposed matches that are incorrect;
- **TN**: true negatives, non-matches that were correctly rejected.

Table 7.1 shows a sample *confusion matrix* (contingency table) containing such numbers.

We can convert these numbers into *unit rates* by defining the following quantities (Fawcett 2006):

- true positive rate (TPR),

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{\text{TP}}{P}; \quad (7.14)$$

- false positive rate (FPR),

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = \frac{\text{FP}}{N}; \quad (7.15)$$

- positive predictive value (PPV),

$$\text{PPV} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{\text{TP}}{P'}; \quad (7.16)$$

- accuracy (ACC),

$$\text{ACC} = \frac{\text{TP} + \text{TN}}{P + N}. \quad (7.17)$$

In the *information retrieval* (or document retrieval) literature (Baeza-Yates and Ribeiro-Neto 1999; Manning, Raghavan, and Schütze 2008), the term *precision* (how many returned documents are relevant) is used instead of PPV and *recall* (what fraction of relevant documents was found) is used instead of TPR (see also Section 6.3.3). The precision and recall can be combined into a single measure called the *F-score*, which is their harmonic mean. This single measure is often used to rank vision algorithms (Knapitsch, Park *et al.* 2017).

Any particular matching strategy (at a particular threshold or parameter setting) can be rated by the TPR and FPR numbers; ideally, the true positive rate will be close to 1 and the false positive rate close to 0. As we vary the matching threshold, we obtain a family of such points, which are collectively known as the *receiver operating characteristic (ROC) curve* (Fawcett 2006) (Figure 7.22a). The closer this curve lies to the upper left corner, i.e., the larger the area under the curve (AUC), the better its performance. Figure 7.22b shows how we can plot the number of matches and non-matches as a function of inter-feature distance d . These curves can then be used to plot an ROC curve (Exercise 7.3). The ROC curve can also be used to calculate the *mean average precision*, which is the average precision (PPV) as you vary the threshold to select the best results, then the two top results, etc. (see Section 6.3.3 and Figure 6.27).

The problem with using a fixed threshold is that it is difficult to set; the useful range of thresholds can vary a lot as we move to different parts of the feature space (Lowe 2004; Mikolajczyk and Schmid 2005). A better strategy in such cases is to simply match the *nearest neighbor* in feature space. Since some features may have no matches (e.g., they may be part of background clutter in object recognition or they may be occluded in the other image), a threshold is still used to reduce the number of false positives.

Ideally, this threshold itself will adapt to different regions of the feature space. If sufficient training data is available (Hua, Brown, and Winder 2007), it is sometimes possible to learn

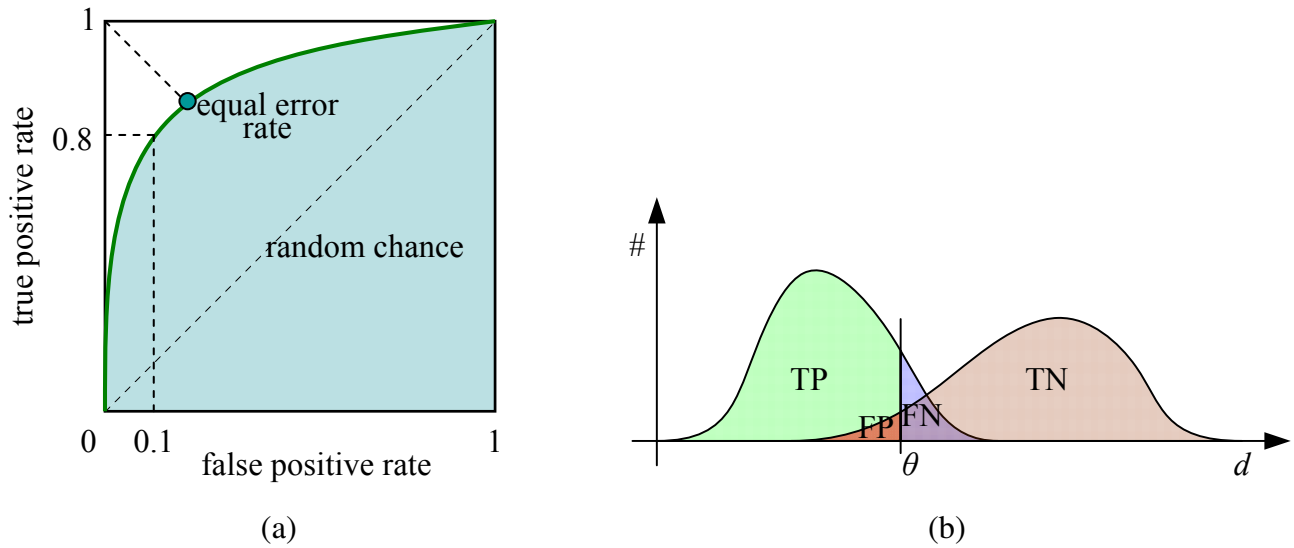


Figure 7.22 *ROC curve and its related rates: (a) The ROC curve plots the true positive rate against the false positive rate for a particular combination of feature extraction and matching algorithms. Ideally, the true positive rate should be close to 1, while the false positive rate is close to 0. The area under the ROC curve (AUC) is often used as a single (scalar) measure of algorithm performance. Alternatively, the equal error rate is sometimes used. (b) The distribution of positives (matches) and negatives (non-matches) as a function of inter-feature distance d . As the threshold θ is increased, the number of true positives (TP) and false positives (FP) increases.*

different thresholds for different features. Often, however, we are simply given a collection of images to match, e.g., when stitching images or constructing 3D models from unordered photo collections (Brown and Lowe 2007, 2005; Snavely, Seitz, and Szeliski 2006). In this case, a useful heuristic can be to compare the nearest neighbor distance to that of the second nearest neighbor, preferably taken from an image that is known not to match the target (e.g., a different object in the database) (Brown and Lowe 2002; Lowe 2004; Mishkin, Matas, and Perdoch 2015). We can define this *nearest neighbor distance ratio* (Mikolajczyk and Schmid 2005) as

$$\text{NNDR} = \frac{d_1}{d_2} = \frac{\|D_A - D_B\|}{\|D_A - D_C\|}, \quad (7.18)$$

where d_1 and d_2 are the nearest and second nearest neighbor distances, D_A is the target descriptor, and D_B and D_C are its closest two neighbors (Figure 7.23). Recent work has shown that mutual NNDR (or, at least NNDR with cross-consistency check) work noticeably better than one-way NNDR (Bellavia and Colombo 2020; Jin, Mishkin *et al.* 2021).

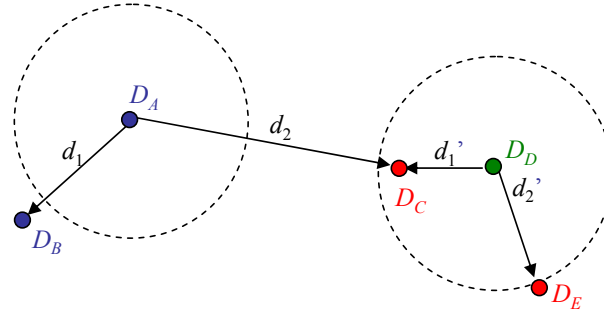


Figure 7.23 *Fixed threshold, nearest neighbor, and nearest neighbor distance ratio matching. At a fixed distance threshold (dashed circles), descriptor D_A fails to match D_B and D_D incorrectly matches D_C and D_E . If we pick the nearest neighbor, D_A correctly matches D_B but D_D incorrectly matches D_C . Using nearest neighbor distance ratio (NNDR) matching, the small NNDR d_1/d_2 correctly matches D_A with D_B , and the large NNDR d'_1/d'_2 correctly rejects matches for D_D .*

Efficient matching

Once we have decided on a matching strategy, we still need to efficiently search for potential candidates. The simplest way to find all corresponding feature points is to compare all features against all other features in each pair of potentially matching images. While traditionally this has been too computationally expensive, modern GPUs have enabled such comparisons.

A more efficient approach is to devise an *indexing structure*, such as a multi-dimensional search tree or a hash table, to rapidly search for features near a given feature. Such indexing structures can either be built for each image independently (which is useful if we want to only consider certain potential matches, e.g., searching for a particular object) or globally for all the images in a given database, which can potentially be faster, since it removes the need to iterate over each image. For extremely large databases (millions of images or more), even more efficient structures based on ideas from document retrieval, e.g., *vocabulary trees* (Nistér and Stewénius 2006), *product quantization* (Jégou, Douze, and Schmid 2010; Johnson, Douze, and Jégou 2021), or an *inverted multi-index* (Babenko and Lempitsky 2015b) can be used, as discussed in Section 7.1.4.

One of the simpler techniques to implement is multi-dimensional hashing, which maps descriptors into fixed size buckets based on some function applied to each descriptor vector. At matching time, each new feature is hashed into a bucket, and a search of nearby buckets is used to return potential candidates, which can then be sorted or graded to determine which are valid matches.

A simple example of hashing is the Haar wavelets used by Brown, Szeliski, and Winder

(2005) in their MOPS paper. During the matching structure construction, each 8×8 scaled, oriented, and normalized MOPS patch is converted into a three-element index by performing sums over different quadrants of the patch. The resulting three values are normalized by their expected standard deviations and then mapped to the two (of $b = 10$) nearest 1D bins. The three-dimensional indices formed by concatenating the three quantized values are used to index the $2^3 = 8$ bins where the feature is stored (added). At query time, only the primary (closest) indices are used, so only a single three-dimensional bin needs to be examined. The coefficients in the bin can then be used to select k approximate nearest neighbors for further processing (such as computing the NNDR).

A more complex, but more widely applicable, version of hashing is called *locality sensitive hashing*, which uses unions of independently computed hashing functions to index the features (Gionis, Indyk, and Motwani 1999; Shakhnarovich, Darrell, and Indyk 2006). Shakhnarovich, Viola, and Darrell (2003) extend this technique to be more sensitive to the distribution of points in parameter space, which they call *parameter-sensitive hashing*. More recent work converts high-dimensional descriptor vectors into binary codes that can be compared using Hamming distances (Torralba, Weiss, and Fergus 2008; Weiss, Torralba, and Fergus 2008) or that can accommodate arbitrary kernel functions (Kulis and Grauman 2009; Raginsky and Lazebnik 2009).

Another widely used class of indexing structures are multi-dimensional search trees. The best known of these are *k-d trees*, also often written as *kd-trees*, which divide the multi-dimensional feature space along alternating axis-aligned hyperplanes, choosing the threshold along each axis so as to maximize some criterion, such as the search tree balance (Samet 1989). Figure 7.24 shows an example of a two-dimensional k-d tree. Here, eight different data points A–H are shown as small diamonds arranged on a two-dimensional plane. The k-d tree recursively splits this plane along axis-aligned (horizontal or vertical) cutting planes. Each split can be denoted using the dimension number and split value (Figure 7.24b). The splits are arranged so as to try to balance the tree, i.e., to keep its maximum depth as small as possible. At query time, a classic k-d tree search first locates the query point (+) in its appropriate bin (D), and then searches nearby leaves in the tree (C, B, ...) until it can guarantee that the nearest neighbor has been found. The best bin first (BBF) search (Beis and Lowe 1999) searches bins in order of their spatial proximity to the query point and is therefore usually more efficient.

Many additional data structures have been developed for solving exact and approximate nearest neighbor problems (Arya, Mount *et al.* 1998; Liang, Liu *et al.* 2001; Hjaltason and Samet 2003). For example, Nene and Nayar (1997) developed a technique they call *slicing* that uses a series of 1D binary searches on the point list sorted along different dimensions

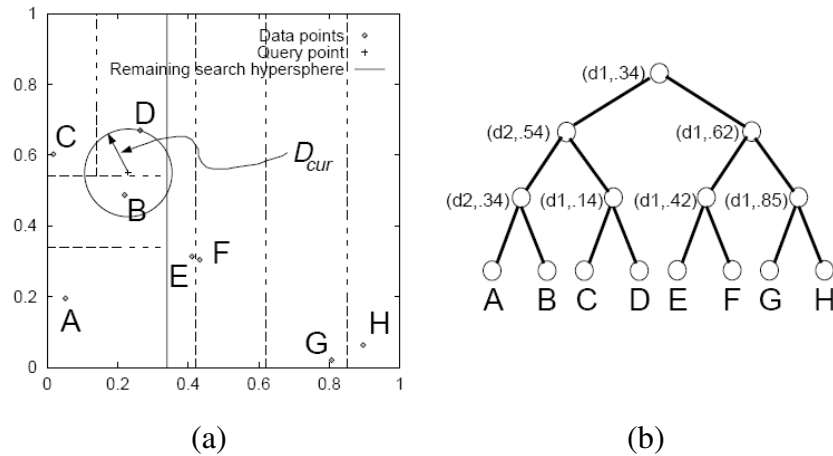


Figure 7.24 *K-d tree and best bin first (BBF) search (Beis and Lowe 1999) © 1999 IEEE:* (a) The spatial arrangement of the axis-aligned cutting planes is shown using dashed lines. Individual data points are shown as small diamonds. (b) The same subdivision can be represented as a tree, where each interior node represents an axis-aligned cutting plane (e.g., the top node cuts along dimension $d1$ at value $.34$) and each leaf node is a data point. During a BBF search, a query point (denoted by “+”) first looks in its containing bin (D) and then in its nearest adjacent bin (B), rather than its closest neighbor in the tree (C).

to efficiently cull down a list of candidate points that lie within a hypercube of the query point. Grauman and Darrell (2005) reweight the matches at different levels of an indexing tree, which allows their technique to be less sensitive to discretization errors in the tree construction. Nistér and Stewénius (2006) use a *metric tree*, which compares feature descriptors to a small number of prototypes at each level in a hierarchy. The resulting quantized *visual words* can then be used with classical information retrieval (document relevance) techniques to quickly winnow down a set of potential candidates from a database of millions of images (Section 7.1.4). Muja and Lowe (2009) compare a number of these approaches, introduce a new one of their own (priority search on hierarchical k-means trees), and conclude that multiple randomized k-d trees often provide the best performance. Modern libraries for computing approximate nearest neighbors include FLANN (Muja and Lowe 2014) and Faiss (Johnson, Douze, and Jégou 2021), which are discussed in Section 5.1.1 and Appendix C.2.

Feature match verification and densification

Once we have some candidate matches, we can use geometric alignment (Section 8.1) to verify which matches are *inliers* and which ones are *outliers*. For example, if we expect the whole image to be translated or rotated in the matching view, we can fit a global geometric

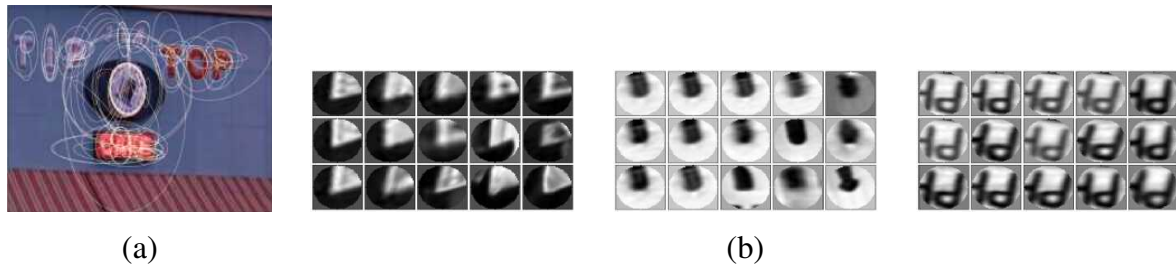


Figure 7.25 Visual words obtained from elliptical normalized affine regions (Sivic and Zisserman 2009) © 2009 IEEE. (a) Affine covariant regions are extracted from each frame and clustered into visual words using *k*-means clustering on SIFT descriptors with a learned Mahalanobis distance. (b) The central patch in each grid shows the query and the surrounding patches show the nearest neighbors.

transform and keep only those feature matches that are sufficiently close to this estimated transformation. The process of selecting a small set of seed matches and then verifying a larger set is often called *random sampling* or RANSAC (Section 8.1.4). Once an initial set of correspondences has been established, some systems look for additional matches, e.g., by looking for additional correspondences along epipolar lines (Section 12.1) or in the vicinity of estimated locations based on the global transform. It is also possible to use deep neural networks to perform feature matching and filtering, as in the SuperGlue system of Sarlin, DeTone *et al.* (2020). These topics are discussed further in Sections 8.1 and 12.2.

7.1.4 Large-scale matching and retrieval

As the number of objects in the database starts to grow (say, billions of objects or video frames), the time it takes to match a new image against each database image can become prohibitive. Instead of comparing the images one at a time, techniques are needed to quickly narrow down the search to a few likely images, which can then be compared using a more conservative verification stage.

The problem of quickly finding partial matches between documents is one of the central problems in *information retrieval* (IR) (Baeza-Yates and Ribeiro-Neto 1999; Manning, Raghavan, and Schütze 2008). In computer vision, the problem of finding a particular object in a large collection is called *content-based image retrieval* (CBIR) (Smeulders, Worring *et al.* 2000; Lew, Sebe *et al.* 2006; Vasconcelos 2007; Datta, Joshi *et al.* 2008) or *instance retrieval* (Zheng, Yang, and Tian 2018). The basic approach in fast document retrieval algorithms is to precompute an *inverted index* between individual words and the documents (or web pages or news stories) where they occur. More precisely, the *frequency* of occurrence of particular

words in a document is used to quickly find documents that match a particular query.

Sivic and Zisserman (2009) were the first to adapt IR techniques to visual search. In their Video Google system, affine invariant features are first detected in all the video frames they are indexing using both *shape adapted* regions around Harris feature points (Schaffalitzky and Zisserman 2002; Mikolajczyk and Schmid 2004) and maximally stable extremal regions (Matas, Chum *et al.* 2004; Section 7.1.1), as shown in Figure 7.25a. Next, 128-dimensional SIFT descriptors are computed from each normalized region (i.e., the patches shown in Figure 7.25b). Then, an average covariance matrix for these descriptors is estimated by accumulating statistics for features tracked from frame to frame. The feature descriptor covariance Σ is then used to define a Mahalanobis distance (5.32) between feature descriptors. In practice, feature descriptors are *whitened* by pre-multiplying them by $\Sigma^{-1/2}$ so that Euclidean distances can be used.⁵

To apply fast information retrieval techniques to images, the high-dimensional feature descriptors that occur in each image must first be mapped into discrete *visual words*. Sivic and Zisserman (2003) perform this mapping using k-means clustering, while some of the later methods (Nistér and Stewénus 2006; Philbin, Chum *et al.* 2007) use alternative techniques, such as vocabulary trees or randomized forests. To keep the clustering time manageable, only a few hundred video frames are used to learn the cluster centers, which still involves estimating several thousand clusters from about 300,000 descriptors, although subsequent work has greatly extended this capacity (Nistér and Stewénus 2006; Philbin, Chum *et al.* 2007; Mikulik, Perdoch *et al.* 2013). At visual query time, each feature in a new query region (e.g., Figure 7.25a, which is a cropped region from a larger video frame) is mapped to its corresponding visual word. To keep very common patterns from contaminating the results, a *stop list* of the most common visual words is created and such words are dropped from further consideration.

Once a query image or region has been mapped into its constituent visual words, likely matching images must then be retrieved from the database. The exact details of how this is done can be found in Sivic and Zisserman (2009), Nistér and Stewénus (2006), Philbin, Chum *et al.* (2007), Chum, Philbin *et al.* (2007), Philbin, Chum *et al.* (2008), and also in the first edition of this book (Szeliski 2010, Section 14.3.2). Because of the high efficiency in both quantizing and scoring features, the vocabulary-tree-based recognition system built by Nistér and Stewénus (2006) was able to process incoming images in real time against a database of 40,000 CD covers and at 1Hz when matching a database of one million frames

⁵Note that the computation of feature covariances from matched feature points is much more sensible than simply performing a PCA on the descriptor space (Winder and Brown 2007). This corresponds roughly to the *within-class* scatter matrix we studied in Section 5.2.3.

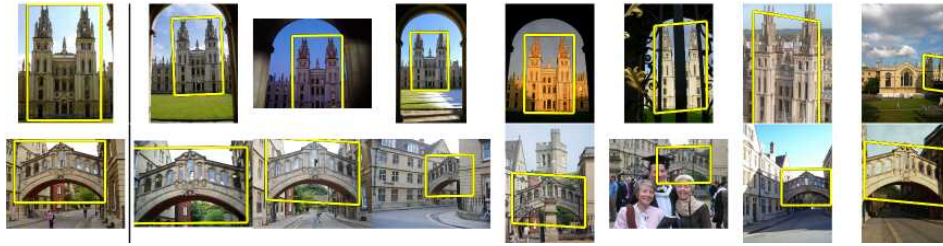


Figure 7.26 *Location or building recognition using randomized trees (Philbin, Chum et al. 2007) © 2007 IEEE. The left image is the query, the other images are the highest-ranked results.*

taken from six feature-length movies.

Instance recognition systems continued to improve rapidly in the 2000s. Philbin, Chum *et al.* (2007) showed that randomized forest of k-d trees perform better than vocabulary trees on a large location recognition task (Figure 7.26). They also compared the effects of using different 2D motion models (Section 2.1.1) in the verification stage. In follow-on work, Chum, Philbin *et al.* (2007) applied another idea from information retrieval, namely *query expansion*, which involves re-submitting top-ranked images from the initial query as additional queries to generate additional candidate results.⁶ Philbin, Chum *et al.* (2008) showed how to mitigate quantization problems in visual words selection using *soft assignment*, where each feature descriptor is mapped to a number of nearby visual words, which is similar to the multiple assignment idea proposed earlier by Jégou, Harzallah, and Schmid (2007). However, such techniques tend to reduce the sparsity of visual word vectors and increase the memory and computation costs. Jégou, Douze, and Schmid (2008) incorporated partial geometrical information and an explicit matching scheme between local descriptors in the initial large-scale image ranking stage. Taken together, these algorithms helped instance recognition algorithms perform Web-scale retrieval, matching, 3D reconstruction tasks (Agarwal, Furukawa *et al.* 2010, 2011; Frahm, Fite-Georgel *et al.* 2010; Snavely, Simon *et al.* 2010).

Since the “deep learning revolution” in 2012, researchers have started developing neural feature detectors and descriptors (Sections 7.1.1 and 7.1.2) and sometimes combining them into end-to-end matching systems.⁷ Figure 7.27 shows some of the major milestones in instance retrieval, while Figure 7.28 shows the variety of different classic and CNN-based retrieval architectures that have been considered. The survey paper by Zheng, Yang, and Tian (2018) describes and contrasts these various algorithms in more detail and also provides an

⁶An alternative to query expansion is *database-side augmentation* (Arandjelović and Zisserman 2012).

⁷But note that some popular open-source large-scale reconstruction systems such as COLMAP still use traditional features and indexing schemes (Schönberger and Frahm 2016).

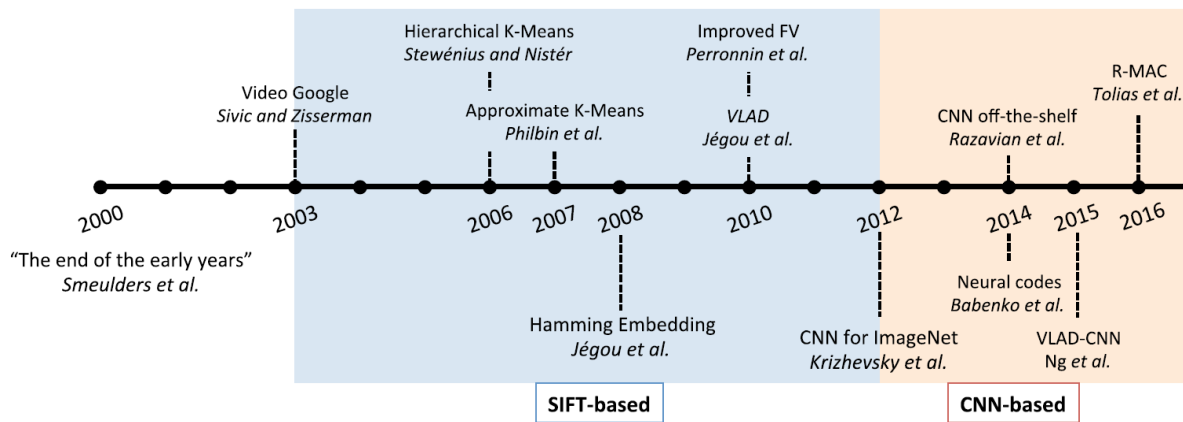


Figure 7.27 Milestones in instance retrieval (Zheng, Yang, and Tian 2018) © 2018 IEEE, showing the shift from hand-crafted feature-based retrieval to CNN-based approaches.

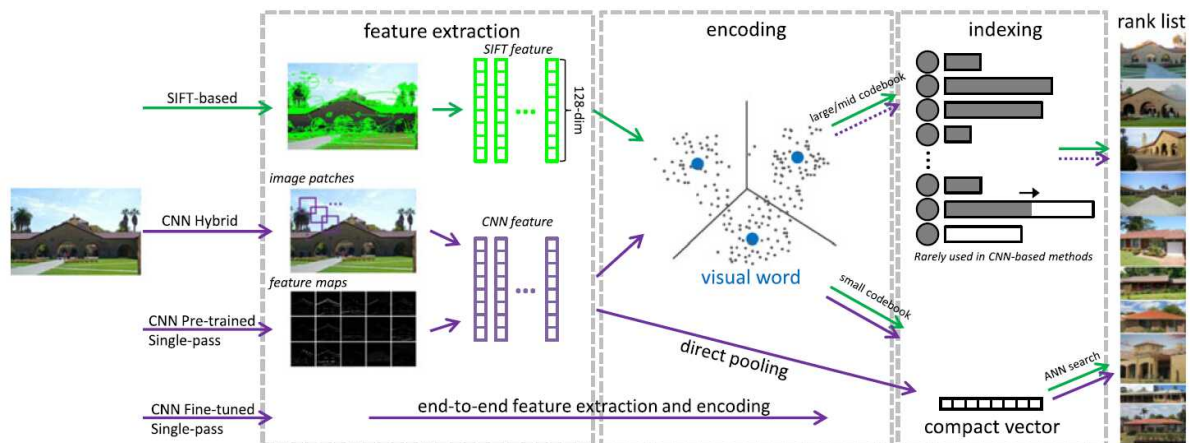


Figure 7.28 Typical pipeline for feature-based instance retrieval (Zheng, Yang, and Tian 2018) © 2018 IEEE, showing the feature extraction, encoding, and indexing portions, which are often collapsed when using a deep learning framework.

experimental comparison of some of these algorithms on image retrieval datasets. You can also find more details on related techniques and systems in Section 6.2.3 on visual similarity search, which discusses global descriptors that represent an image with a single vector (Arandjelovic, Gronat *et al.* 2016; Radenović, Tolias, and Chum 2019; Yang, Kien Nguyen *et al.* 2019; Cao, Araujo, and Sim 2020; Ng, Balntas *et al.* 2020; Tolias, Jenicek, and Chum 2020) as alternatives to bags of local features, Section 11.2.3 on location recognition, and Section 11.4.6 on large-scale 3D reconstruction from community (internet) photos.

7.1.5 Feature tracking

An alternative to independently finding features in all candidate images and then matching them is to find a set of likely feature locations in a first image and to then *search* for their corresponding locations in subsequent images. This kind of *detect then track* approach is more widely used for video tracking applications, where the expected amount of motion and appearance deformation between adjacent frames is expected to be small.

The process of selecting good features to track is closely related to selecting good features for more general recognition applications. In practice, regions containing high gradients in both directions, i.e., which have high eigenvalues in the auto-correlation matrix (7.8), provide stable locations at which to find correspondences (Shi and Tomasi 1994).

In subsequent frames, searching for locations where the corresponding patch has low squared difference (7.1) often works well enough. However, if the images are undergoing brightness change, explicitly compensating for such variations (9.9) or using *normalized cross-correlation* (9.11) may be preferable. If the search range is large, it is also often more efficient to use a *hierarchical* search strategy, which uses matches in lower-resolution images to provide better initial guesses and hence speed up the search (Section 9.1.1). Alternatives to this strategy involve learning what the appearance of the patch being tracked should be and then searching for it in the vicinity of its predicted position (Avidan 2001; Jurie and Dhome 2002; Williams, Blake, and Cipolla 2003). These topics are all covered in more detail in Section 9.1.3.

If features are being tracked over longer image sequences, their appearance can undergo larger changes. You then have to decide whether to continue matching against the originally detected patch (feature) or to re-sample each subsequent frame at the matching location. The former strategy is prone to failure, as the original patch can undergo appearance changes such as foreshortening. The latter runs the risk of the feature drifting from its original location to some other location in the image (Shi and Tomasi 1994). (Mathematically, small misregistration errors compound to create a *Markov random walk*, which leads to larger drift over time.)



Figure 7.29 *Feature tracking using an affine motion model (Shi and Tomasi 1994) © 1994 IEEE, Top row: image patch around the tracked feature location. Bottom row: image patch after warping back toward the first frame using an affine deformation. Even though the speed sign gets larger from frame to frame, the affine transformation maintains a good resemblance between the original and subsequent tracked frames.*

A preferable solution is to compare the original patch to later image locations using an *affine* motion model (Section 9.2). Shi and Tomasi (1994) first compare patches in neighboring frames using a translational model and then use the location estimates produced by this step to initialize an affine registration between the patch in the current frame and the base frame where a feature was first detected (Figure 7.29). In their system, features are only detected infrequently, i.e., only in regions where tracking has failed. In the usual case, an area around the current *predicted* location of the feature is searched with an incremental registration algorithm (Section 9.1.3). The resulting tracker is often called the Kanade–Lucas–Tomasi (KLT) tracker.

Since their original work on feature tracking, Shi and Tomasi’s approach has generated a plethora of follow-on papers and applications. Beardsley, Torr, and Zisserman (1996) use extended feature tracking combined with structure from motion (Chapter 11) to incrementally build up sparse 3D models from video sequences. Kang, Szeliski, and Shum (1997) tie together the corners of adjacent (regularly gridded) patches to provide some additional stability to the tracking, at the cost of poorer handling of occlusions. Tommasini, Fusiello *et al.* (1998) provide a better spurious match rejection criterion for the basic Shi and Tomasi algorithm, Collins and Liu (2003) provide improved mechanisms for feature selection and dealing with larger appearance changes over time, and Shafique and Shah (2005) develop algorithms for feature matching (data association) for videos with large numbers of moving objects or points. Lepetit and Fua (2005) and Yilmaz, Javed, and Shah (2006) survey the larger field of object tracking, which includes not only feature-based techniques but also alternative techniques based on contour and region (Section 7.3).



Figure 7.30 *Real-time head tracking using fast trained classifiers (Lepetit, Pilet, and Fua 2004) © 2004 IEEE.*

A more recent development in feature tracking is the use of learning algorithms to build special-purpose recognizers to rapidly search for matching features anywhere in an image (Lepetit, Pilet, and Fua 2006; Hinterstoisser, Benhimane *et al.* 2008; Rogez, Rihan *et al.* 2008; Özuysal, Calonder *et al.* 2010). By taking the time to train classifiers on sample patches and their affine deformations, extremely fast and reliable feature detectors can be constructed, which enables much faster motions to be supported (Figure 7.30). Coupling such features to deformable models (Pilet, Lepetit, and Fua 2008) or structure-from-motion algorithms (Klein and Murray 2008) can result in even higher stability.

While feature-based tracking is still widely used in real-time applications such as SLAM, autonomous navigation, and augmented reality (Section 11.5), a lot of current work on tracking is focused on whole *object tracking* (Chellappa, Sankaranarayanan *et al.* 2010; Smeulders, Chu *et al.* 2014), which we study in more detail in Section 9.4.4.

7.1.6 Application: Performance-driven animation

One of the most compelling applications of fast feature tracking is *performance-driven animation*, i.e., the interactive deformation of a 3D graphics model based on tracking a user's motions (Williams 1990; Litwinowicz and Williams 1994; Lepetit, Pilet, and Fua 2004).

Buck, Finkelstein *et al.* (2000) present a system that tracks a user's facial expressions and head motions and then uses them to morph among a series of hand-drawn sketches. An animator first extracts the eye and mouth regions of each sketch and draws control lines over each image (Figure 7.31a). At run time, a face-tracking system (Toyama 1998) determines the current location of these features (Figure 7.31b). The animation system decides which

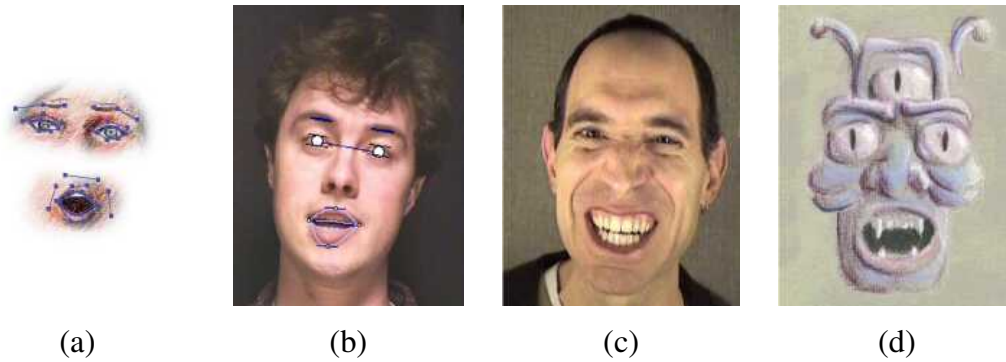


Figure 7.31 *Performance-driven, hand-drawn animation (Buck, Finkelstein et al. 2000)*
 © 2000 ACM: (a) eye and mouth portions of hand-drawn sketch with their overlaid control lines; (b) an input video frame with the tracked features overlaid; (c) a different input video frame along with its (d) corresponding hand-drawn animation.

input images to morph based on nearest neighbor feature appearance matching and triangular barycentric interpolation. It also computes the global location and orientation of the head from the tracked features. The resulting morphed eye and mouth regions are then composited back into the overall head model to yield a frame of hand-drawn animation (Figure 7.31d).

In more recent work, Barnes, Jacobs *et al.* (2008) watch users animate paper cutouts on a desk and then turn the resulting motions and drawings into seamless 2D animations. Feature-based facial trackers continue to be widely used (Zollhöfer, Thies *et al.* 2018), both in the visual effects industry, as well as for real-time smartphone augmented reality effects such as Facebook’s Spark AR Face Masks.

7.2 Edges and contours

While interest points are useful for finding image locations that can be accurately matched in 2D, edge points are far more plentiful and often carry important semantic associations. For example, the boundaries of objects, which also correspond to occlusion events in 3D, are usually delineated by visible contours. Other kinds of edges correspond to shadow boundaries or crease edges, where surface orientation changes rapidly. Isolated edge points can also be grouped into longer *curves* or *contours*, as well as *straight line segments* (Section 7.4). It is interesting that even young children have no difficulty in recognizing familiar objects or animals from such simple line drawings.

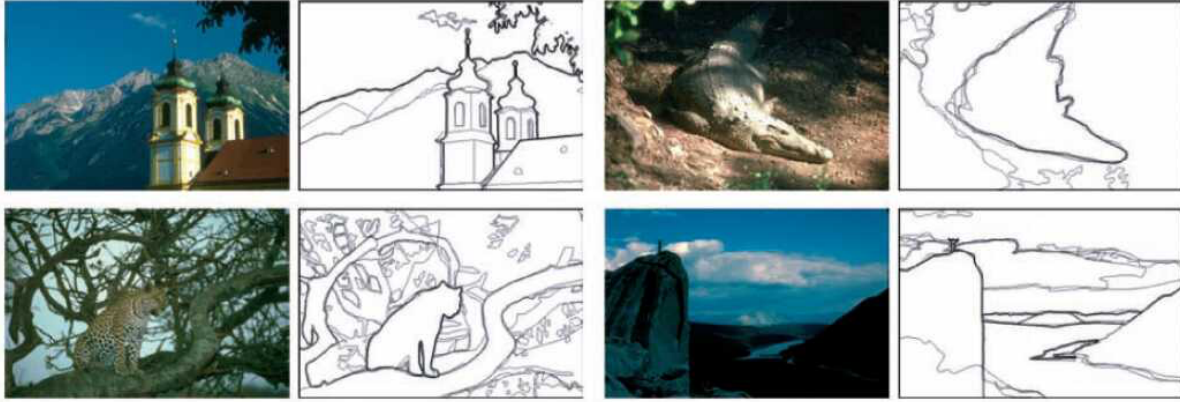


Figure 7.32 Human boundary detection (Martin, Fowlkes, and Malik 2004) © 2004 IEEE. The darkness of the edges corresponds to how many human subjects marked an object boundary at that location.

7.2.1 Edge detection

Given an image, how can we find the salient edges? Consider the color images in Figure 7.32. If someone asked you to point out the most “salient” or “strongest” edges or the object boundaries, which ones would you trace? How closely do your perceptions match the edge images shown in Figure 7.32?

Qualitatively, edges occur at boundaries between regions of different color, intensity, or texture (Martin, Fowlkes, and Malik 2004; Arbeláez, Maire *et al.* 2011; Pont-Tuset, Arbeláez *et al.* 2017). Unfortunately, segmenting an image into coherent regions is a difficult task, which we address in Section 7.5. Often, it is preferable to detect edges using only purely local information.

Under such conditions, a reasonable approach is to define an edge as a location of *rapid intensity or color variation*. Think of an image as a height field. On such a surface, edges occur at locations of *steep slopes*, or equivalently, in regions of closely packed contour lines (on a topographic map).

A mathematical way to define the slope and direction of a surface is through its gradient,

$$\mathbf{J}(\mathbf{x}) = \nabla I(\mathbf{x}) = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right) (\mathbf{x}). \quad (7.19)$$

The local gradient vector $\mathbf{J}$ points in the direction of *steepest ascent* in the intensity function. Its magnitude is an indication of the slope or strength of the variation, while its orientation points in a direction *perpendicular* to the local contour.

Unfortunately, taking image derivatives accentuates high frequencies and hence amplifies noise, as the proportion of noise to signal is larger at high frequencies. It is therefore prudent

to smooth the image with a low-pass filter prior to computing the gradient. Because we would like the response of our edge detector to be independent of orientation, a circularly symmetric smoothing filter is desirable. As we saw in Section 3.2, the Gaussian is the only separable circularly symmetric filter, so it is used in most edge detection algorithms. Canny (1986) discusses alternative filters and a number of researchers review alternative edge detection algorithms and compare their performance (Davis 1975; Nalwa and Binford 1986; Nalwa 1987; Deriche 1987; Freeman and Adelson 1991; Nalwa 1993; Heath, Sarkar *et al.* 1998; Crane 1997; Ritter and Wilson 2000; Bowyer, Kranenburg, and Dougherty 2001; Arbeláez, Maire *et al.* 2011; Pont-Tuset, Arbeláez *et al.* 2017).

Because differentiation is a linear operation, it commutes with other linear filtering operations. The gradient of the smoothed image can therefore be written as

$$\mathbf{J}_\sigma(\mathbf{x}) = \nabla[G_\sigma(\mathbf{x}) * I(\mathbf{x})] = [\nabla G_\sigma](\mathbf{x}) * I(\mathbf{x}), \quad (7.20)$$

i.e., we can convolve the image with the horizontal and vertical derivatives of the Gaussian kernel function,

$$\nabla G_\sigma(\mathbf{x}) = \left(\frac{\partial G_\sigma}{\partial x}, \frac{\partial G_\sigma}{\partial y} \right) (\mathbf{x}) = [-x \ -y] \frac{1}{\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right), \quad (7.21)$$

where the parameter σ indicates the width of the Gaussian. This is the same computation that is performed by Freeman and Adelson's (1991) first-order steerable filter, which we have already covered in Section 3.2.3.

For many applications, however, we wish to thin such a continuous gradient image to return isolated edges only, i.e., as single pixels at discrete locations along the edge contours. This can be achieved by looking for *maxima* in the edge strength (gradient magnitude) in a direction *perpendicular* to the edge orientation, i.e., along the gradient direction.

Finding this maximum corresponds to taking a directional derivative of the strength field in the direction of the gradient and then looking for zero crossings. The desired directional derivative is equivalent to the dot product between a second gradient operator and the results of the first,

$$S_\sigma(\mathbf{x}) = \nabla \cdot \mathbf{J}_\sigma(\mathbf{x}) = [\nabla^2 G_\sigma](\mathbf{x}) * I(\mathbf{x}). \quad (7.22)$$

The gradient operator dot product with the gradient is called the *Laplacian*. The convolution kernel

$$\nabla^2 G_\sigma(\mathbf{x}) = \left(\frac{x^2 + y^2}{\sigma^4} - \frac{2}{\sigma^2} \right) G_\sigma(\mathbf{x}), \quad (7.23)$$

is therefore called the *Laplacian of Gaussian* (LoG) kernel (Marr and Hildreth 1980). This kernel can be split into two separable parts,

$$\nabla^2 G_\sigma(\mathbf{x}) = \left(\frac{x^2}{2\sigma^4} - \frac{1}{\sigma^2} \right) G_\sigma(x)G_\sigma(y) + \left(\frac{y^2}{2\sigma^4} - \frac{1}{\sigma^2} \right) G_\sigma(y)G_\sigma(x) \quad (7.24)$$

(Wiejak, Buxton, and Buxton 1985), which allows for a much more efficient implementation using separable filtering (Section 3.2.1).

In practice, it is quite common to replace the Laplacian of Gaussian convolution with a difference of Gaussian (DoG) computation, since the kernel shapes are qualitatively similar (Figure 3.34). This is especially convenient if a “Laplacian pyramid” (Section 3.5) has already been computed.⁸

In fact, it is not strictly necessary to take differences between adjacent levels when computing the edge field. Think about what a zero crossing in a “generalized” difference of Gaussians image represents. The finer (smaller kernel) Gaussian is a noise-reduced version of the original image. The coarser (larger kernel) Gaussian is an estimate of the average intensity over a larger region. Thus, whenever the DoG image changes sign, this corresponds to the (slightly blurred) image going from relatively darker to relatively lighter, as compared to the average intensity in that neighborhood.

Once we have computed the sign function $S(\mathbf{x})$, we must find its *zero crossings* and convert these into edge elements (*edgels*). An easy way to detect and represent zero crossings is to look for adjacent pixel locations $\mathbf{x}_i$ and $\mathbf{x}_j$ where the sign changes value, i.e., $[S(\mathbf{x}_i) > 0] \neq [S(\mathbf{x}_j) > 0]$.

The sub-pixel location of this crossing can be obtained by computing the “ x -intercept” of the “line” connecting $S(\mathbf{x}_i)$ and $S(\mathbf{x}_j)$,

$$\mathbf{x}_z = \frac{\mathbf{x}_i S(\mathbf{x}_j) - \mathbf{x}_j S(\mathbf{x}_i)}{S(\mathbf{x}_j) - S(\mathbf{x}_i)}. \quad (7.25)$$

The orientation and strength of such edgels can be obtained by linearly interpolating the gradient values computed on the original pixel grid.

An alternative edgel representation can be obtained by linking adjacent edgels on the dual grid to form edgels that live *inside* each square formed by four adjacent pixels in the original pixel grid.⁹ The advantage of this representation is that the edgels now live on a grid offset by half a pixel from the original pixel grid and are thus easier to store and access. As before, the orientations and strengths of the edges can be computed by interpolating the gradient field or estimating these values from the difference of Gaussian image (see Exercise 7.7).

In applications where the accuracy of the edge orientation is more important, higher-order steerable filters can be used (Freeman and Adelson 1991) (see Section 3.2.3). Such filters are more selective for more elongated edges and also have the possibility of better modeling curve

⁸Recall that Burt and Adelson’s (1983a) “Laplacian pyramid” actually computes differences of Gaussian-filtered levels.

⁹This algorithm is a 2D version of the 3D *marching cubes* isosurface extraction algorithm (Lorensen and Cline 1987).

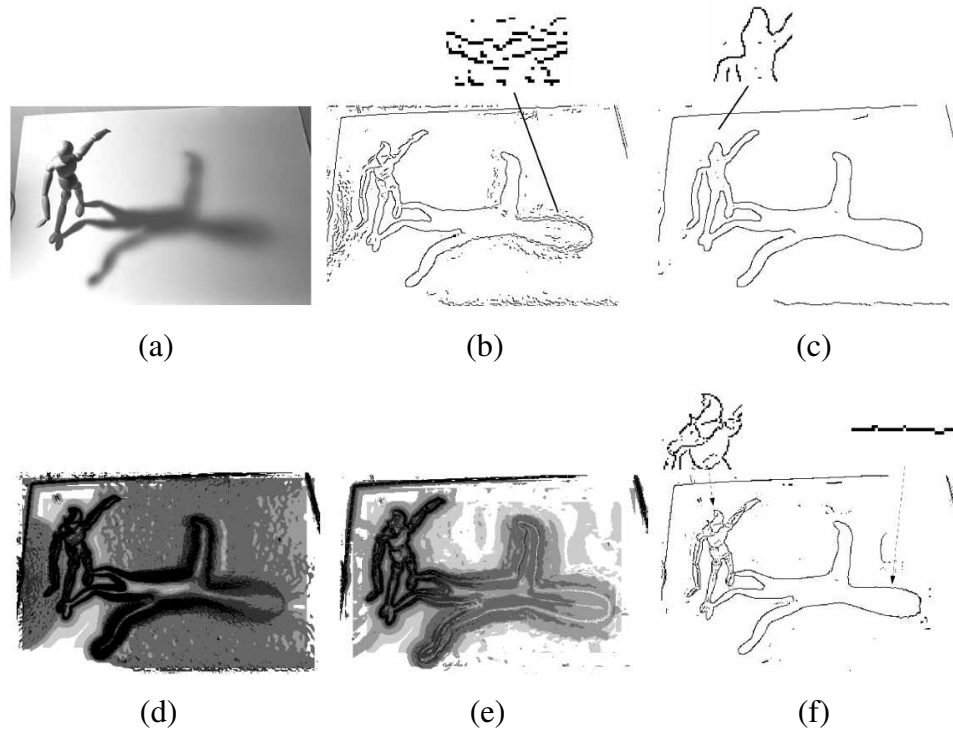


Figure 7.33 Scale selection for edge detection (*Elder and Zucker 1998*) © 1998 IEEE: (a) original image; (b–c) Canny/Deriche edge detector tuned to the finer (mannequin) and coarser (shadow) scales; (d) minimum reliable scale for gradient estimation; (e) minimum reliable scale for second derivative estimation; (f) final detected edges.

intersections because they can represent multiple orientations at the same pixel (Figure 3.16). Their disadvantage is that they are more expensive to compute and the directional derivative of the edge strength does not have a simple closed form solution.¹⁰

Scale selection and blur estimation

As we mentioned before, the derivative, Laplacian, and Difference of Gaussian filters (7.20–7.23) all require the selection of a spatial scale parameter σ . If we are only interested in detecting sharp edges, the width of the filter can be determined from image noise characteristics (Canny 1986; Elder and Zucker 1998). However, if we want to detect edges that occur at different resolutions (Figures 7.33b–c), a *scale-space* approach that detects and then selects edges at different scales may be necessary (Witkin 1983; Lindeberg 1994, 1998a; Nielsen, Florack, and Deriche 1997).

¹⁰In fact, the edge orientation can have a 180° ambiguity for “bar edges”, which makes the computation of zero crossings in the derivative more tricky.

Elder and Zucker (1998) present a principled approach to solving this problem. Given a known image noise level, their technique computes, for every pixel, the minimum scale at which an edge can be reliably detected (Figure 7.33d). Their approach first computes gradients densely over an image by selecting among gradient estimates computed at different scales, based on their gradient magnitudes. It then performs a similar estimate of minimum scale for directed second derivatives and uses zero crossings of this latter quantity to robustly select edges (Figures 7.33e–f). As an optional final step, the blur width of each edge can be computed from the distance between extrema in the second derivative response minus the width of the Gaussian filter.

Color edge detection

While most edge detection techniques have been developed for grayscale images, color images can provide additional information. For example, noticeable edges between *iso-luminant* colors (colors that have the same luminance) are useful cues but fail to be detected by grayscale edge operators.

One simple approach is to combine the outputs of grayscale detectors run on each color band separately.¹¹ However, some care must be taken. For example, if we simply sum up the gradients in each of the color bands, the signed gradients may actually cancel each other! (Consider, for example a pure red-to-green edge.) We could also detect edges independently in each band and then take the union of these, but this might lead to thickened or doubled edges that are hard to link.

A better approach is to compute the *oriented energy* in each band (Morrone and Burr 1988; Perona and Malik 1990a), e.g., using a second-order steerable filter (Section 3.2.3) (Freeman and Adelson 1991), and then sum up the orientation-weighted energies and find their joint best orientation. Unfortunately, the directional derivative of this energy may not have a closed form solution (as in the case of signed first-order steerable filters), so a simple zero crossing-based strategy cannot be used. However, the technique described by Elder and Zucker (1998) can be used to compute these zero crossings numerically instead.

An alternative approach is to estimate local color statistics in regions around each pixel (Ruzon and Tomasi 2001; Martin, Fowlkes, and Malik 2004). This has the advantage that more sophisticated techniques (e.g., 3D color histograms) can be used to compare regional statistics and that additional measures, such as texture, can also be considered. Figure 7.34 shows the output of such detectors.

¹¹ Instead of using the raw RGB space, a more perceptually uniform color space such as L*a*b* (see Section 2.3.2) can be used instead. When trying to match human performance (Martin, Fowlkes, and Malik 2004), this makes sense. However, in terms of the physics of the underlying image formation and sensing, it may be a questionable strategy.

Over the years, many other approaches have been developed for detecting color edges, dating back to early work by Nevatia (1977). Ruzon and Tomasi (2001) and Gevers, van de Weijer, and Stokman (2006) provide good reviews of these approaches, which include ideas such as fusing outputs from multiple channels, using multidimensional gradients, and vector-based methods.

Combining edge feature cues

If the goal of edge detection is to match human *boundary detection* performance (Bowyer, Kranenburg, and Dougherty 2001; Martin, Fowlkes, and Malik 2004; Arbeláez, Maire *et al.* 2011; Pont-Tuset, Arbeláez *et al.* 2017), as opposed to simply finding stable features for matching, even better detectors can be constructed by combining multiple low-level cues such as brightness, color, and texture.

Martin, Fowlkes, and Malik (2004) describe a system that combines brightness, color, and texture edges to produce state-of-the-art performance on a database of hand-segmented natural color images (Martin, Fowlkes *et al.* 2001). First, they construct and train separate oriented half-disc detectors for measuring significant differences in brightness (luminance), color (a^* and b^* channels, summed responses), and texture (un-normalized filter bank responses from the work of Malik, Belongie *et al.* (2001)). Some of the responses are then sharpened using a soft non-maximal suppression technique. Finally, the outputs of the three detectors are combined using a variety of machine-learning techniques, from which logistic regression is found to have the best tradeoff between speed, space, and accuracy. The resulting system (see Figure 7.34 for some examples) is shown to outperform previously developed techniques. Maire, Arbelaez *et al.* (2008) improve on these results by combining the detector based on local appearance with a *spectral* (segmentation-based) detector (Belongie and Malik 1998). In follow-on work, Arbeláez, Maire *et al.* (2011) build a hierarchical segmentation on top of this edge detector using a variant of the watershed algorithm.

7.2.2 Contour detection

While isolated edges can be useful for a variety of applications, such as line detection (Section 7.4) and sparse stereo matching (Section 12.2), they become even more useful when linked into continuous contours.

If the edges have been detected using zero crossings of some function, linking them up is straightforward, since adjacent edgels share common endpoints. Linking the edgels into chains involves picking up an unlinked edgel and following its neighbors in both directions. Either a sorted list of edgels (sorted first by x coordinates and then by y coordinates, for



Figure 7.34 Combined brightness, color, texture boundary detector (*Martin, Fowlkes, and Malik 2004*) © 2004 IEEE. Successive rows show the outputs of the brightness gradient (BG), color gradient (CG), texture gradient (TG), and combined (BG+CG+TG) detectors. The final row shows human-labeled boundaries derived from a database of hand-segmented images (*Martin, Fowlkes et al. 2001*).

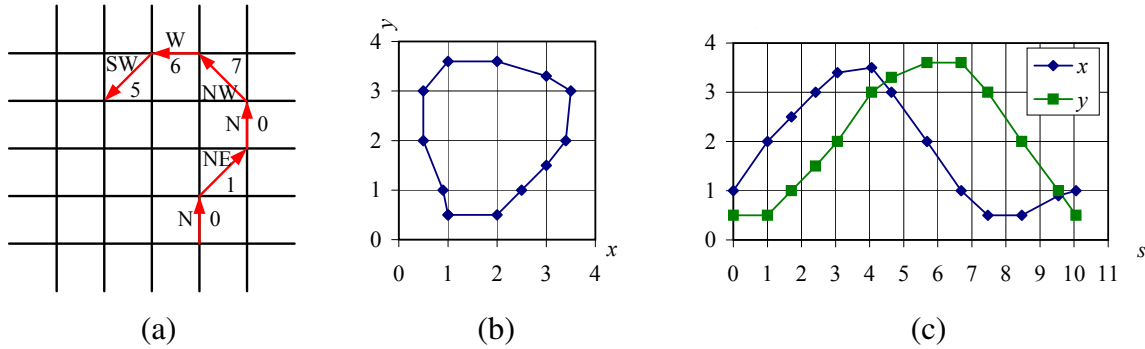


Figure 7.35 Some coding alternatives for linked contours. (a) A chain code representation of a grid-aligned linked edge chain. The code is represented as a series of direction codes, e.g., 0 1 0 7 6 5, which can further be compressed using predictive and run-length coding. (b–c) Arc-length parameterization of a contour. Discrete points along the contour (b) are first transcribed as (c) (x, y) pairs along the arc length s . This curve can then be regularly re-sampled or converted into alternative (e.g., Fourier) representations.

example) or a 2D array can be used to accelerate the neighbor finding. If edges were not detected using zero crossings, finding the continuation of an edgel can be tricky. In this case, comparing the orientation (and, optionally, phase) of adjacent edgels can be used for disambiguation. Ideas from connected component computation can also sometimes be used to make the edge linking process even faster (see Exercise 7.8).

Once the edgels have been linked into chains, we can apply an optional thresholding with hysteresis to remove low-strength contour segments (Canny 1986). The basic idea of hysteresis is to set two different thresholds and allow a curve being tracked above the higher threshold to dip in strength down to the lower threshold.

Linked edgel lists can be encoded more compactly using a variety of alternative representations. A *chain code* encodes a list of connected points lying on an $\mathcal{N}_8$ grid using a three-bit code corresponding to the eight cardinal directions (N, NE, E, SE, S, SW, W, NW) between a point and its successor (Figure 7.35a). While this representation is more compact than the original edgel list (especially if predictive variable-length coding is used), it is not very suitable for further processing.

A more useful representation is the *arc length parameterization* of a contour, $\mathbf{x}(s)$, where s denotes the arc length along a curve. Consider the linked set of edgels shown in Figure 7.35b. We start at one point (the dot at $(1.0, 0.5)$ in Figure 7.35c) and plot it at coordinate $s = 0$ (Figure 7.35c). The next point at $(2.0, 0.5)$ gets plotted at $s = 1$, and the next point at $(2.5, 1.0)$ gets plotted at $s = 1.7071$, i.e., we increment s by the length of each edge segment. The resulting plot can be resampled on a regular (say, integral) s grid before further

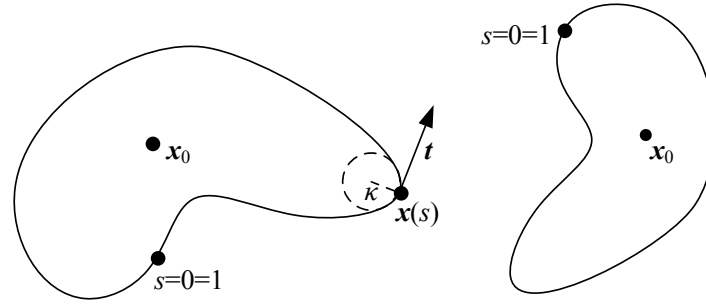


Figure 7.36 Matching two contours using their arc-length parameterization. If both curves are normalized to unit length, $s \in [0, 1]$ and centered around their centroid $\mathbf{x}_0$, they will have the same descriptor up to an overall “temporal” shift (due to different starting points for $s = 0$) and a phase (x - y) shift (due to rotation).

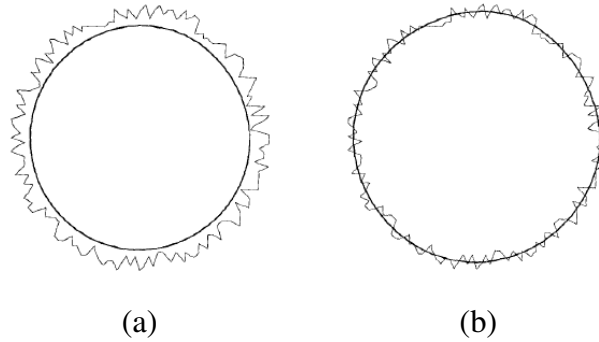


Figure 7.37 Curve smoothing with a Gaussian kernel (Lowe 1988) © 1998 IEEE: (a) without a shrinkage correction term; (b) with a shrinkage correction term.

processing.

The advantage of the arc-length parameterization is that it makes matching and processing (e.g., smoothing) operations much easier. Consider the two curves describing similar shapes shown in Figure 7.36. To compare the curves, we first subtract the average values $\mathbf{x}_0 = \int_s \mathbf{x}(s)$ from each descriptor. Next, we rescale each descriptor so that s goes from 0 to 1 instead of 0 to S , i.e., we divide $\mathbf{x}(s)$ by S . Finally, we take the Fourier transform of each normalized descriptor, treating each $\mathbf{x} = (x, y)$ value as a complex number. If the original curves are the same (up to an unknown scale and rotation), the resulting Fourier transforms should differ only by a scale change in magnitude plus a constant complex phase shift, due to rotation, and a linear phase shift in the domain, due to different starting points for s (see Exercise 7.9).

Arc-length parameterization can also be used to smooth curves to remove digitization

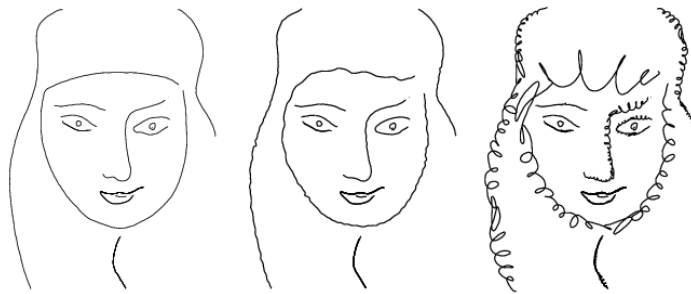


Figure 7.38 *Changing the character of a curve without affecting its sweep (Finkelstein and Salesin 1994) © 1994 ACM: higher frequency wavelets can be replaced with exemplars from a style library to effect different local appearances.*

noise. However, if we just apply a regular smoothing filter, the curve tends to shrink on itself (Figure 7.37a). Lowe (1989) and Taubin (1995) describe techniques that compensate for this shrinkage by adding an offset term based on second derivative estimates or a larger smoothing kernel (Figure 7.37b). An alternative approach, based on selectively modifying different frequencies in a wavelet decomposition, is presented by Finkelstein and Salesin (1994). In addition to controlling shrinkage without affecting its “sweep”, wavelets allow the “character” of a curve to be interactively modified, as shown in Figure 7.38.

The evolution of curves as they are smoothed and simplified is related to “grassfire” (distance) transforms and region skeletons (Section 3.3.3) (Tek and Kimia 2003), and can be used to recognize objects based on their contour shape (Sebastian and Kimia 2005). More local descriptors of curve shape such as *shape contexts* (Belongie, Malik, and Puzicha 2002) can also be used for recognition and are potentially more robust to missing parts due to occlusions.

The field of contour detection and linking continues to evolve rapidly and now includes techniques for global contour grouping, boundary completion, and junction detection (Maire, Arbelaez *et al.* 2008), as well as grouping contours into likely regions (Arbeláez, Maire *et al.* 2011) and wide-baseline correspondence (Meltzer and Soatto 2008). Some additional papers that address contour detection include Xiaofeng and Bo (2012), Lim, Zitnick, and Dollár (2013), Dollár and Zitnick (2015), Xie and Tu (2015), and Pont-Tuset, Arbeláez *et al.* (2017).

7.2.3 Application: Edge editing and enhancement

While edges can serve as components for object recognition or features for matching, they can also be used directly for image editing.

In fact, if the edge magnitude and blur estimate are kept along with each edge, a visually

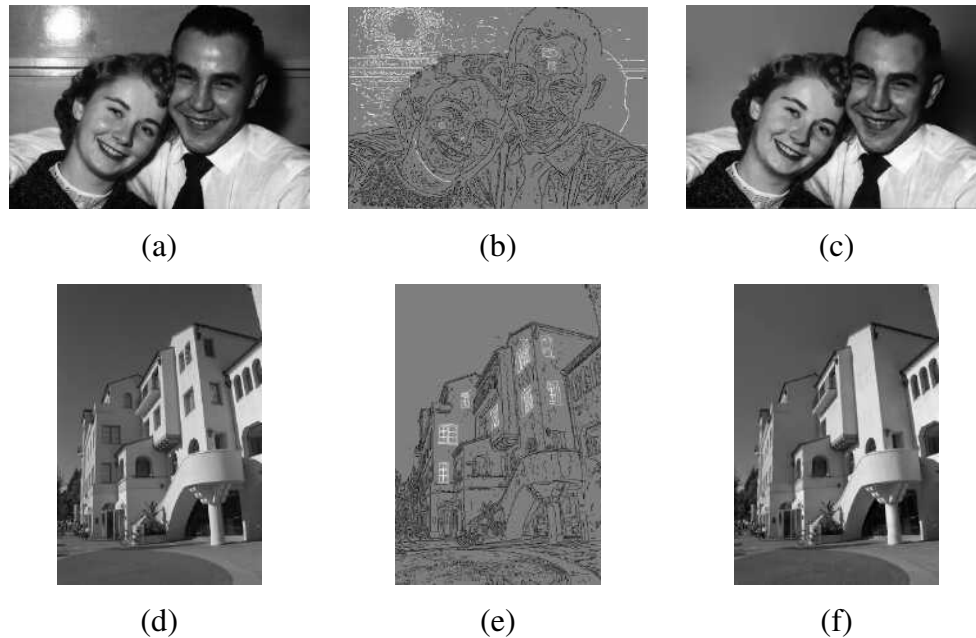


Figure 7.39 *Image editing in the contour domain (Elder and Goldberg 2001) © 2001 IEEE: (a) and (d) original images; (b) and (e) extracted edges (edges to be deleted are marked in white); (c) and (f) reconstructed edited images.*

similar image can be reconstructed from this information (Elder 1999). Based on this principle, Elder and Goldberg (2001) propose a system for “image editing in the contour domain”. Their system allows users to selectively remove edges corresponding to unwanted features such as specularities, shadows, or distracting visual elements. After reconstructing the image from the remaining edges, the undesirable visual features have been removed (Figure 7.39).

Another potential application is to enhance perceptually salient edges while simplifying the underlying image to produce a cartoon-like or “pen-and-ink” stylized image (DeCarlo and Santella 2002). This application is discussed in more detail in Section 10.5.2.

7.3 Contour tracking

While lines, vanishing points, and rectangles are commonplace in the human-made world, curves corresponding to object boundaries are even more common, especially in the natural environment. In this section, we describe some approaches to locating such boundary curves in images.

The first, originally called *snakes* by its inventors (Kass, Witkin, and Terzopoulos 1988) (Section 7.3.1), is an energy-minimizing, two-dimensional spline curve that evolves (moves)

towards image features such as strong edges. The second, *intelligent scissors* (Mortensen and Barrett 1995) (Section 7.3.1), allows the user to sketch in real time a curve that clings to object boundaries. Finally, *level set* techniques (Section 7.3.2) evolve the curve as the zero-set of a *characteristic function*, which allows them to easily change topology and incorporate region-based statistics.

All three of these are examples of *active contours* (Blake and Isard 1998; Mortensen 1999), since these boundary detectors iteratively move towards their final solution under the combination of image and optional user-guidance forces. The presentation below is heavily shortened from that presented in the first edition of this book (Szeliski 2010, Section 5.1), where interested readers can find more details.

7.3.1 Snakes and scissors

Snakes are a two-dimensional generalization of the 1D energy-minimizing splines first introduced in Section 4.2,

$$\mathcal{E}_{\text{int}} = \int \alpha(s) \|\mathbf{f}_s(s)\|^2 + \beta(s) \|\mathbf{f}_{ss}(s)\|^2 ds, \quad (7.26)$$

where s is the arc-length along the curve $\mathbf{f}(s) = (x(s), y(s))$ and $\alpha(s)$ and $\beta(s)$ are first- and second-order continuity weighting functions analogous to the $s(x, y)$ and $c(x, y)$ terms introduced in (4.24–4.25). We can discretize this energy by sampling the initial curve position evenly along its length (Figure 7.35c) to obtain

$$\begin{aligned} E_{\text{int}} = \sum_i \alpha(i) \|f(i+1) - f(i)\|^2 / h^2 \\ + \beta(i) \|f(i+1) - 2f(i) + f(i-1)\|^2 / h^4, \end{aligned} \quad (7.27)$$

where h is the step size, which can be neglected if we resample the curve along its arc-length after each iteration.

In addition to this *internal* spline energy, a snake simultaneously minimizes external image-based and constraint-based potentials. The image-based potentials are the sum of several terms

$$\mathcal{E}_{\text{image}} = w_{\text{line}} \mathcal{E}_{\text{line}} + w_{\text{edge}} \mathcal{E}_{\text{edge}} + w_{\text{term}} \mathcal{E}_{\text{term}}, \quad (7.28)$$

where the *line* term attracts the snake to dark ridges, the *edge* term attracts it to strong gradients (edges), and the *term* term attracts it to line terminations. As the snakes evolve by minimizing their energy, they often “wiggle” and “slither”, which accounts for their popular name.

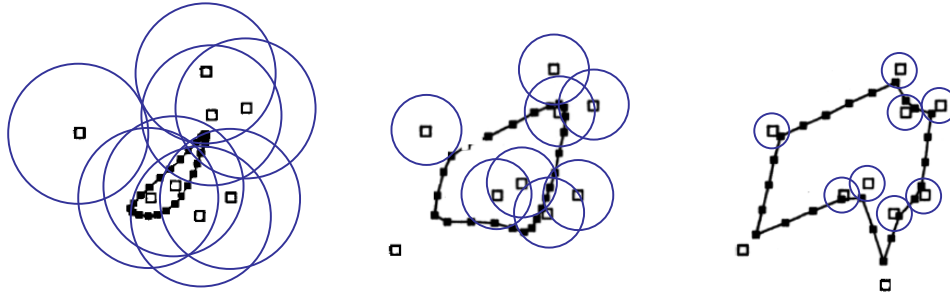


Figure 7.40 *Elastic net: The open squares indicate the cities and the closed squares linked by straight line segments are the tour points. The blue circles indicate the approximate extent of the attraction force of each city, which is reduced over time. Under the Bayesian interpretation of the elastic net, the blue circles correspond to one standard deviation of the circular Gaussian that generates each city from some unknown tour point.*

Because regular snakes have a tendency to shrink, it is usually better to initialize them by drawing the snake outside the object of interest to be tracked. Alternatively, an expansion *ballooning* force can be added to the dynamics (Cohen and Cohen 1993), essentially moving each point outwards along its normal. It is also possible to replace the energy-minimizing variational evolution equations with a deep neural network to significantly improve performance (Peng, Jiang *et al.* 2020).

Elastic nets and slippery springs

An interesting variant on snakes, first proposed by Durbin and Willshaw (1987) and later re-formulated in an energy-minimizing framework by Durbin, Szeliski, and Yuille (1989), is the *elastic net* formulation of the Traveling Salesman Problem (TSP). Recall that in a TSP, the salesman must visit each city once while minimizing the total distance traversed. A snake that is constrained to pass through each city could solve this problem (without any optimality guarantees) but it is impossible to tell ahead of time which snake control point should be associated with each city.

Instead of having a fixed constraint between snake nodes and cities, a city is assumed to pass near *some* point along the tour (Figure 7.40). In a probabilistic interpretation, each city is generated as a *mixture* of Gaussians centered at each tour point,

$$p(\mathbf{d}(j)) = \sum_i p_{ij} \quad \text{with} \quad p_{ij} = e^{-d_{ij}^2/(2\sigma^2)}, \quad (7.29)$$

where σ is the standard deviation of the Gaussian and

$$d_{ij} = \|\mathbf{f}(i) - \mathbf{d}(j)\| \quad (7.30)$$

is the Euclidean distance between a tour point $\mathbf{f}(i)$ and a city location $\mathbf{d}(j)$. The corresponding data fitting energy (negative log likelihood) is

$$E_{\text{slippery}} = - \sum_j \log p(\mathbf{d}(j)) = - \sum_j \log \left[\sum_i e^{-\|\mathbf{f}(i) - \mathbf{d}(j)\|^2 / 2\sigma^2} \right]. \quad (7.31)$$

This energy derives its name from the fact that, unlike a regular spring, which couples a given snake point to a given constraint, this alternative energy defines a *slippery spring* that allows the association between constraints (cities) and curve (tour) points to evolve over time (Szeliski 1989). Note that this is a soft variant of the popular *iterative closest point* data constraint that is often used in fitting or aligning surfaces to data points or to each other (Section 13.2.1) (Besl and McKay 1992; Chen and Medioni 1992; Zhang 1994).

To compute a good solution to the TSP, the slippery spring data association energy is combined with a regular first-order internal smoothness energy (7.27) to define the cost of a tour. The tour $\mathbf{f}(s)$ is initialized as a small circle around the mean of the city points and σ is progressively lowered (Figure 7.40). For large σ values, the tour tries to stay near the centroid of the points but as σ decreases each city pulls more and more strongly on its closest tour points (Durbin, Szeliski, and Yuille 1989). In the limit as $\sigma \rightarrow 0$, each city is guaranteed to capture at least one tour point and the tours between subsequent cities become straight lines.

Splines and shape priors

While snakes can be very good at capturing the fine and irregular detail in many real-world contours, they sometimes exhibit too many degrees of freedom, making it more likely that they can get trapped in local minima during their evolution.

One solution to this problem is to control the snake with fewer degrees of freedom through the use of B-spline approximations (Menet, Saint-Marc, and Medioni 1990b,a; Cipolla and Blake 1990). The resulting *B-snake* can be written as

$$\mathbf{f}(s) = \sum_k B_k(s) \mathbf{x}_k. \quad (7.32)$$

If the object being tracked or recognized has large variations in location, scale, or orientation, these can be modeled as an additional transformation on the control points, e.g., $\mathbf{x}'_k = s\mathbf{R}\mathbf{x}_k + \mathbf{t}$ (2.18), which can be estimated at the same time as the values of the control points. Alternatively, separate *detection* and *alignment* stages can be run to first localize and orient the objects of interest (Cootes, Cooper *et al.* 1995).

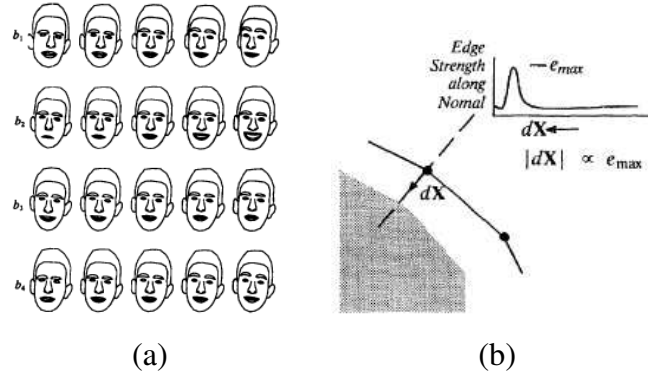


Figure 7.41 *Active Shape Model (ASM): (a) the effect of varying the first four shape parameters for a set of faces (Cootes, Taylor et al. 1993) © 1993 IEEE; (b) searching for the strongest gradient along the normal to each control point (Cootes, Cooper et al. 1995) © 1995 Elsevier.*

In a B-snake, because the snake is controlled by fewer degrees of freedom, there is less need for the internal smoothness forces used with the original snakes, although these can still be derived and implemented using finite element analysis, i.e., taking derivatives and integrals of the B-spline basis functions (Terzopoulos 1983; Bathe 2007).

In practice, it is more common to estimate a set of *shape priors* on the typical distribution of the control points $\{\mathbf{x}_k\}$ (Cootes, Cooper et al. 1995). One potential way of describing this distribution would be by the location $\bar{\mathbf{x}}_k$ and 2D covariance $\mathbf{C}_k$ of each individual point $\mathbf{x}_k$. These could then be turned into a quadratic penalty (prior energy) on the point location. In practice, however, the variation in point locations is usually highly correlated.

A preferable approach is to estimate the joint covariance of all the points simultaneously. First, concatenate all of the point locations $\{\mathbf{x}_k\}$ into a single vector $\mathbf{x}$, e.g., by interleaving the x and y locations of each point. The distribution of these vectors across all training examples can be described with a mean $\bar{\mathbf{x}}$ and a covariance

$$\mathbf{C} = \frac{1}{P} \sum_p (\mathbf{x}_p - \bar{\mathbf{x}})(\mathbf{x}_p - \bar{\mathbf{x}})^T, \quad (7.33)$$

where $\mathbf{x}_p$ are the P training examples. Using *eigenvalue analysis* (Appendix A.1.2), which is also known as *principal component analysis* (PCA) (Section 5.2.3 and Appendix B.1), the covariance matrix can be written as,

$$\mathbf{C} = \Phi \text{diag}(\lambda_0 \dots \lambda_{K-1}) \Phi^T. \quad (7.34)$$

In most cases, the likely appearance of the points can be modeled using only a few eigenvectors with the largest eigenvalues. The resulting *point distribution model* (Cootes, Taylor *et al.* 1993; Cootes, Cooper *et al.* 1995) can be written as

$$\mathbf{x} = \bar{\mathbf{x}} + \hat{\Phi} \mathbf{b}, \quad (7.35)$$

where $\mathbf{b}$ is an $M \ll K$ element *shape parameter* vector and $\hat{\Phi}$ are the first m columns of Φ . To constrain the shape parameters to reasonable values, we can use a quadratic penalty of the form

$$E_{\text{shape}} = \frac{1}{2} \mathbf{b}^T \text{diag}(\lambda_0 \dots \lambda_{M-1}) \mathbf{b} = \sum_m b_m^2 / 2\lambda_m. \quad (7.36)$$

Alternatively, the range of allowable b_m values can be limited to some range, e.g., $|b_m| \leq 3\sqrt{\lambda_m}$ (Cootes, Cooper *et al.* 1995). Alternative approaches for deriving a set of shape vectors are reviewed by Isard and Blake (1998). Varying the individual shape parameters b_m over the range $-2\sqrt{\lambda_m} \leq 2\sqrt{\lambda_m}$ can give a good indication of the expected variation in appearance, as shown in Figure 7.41a.

To align a point distribution model with an image, each control point searches in a direction normal to the contour to find the most likely corresponding image edge point (Figure 7.41b). These individual measurements can be combined with priors on the shape parameters (and, if desired, position, scale, and orientation parameters) to estimate a new set of parameters. The resulting *active shape model* (ASM) can be iteratively minimized to fit images to non-rigidly deforming objects, such as medical images, or body parts, such as hands (Cootes, Cooper *et al.* 1995). The ASM can also be combined with a PCA analysis of the underlying gray-level distribution to create an *active appearance model* (AAM) (Cootes, Edwards, and Taylor 2001), which we discussed in more detail in Section 6.2.4.

Dynamic snakes and CONDENSATION

In many applications of active contours, the object of interest is being tracked from frame to frame as it deforms and evolves. In this case, it makes sense to use estimates from the previous frame to predict and constrain the new estimates.

One way to do this is to use Kalman filtering, which results in a formulation called *Kalman snakes* (Terzopoulos and Szeliski 1992; Blake, Curwen, and Zisserman 1993). The Kalman filter is based on a linear dynamic model of shape parameter evolution,

$$\mathbf{x}_t = \mathbf{A}\mathbf{x}_{t-1} + \mathbf{w}_t, \quad (7.37)$$

where $\mathbf{x}_t$ and $\mathbf{x}_{t-1}$ are the current and previous state variables, $\mathbf{A}$ is the linear *transition matrix*, and $\mathbf{w}$ is a noise (perturbation) vector, which is often modeled as a Gaussian (Gelb

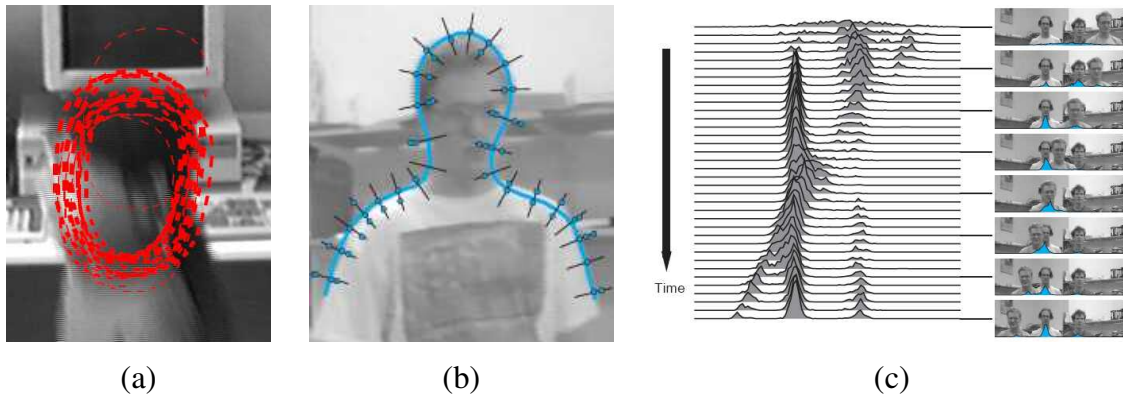


Figure 7.42 *Head tracking using CONDENSATION (Isard and Blake 1998) © 1998 Springer: (a) sample set representation of head estimate distribution; (b) multiple measurements at each control vertex location; (c) multi-hypothesis tracking over time.*

1974). The matrices $\mathbf{A}$ and the noise covariance can be learned ahead of time by observing typical sequences of the object being tracked (Blake and Isard 1998).

In many situations, however, such as when tracking in clutter, a better estimate for the contour can be obtained if we remove the assumptions that the distributions are Gaussian, which is what the Kalman filter requires. In this case, a general multi-modal distribution is propagated. To model such multi-modal distributions, Isard and Blake (1998) introduced the use of *particle filtering* to the computer vision community.¹² Particle filtering techniques represent a probability distribution using a collection of weighted point samples (Andrieu, de Freitas *et al.* 2003; Bishop 2006; Koller and Friedman 2009).

To update the locations of the samples according to the linear dynamics (deterministic drift), the centers of the samples are updated and multiple samples are generated for each point. These are then perturbed to account for the stochastic diffusion, i.e., their locations are moved by random vectors taken from the distribution of $\mathbf{w}$.¹³ Finally, the weights of these samples are multiplied by the measurement probability density, i.e., we take each sample and measure its likelihood given the current (new) measurements. Because the point samples represent and propagate conditional estimates of the multi-modal density, Isard and Blake (1998) dubbed their algorithm CONditiOnal DENsity propaGATION or CONDENSATION.

Figure 7.42a shows what a factored sample of a head tracker might look like, drawing a red B-spline contour for each of (a subset of) the particles being tracked. Figure 7.42b shows why the measurement density itself is often multi-modal: the locations of the edges

¹²Alternatives to modeling multi-modal distributions include *mixtures of Gaussians* (Section 5.2.2) and *multiple hypothesis tracking* (Bar-Shalom and Fortmann 1988; Cham and Rehg 1999).

¹³Note that because of the structure of these steps, non-linear dynamics and non-Gaussian noise can be used.

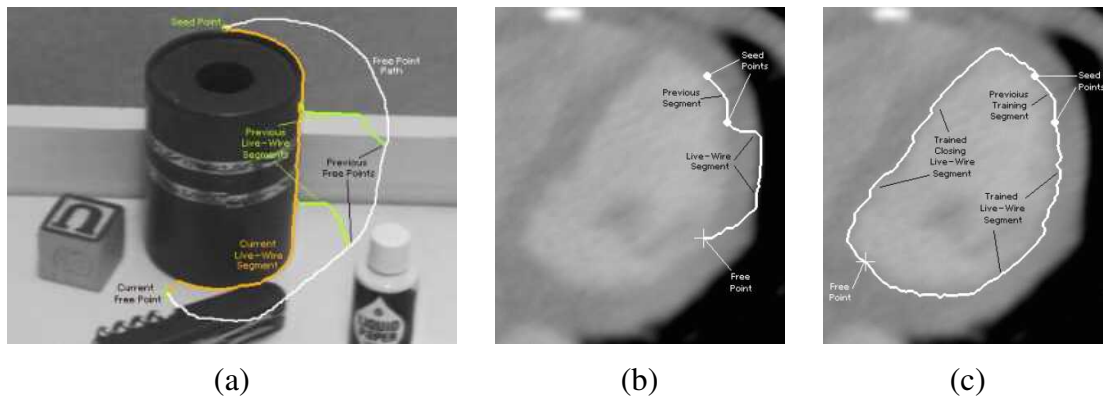


Figure 7.43 *Intelligent scissors: (a) as the mouse traces the white path, the scissors follow the orange path along the object boundary (the green curves show intermediate positions) (Mortensen and Barrett 1995) © 1995 ACM; (b) regular scissors can sometimes jump to a stronger (incorrect) boundary; (c) after training to the previous segment, similar edge profiles are preferred (Mortensen and Barrett 1998) © 1995 Elsevier.*

perpendicular to the spline curve can have multiple local maxima due to background clutter. Finally, Figure 7.42c shows the temporal evolution of the conditional density (x coordinate of the head and shoulder tracker centroid) as it tracks several people over time.

Scissors

Active contours allow a user to roughly specify a boundary of interest and have the system evolve the contour towards a more accurate location as well as track it over time. The results of this curve evolution, however, may be unpredictable and may require additional user-based hints to achieve the desired result.

An alternative approach is to have the system optimize the contour in real time as the user is drawing (Mortensen 1999). The *intelligent scissors* system developed by Mortensen and Barrett (1995) does just that. As the user draws a rough outline (the white curve in Figure 7.43a), the system computes and draws a better curve that clings to high-contrast edges (the orange curve).

To compute the optimal curve path (*live-wire*), the image is first pre-processed to associate low costs with edges (links between neighboring horizontal, vertical, and diagonal, i.e., $\mathcal{N}_8$ neighbors) that are likely to be boundary elements. Their system uses a combination of zero-crossing, gradient magnitudes, and gradient orientations to compute these costs.

Next, as the user traces a rough curve, the system continuously recomputes the lowest-cost path between the starting *seed point* and the current mouse location using Dijkstra's al-

gorithm, a breadth-first dynamic programming algorithm that terminates at the current target location.

In order to keep the system from jumping around unpredictably, the system will “freeze” the curve to date (reset the seed point) after a period of inactivity. To prevent the live wire from jumping onto adjacent higher-contrast contours, the system also “learns” the intensity profile under the current optimized curve, and uses this to preferentially keep the wire moving along the same (or a similar looking) boundary (Figure 7.43b–c).

Several extensions have been proposed to the basic algorithm, which works remarkably well even in its original form. Mortensen and Barrett (1999) use *tobogganing*, which is a simple form of watershed region segmentation, to pre-segment the image into regions whose boundaries become candidates for optimized curve paths. The resulting region boundaries are turned into a much smaller graph, where nodes are located wherever three or four regions meet. The Dijkstra algorithm is then run on this reduced graph, resulting in much faster (and often more stable) performance. Another extension to intelligent scissors is to use a probabilistic framework that takes into account the current trajectory of the boundary, resulting in a system called JetStream (Pérez, Blake, and Gangnet 2001).

Instead of re-computing an optimal curve at each time instant, a simpler system can be developed by simply “snapping” the current mouse position to the nearest likely boundary point (Gleicher 1995). Applications of these boundary extraction techniques to image cutting and pasting are presented in Section 10.4.

7.3.2 Level Sets

A limitation of active contours based on parametric curves of the form $f(s)$, e.g., snakes, B-snakes, and CONDENSATION, is that it is challenging to change the topology of the curve as it evolves (McInerney and Terzopoulos 1999, 2000). Furthermore, if the shape changes dramatically, curve reparameterization may also be required.

An alternative representation for such closed contours is to use a *level set*, where the *zero-crossing(s)* of a *characteristic* (or signed distance (Section 3.3.3)) function define the curve. Level sets evolve to fit and track objects of interest by modifying the underlying *embedding function* (another name for this 2D function) $\phi(x, y)$ instead of the curve $f(s)$ (Malladi, Sethian, and Vemuri 1995; Sethian 1999; Sapiro 2001; Osher and Paragios 2003). To reduce the amount of computation required, only a small strip (frontier) around the locations of the current zero-crossing needs to be updated at each step, which results in what are called *fast marching methods* (Sethian 1999).

An example of an evolution equation is the *geodesic active contour* proposed by Caselles,

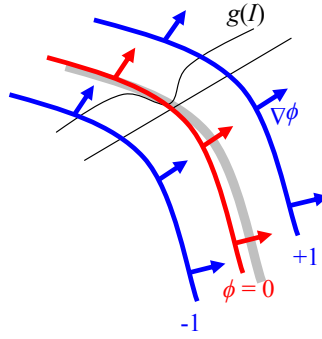


Figure 7.44 Level set evolution for a geodesic active contour. The embedding function ϕ is updated based on the curvature of the underlying surface modulated by the edge/speed function $g(I)$, as well as the gradient of $g(I)$, thereby attracting it to strong edges.

Kimmel, and Sapiro (1997) and Yezzi, Kichenassamy *et al.* (1997),

$$\begin{aligned}\frac{d\phi}{dt} &= |\nabla\phi| \operatorname{div} \left(g(I) \frac{\nabla\phi}{|\nabla\phi|} \right) \\ &= g(I) |\nabla\phi| \operatorname{div} \left(\frac{\nabla\phi}{|\nabla\phi|} \right) + \nabla g(I) \cdot \nabla\phi,\end{aligned}\tag{7.38}$$

where $g(I)$ is a generalized version of the snake edge potential. To get an intuitive sense of the curve's behavior, assume that the embedding function ϕ is a signed distance function away from the curve (Figure 7.44), in which case $|\phi| = 1$. The first term in Equation (7.38) moves the curve in the direction of its curvature, i.e., it acts to straighten the curve, under the influence of the modulation function $g(I)$. The second term moves the curve down the gradient of $g(I)$, encouraging the curve to migrate towards minima of $g(I)$.

While this level-set formulation can readily change topology, it is still susceptible to local minima, since it is based on local measurements such as image gradients. An alternative approach is to re-cast the problem in a segmentation framework, where the energy measures the consistency of the image statistics (e.g., color, texture, motion) inside and outside the segmented regions (Cremers, Rousson, and Deriche 2007; Rousson and Paragios 2008; Houhou, Thiran, and Bresson 2008). These approaches build on earlier energy-based segmentation frameworks introduced by Leclerc (1989), Mumford and Shah (1989), and Chan and Vese (2001), which are discussed in more detail in Section 4.3.2.

For more information on level sets and their applications, please see the collection of papers edited by Osher and Paragios (2003) as well as the series of Workshops on Variational and Level Set Methods in Computer Vision (Paragios, Faugeras *et al.* 2005) and Special Issues on Scale Space and Variational Methods in Computer Vision (Paragios and Sgallari

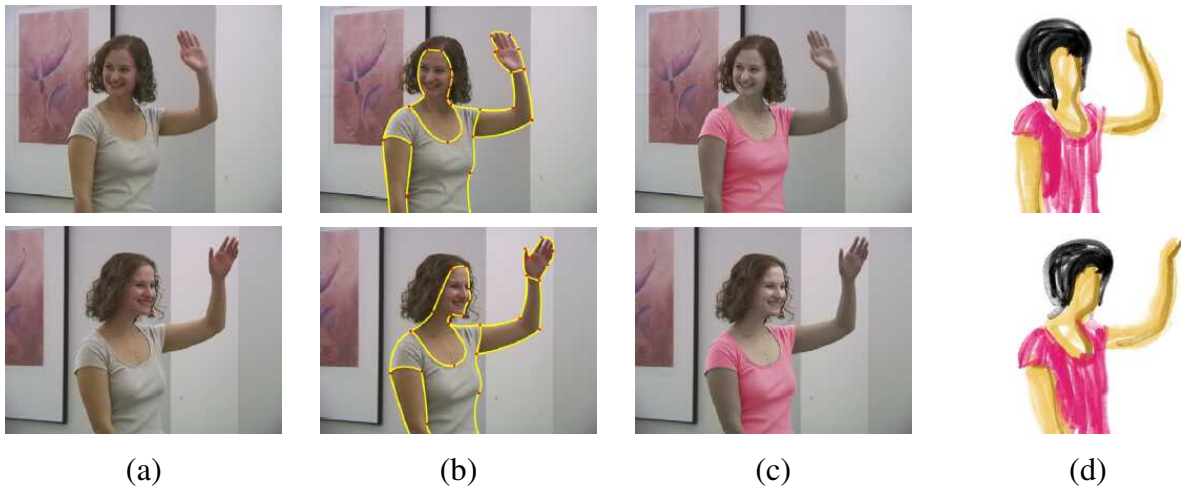


Figure 7.45 *Keyframe-based rotoscoping (Agarwala, Hertzmann et al. 2004) © 2004 ACM: (a) original frames; (b) rotoscoped contours; (c) re-colored blouse; (d) rotoscoped hand-drawn animation.*

2009).

7.3.3 Application: Contour tracking and rotoscoping

Active contours can be used in a wide variety of object-tracking applications (Blake and Isard 1998; Yilmaz, Javed, and Shah 2006). For example, they can be used to track facial features for performance-driven animation (Terzopoulos and Waters 1990; Lee, Terzopoulos, and Waters 1995; Parke and Waters 1996; Bregler, Covell, and Slaney 1997). They can also be used to track heads and people, as shown in Figure 7.42, as well as moving vehicles (Paragios and Deriche 2000). Additional applications include medical image segmentation, where contours can be tracked from slice to slice in computed tomography (Cootes and Taylor 2001), or over time, as in ultrasound scans.

An interesting application that is closer to computer animation and visual effects is *roto-scoping*, which uses the tracked contours to deform a set of hand-drawn animations (or to modify or replace the original video frames).¹⁴ Agarwala, Hertzmann *et al.* (2004) present a system based on tracking hand-drawn B-spline contours drawn at selected keyframes, using a combination of geometric and appearance-based criteria (Figure 7.45). They also provide an excellent review of previous rotoscoping and image-based, contour-tracking systems.

Additional applications of rotoscoping (object contour detection and segmentation), such

¹⁴The term comes from a device (a rotoscope) that projected frames of a live-action film underneath an acetate so that artists could draw animations directly over the actors' shapes.

as cutting and pasting objects from one photograph into another, are presented in Section 10.4.

7.4 Lines and vanishing points

While edges and general curves are suitable for describing the contours of natural objects, the human-made world is full of straight lines. Detecting and matching these lines can be useful in a variety of applications, including architectural modeling, pose estimation in urban environments, and the analysis of printed document layouts.

In this section, we present some techniques for extracting *piecewise linear* descriptions from the curves computed in the previous section. We begin with some algorithms for approximating a curve as a piecewise-linear polyline. We then describe the *Hough transform*, which can be used to group edgels into line segments even across gaps and occlusions. Finally, we describe how 3D lines with common *vanishing points* can be grouped together. These vanishing points can be used to calibrate a camera and to determine its orientation relative to a rectahedral scene, as described in Section 11.1.1.

7.4.1 Successive approximation

As we saw in Section 7.2.2, describing a curve as a series of 2D locations $\mathbf{x}_i = \mathbf{x}(s_i)$ provides a general representation suitable for matching and further processing. In many applications, however, it is preferable to approximate such a curve with a simpler representation, e.g., as a piecewise-linear polyline or as a B-spline curve (Farin 2002).

Many techniques have been developed over the years to perform this approximation, which is also known as *line simplification*. One of the oldest, and simplest, is the one proposed by Ramer (1972) and Douglas and Peucker (1973), who recursively subdivide the curve at the point furthest away from the line joining the two endpoints (or the current coarse polyline approximation). Hershberger and Snoeyink (1992) provide a more efficient implementation and also cite some of the other related work in this area.

Once the line simplification has been computed, it can be used to approximate the original curve. If a smoother representation or visualization is desired, either approximating or interpolating splines or curves can be used (Sections 3.5.1 and 7.3.1) (Szeliski and Ito 1986; Bartels, Beatty, and Barsky 1987; Farin 2002).

7.4.2 Hough transforms

While curve approximation with polylines can often lead to successful line extraction, lines in the real world are sometimes broken up into disconnected components or made up of many

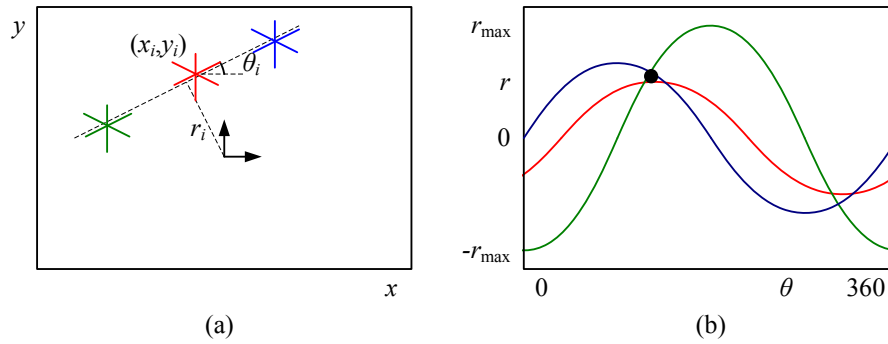


Figure 7.46 *Original Hough transform: (a) each point votes for a complete family of potential lines $r_i(\theta) = x_i \cos \theta + y_i \sin \theta$; (b) each pencil of lines sweeps out a sinusoid in (r, θ) ; their intersection provides the desired line equation.*

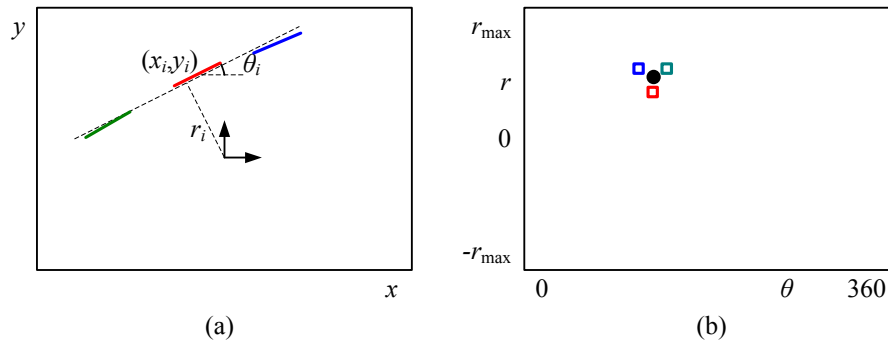


Figure 7.47 *Oriented Hough transform: (a) an edgel re-parameterized in polar (r, θ) coordinates, with $\hat{\mathbf{n}}_i = (\cos \theta_i, \sin \theta_i)$ and $r_i = \hat{\mathbf{n}}_i \cdot \mathbf{x}_i$; (b) (r, θ) accumulator array, showing the votes for the three edgels marked in red, green, and blue.*

collinear line segments. In many cases, it is desirable to group such collinear segments into extended lines. At a further processing stage (described in Section 7.4.3), we can then group such lines into collections with common vanishing points.

The Hough transform, named after its original inventor (Hough 1962), is a well-known technique for having edges “vote” for plausible line locations (Duda and Hart 1972; Ballard 1981; Illingworth and Kittler 1988). In its original formulation (Figure 7.46), each edge point votes for *all* possible lines passing through it, and lines corresponding to high *accumulator* or *bin* values are examined for potential line fits.¹⁵ Unless the points on a line are truly punctate, a better approach is to use the local orientation information at each edgel to vote for a *single* accumulator cell (Figure 7.47), as described below. A hybrid strategy, where each edgel votes

¹⁵The Hough transform can also be *generalized* to look for other geometric features, such as circles (Ballard 1981).

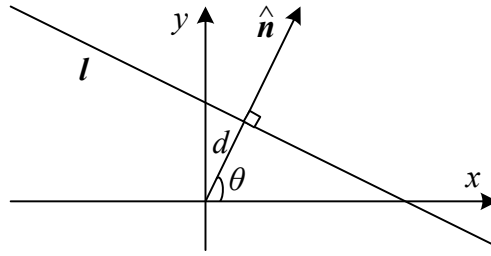


Figure 7.48 2D line equation expressed in terms of the normal $\hat{\mathbf{n}}$ and distance to the origin d .

for a number of possible orientation or location pairs centered around the estimate orientation, may be desirable in some cases.

Before we can vote for line hypotheses, we must first choose a suitable representation. Figure 7.48 (copied from Figure 2.2a) shows the normal-distance ($\hat{\mathbf{n}}, d$) parameterization for a line. Since lines are made up of edge segments, we adopt the convention that the line normal $\hat{\mathbf{n}}$ points in the same direction (i.e., has the same sign) as the image gradient $\mathbf{J}(\mathbf{x}) = \nabla I(\mathbf{x})$ (7.19). To obtain a minimal two-parameter representation for lines, we convert the normal vector into an angle

$$\theta = \tan^{-1} n_y / n_x, \quad (7.39)$$

as shown in Figure 7.48. The range of possible (θ, d) values is $[-180^\circ, 180^\circ] \times [-\sqrt{2}, \sqrt{2}]$, assuming that we are using normalized pixel coordinates (2.61) that lie in $[-1, 1]$. The number of bins to use along each axis depends on the accuracy of the position and orientation estimate available at each edgel and the expected line density, and is best set experimentally with some test runs on sample imagery.

There are a lot of details in getting the Hough transform to work well, including using edge segment lengths or strengths during the voting process, keeping a list of constituent edgels in the accumulator array for easier post-processing, and optionally combining edges of different “polarity” into the same line segments. These are best worked out by writing an implementation and testing it out on sample data.

An alternative to the 2D polar (θ, d) representation for lines is to use the full 3D $\mathbf{m} = (\hat{\mathbf{n}}, d)$ line equation, projected onto the unit sphere. While the sphere can be parameterized using spherical coordinates (2.8),

$$\hat{\mathbf{m}} = (\cos \theta \cos \phi, \sin \theta \cos \phi, \sin \phi), \quad (7.40)$$

this does not uniformly sample the sphere and still requires the use of trigonometry.

An alternative representation can be obtained by using a *cube map*, i.e., projecting $\mathbf{m}$ onto the face of a unit cube (Figure 7.49a). To compute the cube map coordinate of a 3D vector

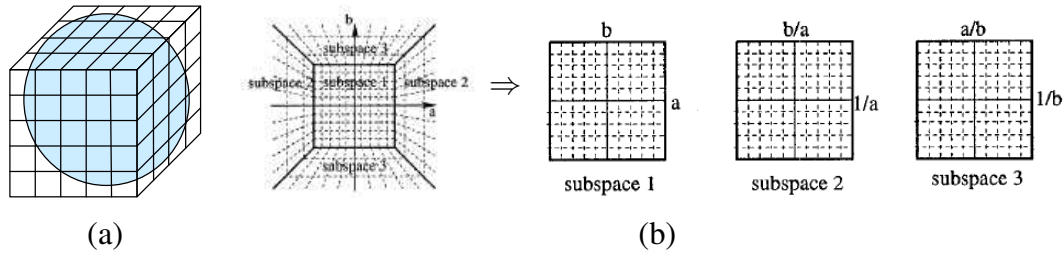


Figure 7.49 Cube map representation for line equations and vanishing points: (a) a cube map surrounding the unit sphere; (b) projecting the half-cube onto three subspaces (Tuytelaars, Van Gool, and Proesmans 1997) © 1997 IEEE.

$\mathbf{m}$, first find the largest (absolute value) component of $\mathbf{m}$, i.e., $m = \pm \max(|n_x|, |n_y|, |d|)$, and use this to select one of the six cube faces. Divide the remaining two coordinates by m and use these as indices into the cube face. While this avoids the use of trigonometry, it does require some decision logic.

One advantage of using the cube map, first pointed out by Tuytelaars, Van Gool, and Proesmans (1997), is that all of the lines passing through a point correspond to line segments on the cube faces, which is useful if the original (full voting) variant of the Hough transform is being used. In their work, they represent the line equation as $ax + b + y = 0$, which does not treat the x and y axes symmetrically. Note that if we restrict $d \geq 0$ by ignoring the polarity of the edge orientation (gradient sign), we can use a half-cube instead, which can be represented using only three cube faces, as shown in Figure 7.49b (Tuytelaars, Van Gool, and Proesmans 1997).

RANSAC-based line detection. Another alternative to the Hough transform is the RANdom SAMple Consensus (RANSAC) algorithm described in more detail in Section 8.1.4. In brief, RANSAC randomly chooses pairs of edgels to form a line hypothesis and then tests how many other edgels fall onto this line. (If the edge orientations are accurate enough, a single edgel can produce this hypothesis.) Lines with sufficiently large numbers of *inliers* (matching edgels) are then selected as the desired line segments.

An advantage of RANSAC is that no accumulator array is needed, so the algorithm can be more space efficient and potentially less prone to the choice of bin size. The disadvantage is that many more hypotheses may need to be generated and tested than those obtained by finding peaks in the accumulator array.

Bottom-up grouping. Yet another approach to line segment detection is to iteratively group edgels with similar orientations into oriented rectangular *line-support regions* (Burns, Han-

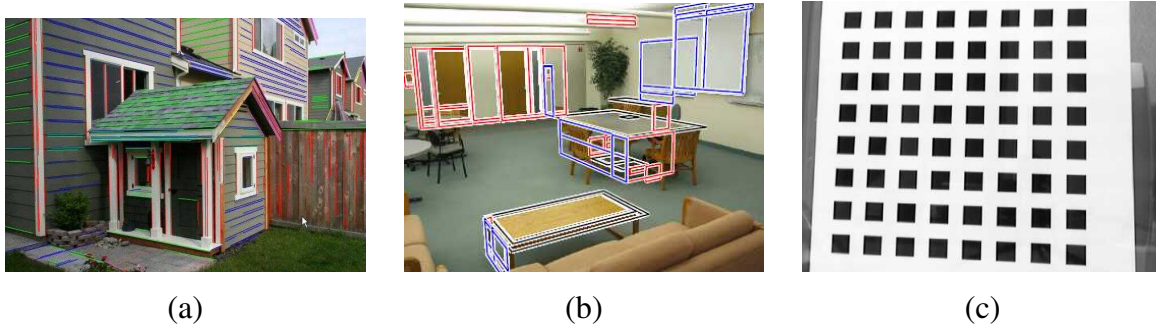


Figure 7.50 *Real-world vanishing points: (a) architecture (Sinha, Steedly et al. 2008), (b) furniture (Mičušík, Wildenauer, and Košecká 2008) © 2008 IEEE, and (c) calibration patterns (Zhang 2000).*

son, and Riseman 1986). The validity of such regions can then be determined using a statistical analysis, as described in the LSD paper by Grompone von Gioi, Jakubowicz *et al.* (2008). The resulting algorithm is quite fast, does a good job of distinguishing line segments from texture, and is widely used in practice because of its performance and open source availability. Recently, deep neural network algorithms have been developed to simultaneously extract line segments and their junctions (Huang, Wang *et al.* 2018; Zhang, Li *et al.* 2019; Huang, Qin *et al.* 2020; Lin, Pinteá, and van Gemert 2020).

In general, there is no clear consensus on which line estimation technique performs best. It is therefore a good idea to think carefully about the problem at hand and to implement several approaches (successive approximation, Hough, and RANSAC) to determine the one that works best for your application.

7.4.3 Vanishing points

In many scenes, structurally important lines have the same vanishing point because they are parallel in 3D. Examples of such lines are horizontal and vertical building edges, zebra crossings, railway tracks, the edges of furniture such as tables and dressers, and of course, the ubiquitous calibration pattern (Figure 7.50). Finding the vanishing points common to such line sets can help refine their position in the image and, in certain cases, help determine the intrinsic and extrinsic orientation of the camera (Section 11.1.1).

Over the years, a large number of techniques have been developed for finding vanishing points (Quan and Mohr 1989; Collins and Weiss 1990; Brillaut-O'Mahoney 1991; McLean and Kotturi 1995; Becker and Bove 1995; Shufelt 1999; Tuytelaars, Van Gool, and Proesmans 1997; Schaffalitzky and Zisserman 2000; Antone and Teller 2002; Rother 2002; Košecká and Zhang 2005; Denis, Elder, and Estrada 2008; Pflugfelder 2008; Tardif 2009; Bazin, Seo *et al.*

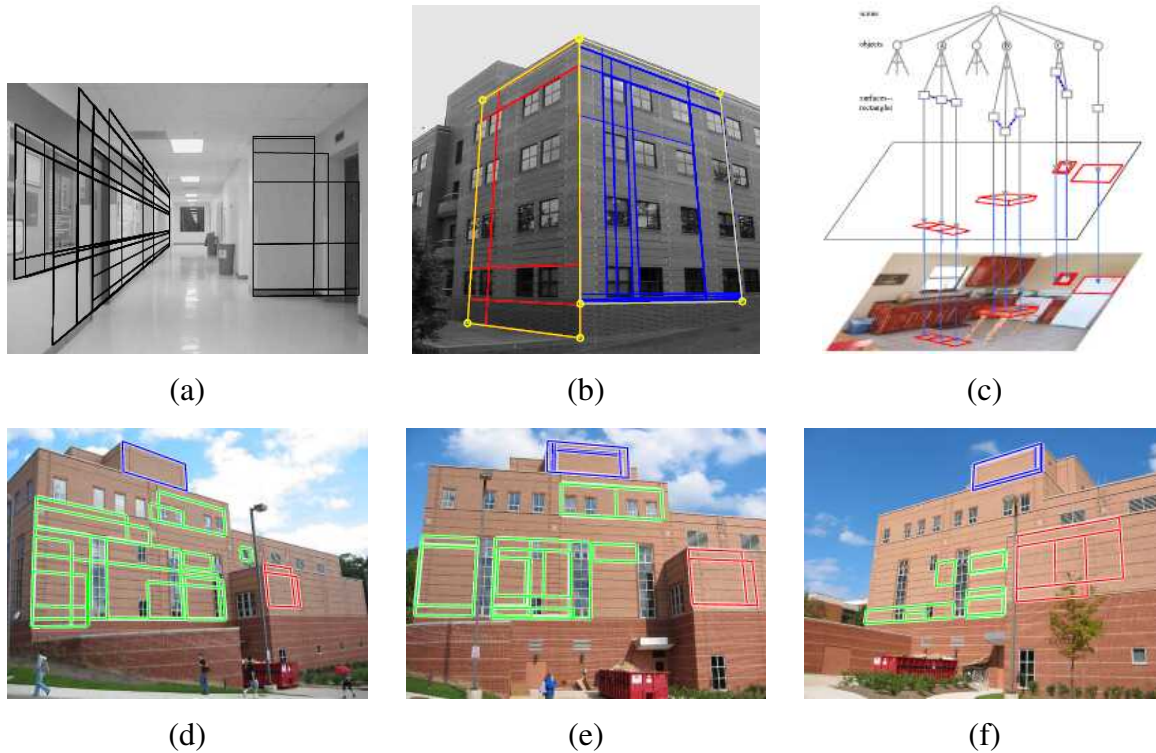


Figure 7.51 *Rectangle detection: (a) indoor corridor and (b) building exterior with grouped facades (Košecká and Zhang 2005) © 2005 Elsevier; (c) grammar-based recognition (Han and Zhu 2005) © 2005 IEEE; (d–f) rectangle matching using a plane sweep algorithm (Mičušík, Wildenauer, and Košecká 2008) © 2008 IEEE.*

2012; Antunes and Barreto 2013; Kluger, Ackermann *et al.* 2017; Zhou, Qi *et al.* 2019a)—see some of the more recent papers for additional references and alternative approaches.

In the first edition of this book (Szeliski 2010, Section 4.3.3), I presented a simple Hough technique based on having line pairs vote for potential vanishing point locations, followed by a robust least squares fitting stage. While my technique proceeds in two discrete stages, better results may be obtained by alternating between assigning lines to vanishing points and refitting the vanishing point locations (Antone and Teller 2002; Košecká and Zhang 2005; Pflugfelder 2008). The results of detecting individual vanishing points can also be made more robust by simultaneously searching for pairs or triplets of mutually orthogonal vanishing points (Shufelt 1999; Antone and Teller 2002; Rother 2002; Sinha, Steedly *et al.* 2008; Li, Kim *et al.* 2020). Some results of such vanishing point detection algorithms can be seen in Figure 7.50. It is also possible to simultaneously detect line segments and their junctions using a neural network (Zhang, Li *et al.* 2019) and to then use these to construct complete 3D wireframe models (Zhou, Qi, and Ma 2019; Zhou, Qi *et al.* 2019b).

Rectangle detection

Once sets of mutually orthogonal vanishing points have been detected, it now becomes possible to search for 3D rectangular structures in the image (Figure 7.51). A variety of techniques have been developed to find such rectangles, primarily focused on architectural scenes (Košecká and Zhang 2005; Han and Zhu 2005; Shaw and Barnes 2006; Mičušík, Wildenauer, and Košecká 2008; Schindler, Krishnamurthy *et al.* 2008).

After detecting orthogonal vanishing directions, Košecká and Zhang (2005) refine the fitted line equations, search for corners near line intersections, and then verify rectangle hypotheses by rectifying the corresponding patches and looking for a preponderance of horizontal and vertical edges (Figures 7.51a–b). In follow-on work, Mičušík, Wildenauer, and Košecká (2008) use a Markov random field (MRF) to disambiguate between potentially overlapping rectangle hypotheses. They also use a plane sweep algorithm to match rectangles between different views (Figures 7.51d–f).

A different approach is proposed by Han and Zhu (2005), who use a grammar of potential rectangle shapes and nesting structures (between rectangles and vanishing points) to infer the most likely assignment of line segments to rectangles (Figure 7.51c). The idea of using regular, repetitive structures as part of the modeling process is now being called *holistic 3D reconstruction* (Zhou, Furukawa, and Ma 2019; Zhou, Furukawa *et al.* 2020; Pintore, Mura *et al.* 2020) and will be discussed in more detail in Section 13.6.1 on modeling 3D architecture.

7.5 Segmentation

Image segmentation is the task of finding groups of pixels that “go together”. In statistics and machine learning, this problem is known as *cluster analysis* or more simply *clustering* and is a widely studied area with hundreds of different algorithms (Jain and Dubes 1988; Kaufman and Rousseeuw 1990; Jain, Duin, and Mao 2000; Jain, Topchy *et al.* 2004; Xu and Wunsch 2005). We’ve already discussed general vector-space clustering algorithms in Section 5.2.1. The main difference between clustering and segmentation is that the former usually ignores pixel layout and neighborhoods, while the latter relies heavily on spatial cues and constraints.

In computer vision, image segmentation is one of the oldest and most widely studied problems (Brice and Fennema 1970; Pavlidis 1977; Riseman and Arbib 1977; Ohlander, Price, and Reddy 1978; Rosenfeld and Davis 1979; Haralick and Shapiro 1985). Early techniques often used region splitting or merging (Brice and Fennema 1970; Horowitz and Pavlidis 1976; Ohlander, Price, and Reddy 1978; Pavlidis and Liow 1990), which correspond to *divisive* and *agglomerative* algorithms (Jain, Topchy *et al.* 2004; Xu and Wunsch 2005), which we intro-

duced in Section 5.2.1. More recent algorithms typically optimize some global criterion, such as intra-region consistency and inter-region boundary lengths or dissimilarity (Leclerc 1989; Mumford and Shah 1989; Shi and Malik 2000; Comaniciu and Meer 2002; Felzenszwalb and Huttenlocher 2004; Cremers, Rousson, and Deriche 2007; Pont-Tuset, Arbeláez *et al.* 2017).

We have already seen examples of image segmentation using image morphology (Section 3.3.3), Markov random fields (Section 4.3), active contours (Section 7.3), and level sets (Section 7.3.2). In the recognition chapter (Section 6.4), we studied *semantic segmentation*, whose goal is to break the image up into semantically labeled regions such as sky, grass, and individual people and animals. In this section, we review some additional techniques for bottom-up general (non-semantic) image segmentation. These include algorithms based on region splitting and merging, graph-based segmentation, and probabilistic aggregation (Section 7.5.1), *mean shift* mode finding (Section 7.5.2), and *normalized cuts* splitting based on pixel similarity metrics (Section 7.5.3). Since many of these algorithms are no longer widely used, a lot of the descriptions have been considerably shortened from those found in the first edition of this book (Szeliski 2010, Chapter 5), where you can find longer descriptions.

Since the literature on image segmentation is so vast, a good way to get a handle on some of the better performing algorithms is to look at experimental comparisons on human-labeled databases (Arbeláez, Maire *et al.* 2011; Pont-Tuset, Arbeláez *et al.* 2017). The best known of these is the Berkeley Segmentation Dataset and Benchmark (Martin, Fowlkes *et al.* 2001), which consists of 1,000 images from a Corel image dataset that were hand-labeled by 30 human subjects, for which Unnikrishnan, Pantofaru, and Hebert (2007) propose new metrics for comparing segmentation algorithms, while Estrada and Jepson (2009) compare four well-known segmentation algorithms. A newer database of foreground and background segmentations, used by Alpert, Galun *et al.* (2007), is also available.

As mentioned in Section 3.3.3, the simplest possible technique for segmenting a grayscale image is to select a threshold and then compute connected components. Unfortunately, a single threshold is rarely sufficient for the whole image because of lighting and intra-object statistical variations.

Region splitting (divisive clustering). Splitting the image into successively finer regions is one of the oldest techniques in computer vision. Ohlander, Price, and Reddy (1978) present such a technique, which first computes a histogram for the whole image and then finds a threshold that best separates the large peaks in the histogram. This process is repeated until regions are either fairly uniform or below a certain size. More recent splitting algorithms often optimize some metric of intra-region similarity and inter-region dissimilarity. These are covered in Sections 4.3.2 and Sections 7.5.3.

Region merging (agglomerative clustering). Region merging techniques also date back to the beginnings of computer vision. Brice and Fennema (1970) use a dual grid for representing boundaries between pixels and merge regions based on their relative boundary lengths and the strength of the visible edges at these boundaries.

A very simple version of pixel-based merging combines adjacent regions whose average color difference is below a threshold or whose regions are too small. Segmenting the image into such *superpixels* (Mori, Ren *et al.* 2004), which are not semantically meaningful, can be a useful pre-processing stage to make higher-level algorithms such as stereo matching (Zitnick, Kang *et al.* 2004; Taguchi, Wilburn, and Zitnick 2008), optical flow (Zitnick, Jojic, and Kang 2005; Brox, Bregler, and Malik 2009), and recognition (Mori, Ren *et al.* 2004; Mori 2005; Gu, Lim *et al.* 2009; Lim, Arbeláez *et al.* 2009) both faster and more robust. It is also possible to combine both splitting and merging by starting with a medium-grain segmentation (in a quadtree representation) and then allowing both merging and splitting operations (Horowitz and Pavlidis 1976; Pavlidis and Liow 1990).

Watershed. A technique related to thresholding, since it operates on a grayscale image, is *watershed* computation (Vincent and Soille 1991). This technique segments an image into several *catchment basins*, which are the regions of an image (interpreted as a height field or landscape) where rain would flow into the same lake. An efficient way to compute such regions is to start flooding the landscape at all of the local minima and to label ridges wherever differently evolving components meet. The whole algorithm can be implemented using a priority queue of pixels and breadth-first search (Vincent and Soille 1991).¹⁶

Since images rarely have dark regions separated by lighter ridges, watershed segmentation is usually applied to a smoothed version of the gradient magnitude image, which also makes it usable with color images. As an alternative, the maximum oriented energy in a steerable filter (3.28–3.29) (Freeman and Adelson 1991) can be used as the basis of the *oriented watershed transform* developed by Arbeláez, Maire *et al.* (2011). Such techniques end up finding smooth regions separated by visible (higher gradient) boundaries. Since such boundaries are what active contours usually follow, active contour algorithms (Mortensen and Barrett 1999; Li, Sun *et al.* 2004) often precompute such a segmentation using either the watershed or the related *tobogganing* technique (Section 7.3.1).

¹⁶A related algorithm can be used to compute maximally stable extremal regions (MSERs) efficiently (Section 7.1.1) (Nistér and Stewénius 2008).

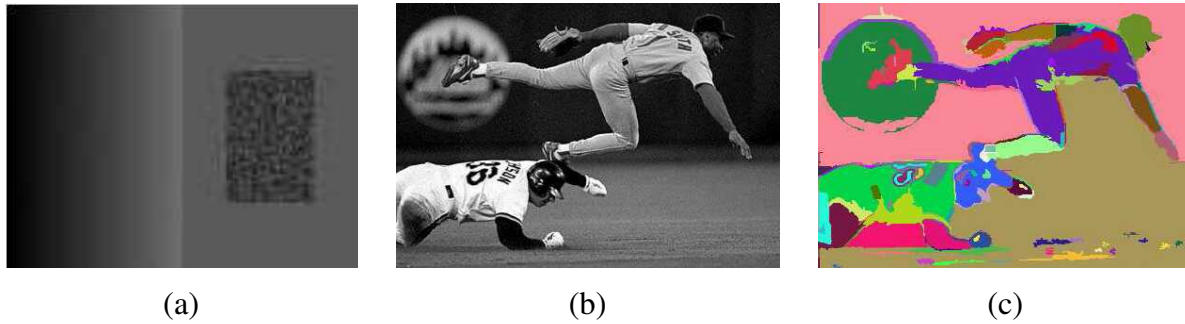


Figure 7.52 Graph-based merging segmentation (*Felzenszwalb and Huttenlocher 2004*)
 © 2004 Springer: (a) input grayscale image that is successfully segmented into three regions even though the variation inside the smaller rectangle is larger than the variation across the middle edge; (b) input grayscale image; (c) resulting segmentation using an $\mathcal{N}_8$ pixel neighborhood.

7.5.1 Graph-based segmentation

While many merging algorithms simply apply a fixed rule that groups pixels and regions together, *Felzenszwalb and Huttenlocher (2004)* present a merging algorithm that uses *relative dissimilarities* between regions to determine which ones should be merged; it produces an algorithm that provably optimizes a global grouping metric. They start with a pixel-to-pixel dissimilarity measure $w(e)$ that measures, for example, intensity differences between $\mathcal{N}_8$ neighbors. Alternatively, they can use the *joint feature space* distances introduced by *Comaniciu and Meer (2002)*, which we discuss in Sections 7.5.2 and 7.5.3. Figure 7.52 shows two examples of images segmented using their technique.

Probabilistic aggregation

Alpert, Galun et al. (2007) develop a probabilistic merging algorithm based on two cues, namely gray-level similarity and texture similarity. The gray-level similarity between regions R_i and R_j is based on the *minimal external difference* from other neighboring regions, which is compared to the *average intensity difference* to compute the likelihoods p_{ij} that two regions should be merged. Merging proceeds in a hierarchical fashion inspired by algebraic multigrid techniques (*Brandt 1986; Briggs, Henson, and McCormick 2000*) and previously used by *Alpert, Galun et al. (2007)* in their segmentation by weighted aggregation (SWA) algorithm (*Sharon, Galun et al. 2006*). Figure 7.56 shows the segmentations produced by this algorithm compared to other popular segmentation algorithms.

7.5.2 Mean shift

Mean-shift and mode finding techniques, such as k-means and mixtures of Gaussians, model the feature vectors associated with each pixel (e.g., color and position) as samples from an unknown probability density function and then try to find clusters (modes) in this distribution.

Consider the color image shown in Figure 7.53a. How would you segment this image based on color alone? Figure 7.53b shows the distribution of pixels in $L^*u^*v^*$ space, which is equivalent to what a vision algorithm that ignores spatial location would see. To make the visualization simpler, let us only consider the L^*u^* coordinates, as shown in Figure 7.53c. How many obvious (elongated) clusters do you see? How would you go about finding these clusters?

The k-means and mixtures of Gaussians techniques we studied in Section 5.2.2 use a *parametric* model of the density function to answer this question, i.e., they assume the density is the superposition of a small number of simpler distributions (e.g., Gaussians) whose locations (centers) and shape (covariance) can be estimated. Mean shift, on the other hand, smoothes the distribution and finds its peaks as well as the regions of feature space that correspond to each peak. Since a complete density is being modeled, this approach is called *non-parametric* (Bishop 2006).

The key to mean shift is a technique for efficiently finding peaks in this high-dimensional data distribution without ever computing the complete function explicitly (Fukunaga and Hostetler 1975; Cheng 1995; Comaniciu and Meer 2002). Consider once again the data points shown in Figure 7.53c, which can be thought of as having been drawn from some probability density function. If we could compute this density function, as visualized in Figure 7.53e, we could find its major peaks (*modes*) and identify regions of the input space that climb to the same peak as being part of the same region. This is the inverse of the *watershed* algorithm described in Section 7.5, which climbs downhill to find *basins of attraction*.

The first question, then, is how to estimate the density function given a sparse set of samples. One of the simplest approaches is to just smooth the data, e.g., by convolving it with a fixed kernel of width h , which, as we saw in Section 4.1.1, is the *Parzen window* approach to density estimation (Duda, Hart, and Stork 2001, Section 4.3; Bishop 2006, Section 2.5.1). Once we have computed $f(\mathbf{x})$, as shown in Figure 7.53e, we can find its local maxima using gradient ascent or some other optimization technique.

The problem with this “brute force” approach is that, for higher dimensions, it becomes computationally prohibitive to evaluate $f(\mathbf{x})$ over the complete search space. Instead, mean shift uses a variant of what is known in the optimization literature as *multiple restart gradient descent*. Starting at some guess for a local maximum, $\mathbf{y}_k$, which can be a random input data point $\mathbf{x}_i$, mean shift computes the gradient of the density estimate $f(\mathbf{x})$ at $\mathbf{y}_k$ and takes an

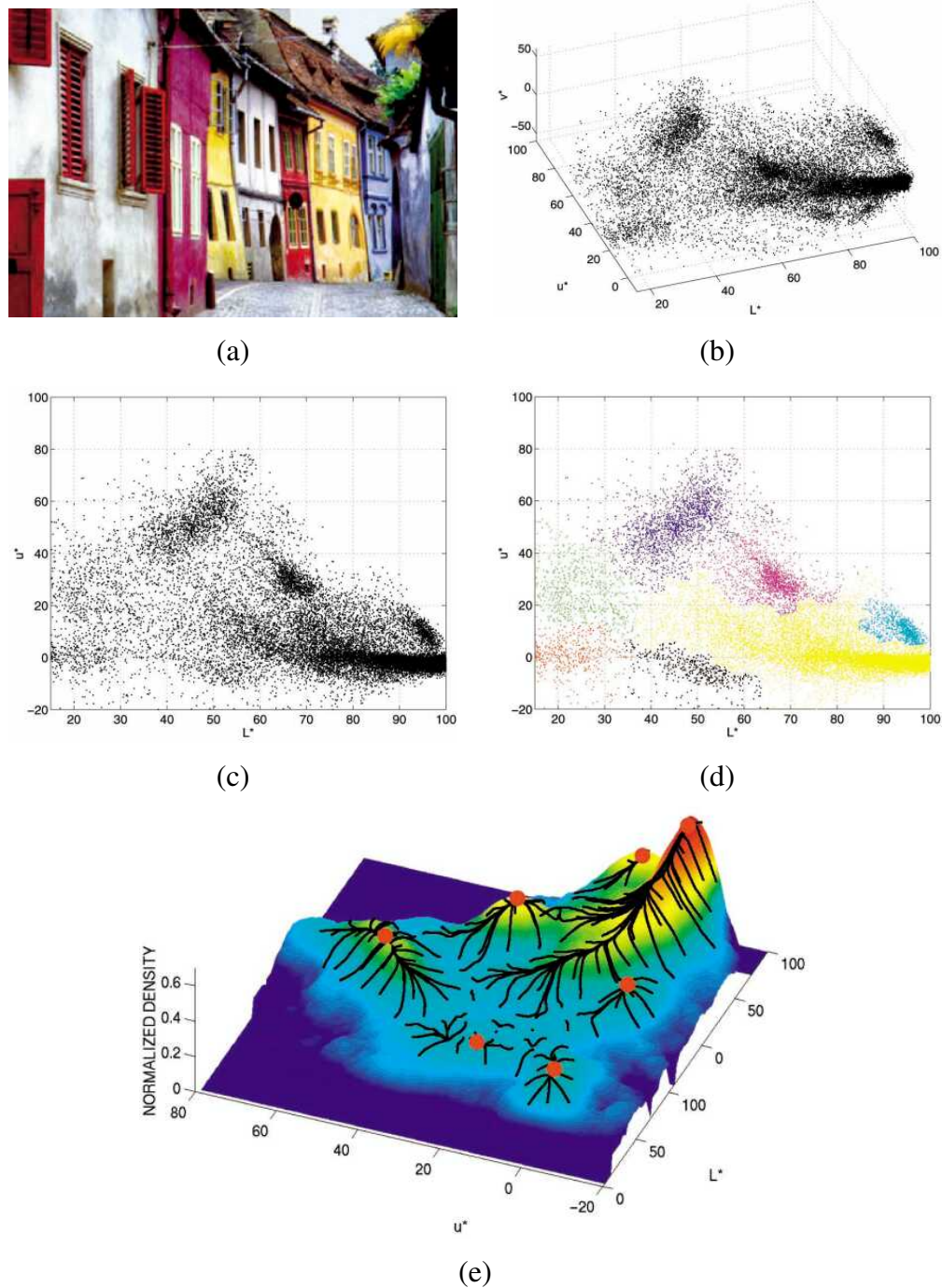


Figure 7.53 Mean-shift image segmentation (*Comaniciu and Meer 2002*) © 2002 IEEE: (a) input color image; (b) pixels plotted in $L^*u^*v^*$ space; (c) L^*u^* space distribution; (d) clustered results after 159 mean-shift procedures; (e) corresponding trajectories with peaks marked as red dots.

uphill step in that direction. Details on how this can be done efficiently can be found in papers on mean shift (Comaniciu and Meer 2002; Paris and Durand 2007) as well as the first edition of this book (Szeliski 2010, Section 5.3.2).

The color-based segmentation shown in Figure 7.53 only looks at pixel colors when determining the best clustering. It may therefore cluster together small isolated pixels that happen to have the same color, which may not correspond to a semantically meaningful segmentation of the image. Better results can usually be obtained by clustering in the *joint domain* of color and location. In this approach, the spatial coordinates of the image $\mathbf{x}_s = (x, y)$, which are called the *spatial domain*, are concatenated with the color values $\mathbf{x}_r$, which are known as the *range domain*, and mean-shift clustering is applied in this five-dimensional space $\mathbf{x}_j$. Since location and color may have different scales, the kernels are adjusted separately, just as in the bilateral filter kernel (3.34–3.37) discussed in Section 3.3.2. The difference between mean shift and bilateral filtering, however, is that in mean shift, the spatial coordinates of each pixel are adjusted along with its color values, so that the pixel migrates more quickly towards other pixels with similar colors, and can therefore later be used for clustering and segmentation.

Mean shift has been applied to a number of different problems in computer vision, including face tracking, 2D shape extraction, and texture segmentation (Comaniciu and Meer 2002), stereo matching (Wei and Quan 2004), non-photorealistic rendering (Section 10.5.2) (DeCarlo and Santella 2002), and video editing (Section 10.4.5) (Wang, Bhat *et al.* 2005). Paris and Durand (2007) provide a nice review of such applications, as well as techniques for more efficiently solving the mean-shift equations and producing hierarchical segmentations.

7.5.3 Normalized cuts

While bottom-up merging techniques aggregate regions into coherent wholes and mean-shift techniques try to find clusters of similar pixels using mode finding, the normalized cuts technique introduced by Shi and Malik (2000) examines the *affinities* (similarities) between nearby pixels and tries to separate groups that are connected by weak affinities.

Consider the simple graph shown in Figure 7.54a. The pixels in group *A* are all strongly connected with high affinities, shown as thick red lines, as are the pixels in group *B*. The connections between these two groups, shown as thinner blue lines, are much weaker. A *normalized cut* between the two groups, shown as a dashed line, separates them into two clusters.

The cut between two groups *A* and *B* is defined as the sum of all the weights being cut, where the weights between two pixels (or regions) *i* and *j* measure their similarity. Using a minimum cut as a segmentation criterion, however, does not result in reasonable clusters, since the smallest cuts usually involve isolating a single pixel.

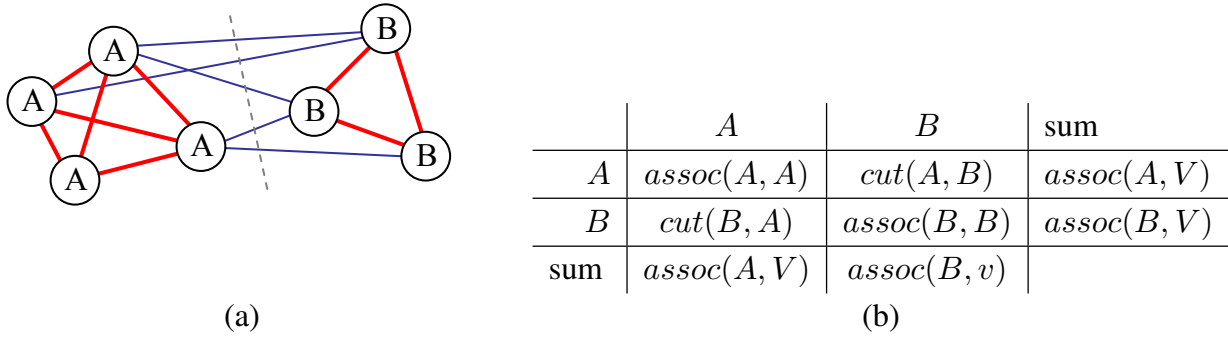


Figure 7.54 Sample weighted graph and its normalized cut: (a) a small sample graph and its smallest normalized cut; (b) tabular form of the associations and cuts for this graph. The *assoc* and *cut* entries are computed as area sums of the associated weight matrix $\mathbf{W}$. Normalizing the table entries by the row or column sums produces normalized associations and cuts N_{assoc} and N_{cut} .

A better measure of segmentation is the normalized cut, which is defined as

$$Ncut(A, B) = \frac{cut(A, B)}{assoc(A, V)} + \frac{cut(A, B)}{assoc(B, V)}, \quad (7.41)$$

where $assoc(A, A) = \sum_{i \in A, j \in A} w_{ij}$ is the *association* (sum of all the weights) within a cluster and $assoc(A, V) = assoc(A, A) + cut(A, B)$ is the sum of *all* the weights associated with nodes in A . Figure 7.54b shows how the cuts and associations can be thought of as area sums in the weight matrix $\mathbf{W} = [w_{ij}]$, where the entries of the matrix have been arranged so that the nodes in A come first and the nodes in B come second. Dividing each of these areas by the corresponding row sum (the rightmost column of Figure 7.54b) results in the normalized cut and association values. These normalized values better reflect the fitness of a particular segmentation, since they look for collections of edges that are weak relative to all of the edges both inside and emanating from a particular region.

Unfortunately, computing the optimal normalized cut is NP-complete. Instead, Shi and Malik (2000) suggest computing a real-valued assignment of nodes to groups, using a generalized eigenvalue analysis of the *normalized* affinity matrix (Weiss 1999), as described in more detail in the normalized cuts paper and (Szeliski 2010, Section 5.4). Because these eigenvectors can be interpreted as the large modes of vibration in a spring-mass system, normalized cuts is an example of a *spectral method* for image segmentation. After the real-valued eigenvector is computed, the variables corresponding to positive and negative eigenvector values are associated with the two cut components. This process can be further repeated to hierarchically subdivide an image, as shown in Figure 7.55.

The original algorithm proposed by Shi and Malik (2000) used spatial position and image

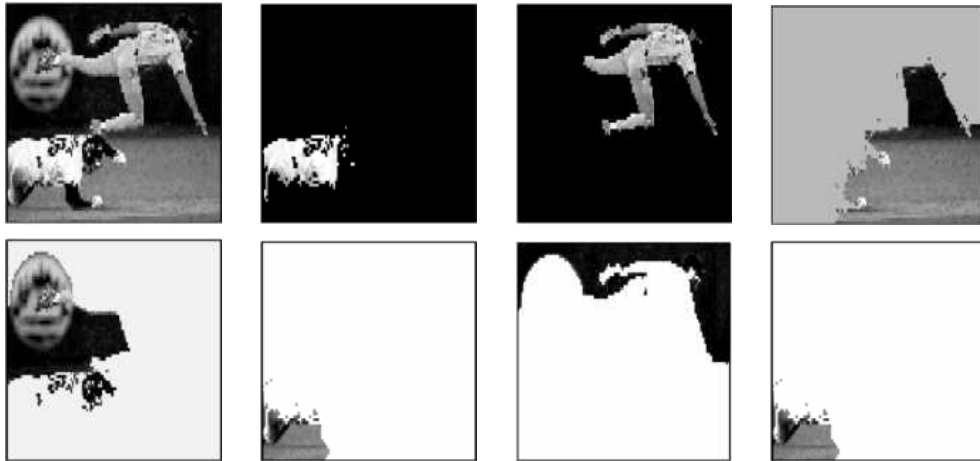


Figure 7.55 *Normalized cuts segmentation (Shi and Malik 2000) © 2000 IEEE: The input image and the components returned by the normalized cuts algorithm.*

feature differences to compute the pixel-wise affinities. In subsequent work, [Malik, Belongie et al. \(2001\)](#) look for *intervening contours* between pixels i and j to define intervening contour weights and then multiply these weights with a texon-based texture similarity metric. They then use an initial over-segmentation based purely on local pixel-wise features to re-estimate intervening contours and texture statistics in a region-based manner. Figure 7.56 shows the results of running this improved algorithm on a number of test images.

Because it requires the solution of large sparse eigenvalue problems, normalized cuts can be quite slow. [Sharon, Galun et al. \(2006\)](#) present a way to accelerate the computation of the normalized cuts using an approach inspired by algebraic multigrid ([Brandt 1986](#); [Briggs, Henson, and McCormick 2000](#)).

An example of the segmentation produced by weighted aggregation (SWA) is shown in Figure 7.56, along with the most recent probabilistic bottom-up merging algorithm by [Alpert, Galun et al. \(2007\)](#). In more recent work, [Pont-Tuset, Arbeláez et al. \(2017\)](#) speed up normalized cuts and extend it to multiple scales to obtain state-of-the-art results on both the Berkeley Segmentation Dataset as well as (at the time) object proposals on the VOC and COCO datasets.

7.6 Additional reading

One of the seminal papers on feature detection, description, and matching is by [Lowe \(2004\)](#). Comprehensive surveys and evaluations of such techniques have been made by [Schmid, Mohr, and Bauckhage \(2000\)](#), [Mikolajczyk and Schmid \(2005\)](#), [Mikolajczyk, Tuytelaars et](#)

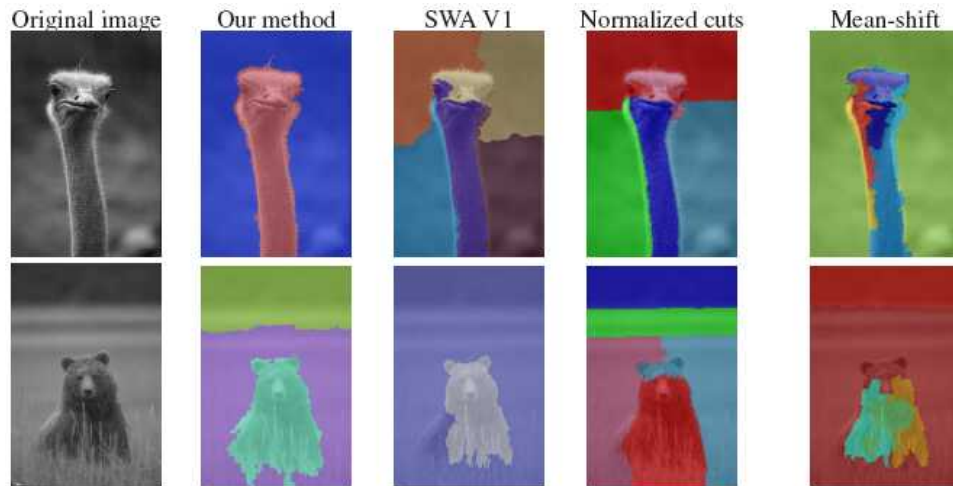


Figure 7.56 Comparative segmentation results (Alpert, Galun *et al.* 2007) © 2007 IEEE. “Our method” refers to the probabilistic bottom-up merging algorithm developed by Alpert *et al.*

al. (2005), and Tuytelaars and Mikolajczyk (2008), while Shi and Tomasi (1994) and Triggs (2004) also provide nice reviews.

In the area of feature detectors (Mikolajczyk, Tuytelaars *et al.* 2005), in addition to such classic approaches as Förstner–Harris (Förstner 1986; Harris and Stephens 1988) and difference of Gaussians (Lindeberg 1993, 1998b; Lowe 2004), maximally stable extremal regions (MSERs) are widely used for applications that require affine invariance (Matas, Chum *et al.* 2004; Nistér and Stewénus 2008). More recent interest point detectors are discussed by Xiao and Shah (2003), Koethe (2003), Carneiro and Jepson (2005), Kenney, Zuliani, and Manjunath (2005), Bay, Ess *et al.* (2008), Platel, Balmachnova *et al.* (2006), and Rosten, Porter, and Drummond (2010), as are techniques based on line matching (Zoghalmi, Faugeras, and Deriche 1997; Bartoli, Coquerelle, and Sturm 2004) and region detection (Kadir, Zisserman, and Brady 2004; Matas, Chum *et al.* 2004; Tuytelaars and Van Gool 2004; Corso and Hager 2005). Three recent papers with nice reviews of DNN-based feature detectors are Balntas, Lenc *et al.* (2020), Barroso-Laguna, Riba *et al.* (2019), and Tian, Balntas *et al.* (2020).

A variety of local feature descriptors (and matching heuristics) are surveyed and compared by Mikolajczyk and Schmid (2005). More recent publications in this area include those by van de Weijer and Schmid (2006), Abdel-Hakim and Farag (2006), Winder and Brown (2007), and Hua, Brown, and Winder (2007) and the recent evaluations by Balntas, Lenc *et al.* (2020) and Jin, Mishkin *et al.* (2021). Techniques for efficiently matching features include k-d trees (Beis and Lowe 1999; Lowe 2004; Muja and Lowe 2009), pyramid matching kernels (Grauman and Darrell 2005), metric (vocabulary) trees (Nistér and Stewénus 2006),

variety of multi-dimensional hashing techniques (Shakhnarovich, Viola, and Darrell 2003; Torralba, Weiss, and Fergus 2008; Weiss, Torralba, and Fergus 2008; Kulis and Grauman 2009; Raginsky and Lazebnik 2009), and product quantization (Jégou, Douze, and Schmid 2010; Johnson, Douze, and Jégou 2021). A good review of large-scale systems for instance retrieval is Zheng, Yang, and Tian (2018).

The classic reference on feature detection and tracking is Shi and Tomasi (1994). More recent work in this field has focused on learning better matching functions for specific features (Avidan 2001; Jurie and Dhome 2002; Williams, Blake, and Cipolla 2003; Lepetit and Fua 2005; Lepetit, Pilet, and Fua 2006; Hinterstoisser, Benhimane *et al.* 2008; Rogez, Rihan *et al.* 2008; Özuysal, Calonder *et al.* 2010).

A highly cited and widely used edge detector is the one developed by Canny (1986). Alternative edge detectors as well as experimental comparisons can be found in publications by Nalwa and Binford (1986), Nalwa (1987), Deriche (1987), Freeman and Adelson (1991), Nalwa (1993), Heath, Sarkar *et al.* (1998), Crane (1997), Ritter and Wilson (2000), Bowyer, Kranenburg, and Dougherty (2001), Arbeláez, Maire *et al.* (2011), and Pont-Tuset, Arbeláez *et al.* (2017). The topic of scale selection in edge detection is nicely treated by Elder and Zucker (1998), while approaches to color and texture edge detection can be found in Ruzon and Tomasi (2001), Martin, Fowlkes, and Malik (2004), and Gevers, van de Weijer, and Stokman (2006). Edge detectors have also been combined with region segmentation techniques to further improve the detection of semantically salient boundaries (Maire, Arbelaez *et al.* 2008; Arbeláez, Maire *et al.* 2011; Xiaofeng and Bo 2012; Pont-Tuset, Arbeláez *et al.* 2017). Edges linked into contours can be smoothed and manipulated for artistic effect (Lowe 1989; Finkelstein and Salesin 1994; Taubin 1995) and used for recognition (Belongie, Malik, and Puzicha 2002; Tek and Kimia 2003; Sebastian and Kimia 2005).

The topic of active contours has a long history, beginning with the seminal work on snakes and other energy-minimizing variational methods (Kass, Witkin, and Terzopoulos 1988; Cootes, Cooper *et al.* 1995; Blake and Isard 1998), continuing through techniques such as intelligent scissors (Mortensen and Barrett 1995, 1999; Pérez, Blake, and Gangnet 2001), and culminating in level sets (Malladi, Sethian, and Vemuri 1995; Caselles, Kimmel, and Sapiro 1997; Sethian 1999; Paragios and Deriche 2000; Sapiro 2001; Osher and Paragios 2003; Paragios, Faugeras *et al.* 2005; Cremers, Rousson, and Deriche 2007; Rousson and Paragios 2008; Paragios and Sgallari 2009), which are currently the most widely used active contour methods.

An early, well-regarded paper on straight line extraction in images was written by Burns, Hanson, and Riseman (1986). Their idea of bottom-up *line-support regions* was extended by Grompone von Gioi, Jakubowicz *et al.* (2008) to construct the popular LSD line segment

detector. The literature on vanishing point detection is quite vast and still evolving (Quan and Mohr 1989; Collins and Weiss 1990; Brillaut-O'Mahoney 1991; McLean and Kotturi 1995; Becker and Bove 1995; Shufelt 1999; Tuytelaars, Van Gool, and Proesmans 1997; Schaffalitzky and Zisserman 2000; Antone and Teller 2002; Rother 2002; Košecká and Zhang 2005; Denis, Elder, and Estrada 2008; Pflugfelder 2008; Tardif 2009; Bazin, Seo *et al.* 2012; Antunes and Barreto 2013; Zhou, Qi *et al.* 2019a). Simultaneous line and junction detection techniques have also been developed (Huang, Wang *et al.* 2018; Zhang, Li *et al.* 2019).

The topic of image segmentation is closely related to clustering techniques, which are treated in a number of monographs and review articles (Jain and Dubes 1988; Kaufman and Rousseeuw 1990; Jain, Duin, and Mao 2000; Jain, Topchy *et al.* 2004). Some early segmentation techniques include those described by Brice and Fennema (1970), Pavlidis (1977), Riesenman and Arbib (1977), Ohlander, Price, and Reddy (1978), Rosenfeld and Davis (1979), and Haralick and Shapiro (1985), while examples of newer techniques are developed by Leclerc (1989), Mumford and Shah (1989), Shi and Malik (2000), and Felzenszwalb and Huttenlocher (2004).

Arbeláez, Maire *et al.* (2011) and Pont-Tuset, Arbeláez *et al.* (2017) provide good reviews of automatic segmentation techniques and compare their performance on the Berkeley Segmentation Dataset and Benchmark (Martin, Fowlkes *et al.* 2001).¹⁷ Additional comparison papers and databases include those by Unnikrishnan, Pantofaru, and Hebert (2007), Alpert, Galun *et al.* (2007), and Estrada and Jepson (2009).

Techniques for segmenting images based on local pixel similarities combined with aggregation or splitting methods include watersheds (Vincent and Soille 1991; Beare 2006; Arbeláez, Maire *et al.* 2011), region splitting (Ohlander, Price, and Reddy 1978), region merging (Brice and Fennema 1970; Pavlidis and Liow 1990; Jain, Topchy *et al.* 2004), as well as graph-based and probabilistic multi-scale approaches (Felzenszwalb and Huttenlocher 2004; Alpert, Galun *et al.* 2007).

Mean-shift algorithms, which find modes (peaks) in a density function representation of the pixels, are presented by Comaniciu and Meer (2002) and Paris and Durand (2007). Parametric mixtures of Gaussians can also be used to represent and segment such pixel densities (Bishop 2006; Ma, Derksen *et al.* 2007).

The seminal work on spectral (eigenvalue) methods for image segmentation is the *normalized cut* algorithm of Shi and Malik (2000). Related work includes that by Weiss (1999), Meilă and Shi (2000), Meilă and Shi (2001), Malik, Belongie *et al.* (2001), Ng, Jordan, and Weiss (2001), Yu and Shi (2003), Cour, Bénézit, and Shi (2005), Sharon, Galun *et al.* (2006), Tolliver and Miller (2006), and Wang and Oliensis (2010).

¹⁷<http://www.eecs.berkeley.edu/Research/Projects/CS/vision/grouping/segbench>.

7.7 Exercises

Ex 7.1: Interest point detector. Implement one or more keypoint detectors and compare their performance (with your own or with a classmate's detector).

Possible detectors:

- Laplacian or Difference of Gaussian;
- Förstner–Harris Hessian (try different formula variants given in (7.9–7.11));
- oriented/steerable filter, looking for either second-order high second response or two edges in a window (Koethe 2003), as discussed in Section 7.1.1.
- any of the newer DNN-based detectors.

Other detectors are described in Mikolajczyk, Tuytelaars *et al.* (2005), Tuytelaars and Mikolajczyk (2008), and Balntas, Lenc *et al.* (2020). Additional optional steps could include:

1. Compute the detections on a sub-octave pyramid and find 3D maxima.
2. Find local orientation estimates using steerable filter responses or a gradient histogramming method.
3. Implement non-maximal suppression, such as the adaptive technique of Brown, Szeliski, and Winder (2005).
4. Vary the window shape and size (prefilter and aggregation).

To test for repeatability, download the code from <https://www.robots.ox.ac.uk/~vgg/research/affine> (Mikolajczyk, Tuytelaars *et al.* 2005; Tuytelaars and Mikolajczyk 2008) or simply rotate or shear your own test images. (Pick a domain you may want to use later, e.g., for outdoor stitching.)

Be sure to measure and report the stability of your scale and orientation estimates.

Ex 7.2: Interest point descriptor. Implement two or more descriptors from Section 7.1.2 (steered to local scale and orientation estimates, if appropriate) and compare their performance on some images of your own choosing.

You can either use the evaluation methodologies (and optionally software) described in Mikolajczyk and Schmid (2005), Balntas, Lenc *et al.* (2020), or Jin, Mishkin *et al.* (2021).

Ex 7.3: ROC curve computation. Given a pair of curves (histograms) plotting the number of matching and non-matching features as a function of Euclidean distance d as shown in

Figure 7.22b, derive an algorithm for plotting a ROC curve (Figure 7.22a). In particular, let $t(d)$ be the distribution of true matches and $f(d)$ be the distribution of (false) non-matches. Write down the equations for the ROC, i.e., TPR(FPR), and the AUC.

(Hint: Plot the cumulative distributions $T(d) = \int t(d)$ and $F(d) = \int f(d)$ and see if these help you derive the TPR and FPR at a given threshold θ .)

Ex 7.4: Feature matcher. After extracting features from a collection of overlapping or distorted images,¹⁸ match them up by their descriptors either using nearest neighbor matching or a more efficient matching strategy such as a k-d tree.

See whether you can improve the accuracy of your matches using techniques such as the nearest neighbor distance ratio.

Ex 7.5: Feature tracker. Instead of finding feature points independently in multiple images and then matching them, find features in the first image of a video or image sequence and then re-locate the corresponding points in the next frames using either search and gradient descent (Shi and Tomasi 1994) or learned feature detectors (Lepetit, Pilet, and Fua 2006; Fossati, Dimitrijevic *et al.* 2007). When the number of tracked points drops below a threshold or new regions in the image become visible, find additional points to track.

(Optional) Winnow out incorrect matches by estimating a homography (8.19–8.23) or fundamental matrix (Section 11.3.3).

(Optional) Refine the accuracy of your matches using the iterative registration algorithm described in Section 9.2 and Exercise 9.2.

Ex 7.6: Facial feature tracker. Apply your feature tracker to tracking points on a person's face, either manually initialized to interesting locations such as eye corners or automatically initialized at interest points.

(Optional) Match features between two people and use these features to perform image morphing (Exercise 3.25).

Ex 7.7: Edge detector. Implement an edge detector of your choice. Compare its performance to that of your classmates' detectors or code downloaded from the internet.

A simple but well-performing sub-pixel edge detector can be created as follows:

1. Blur the input image a little,

$$B_{\sigma}(\mathbf{x}) = G_{\sigma}(\mathbf{x}) * I(\mathbf{x}).$$

¹⁸<https://www.robots.ox.ac.uk/~vgg/research/affine>.

2. Construct a Gaussian pyramid (Exercise 3.17),

$$P = \text{Pyramid}\{B_\sigma(\mathbf{x})\}$$

3. Subtract an interpolated coarser-level pyramid image from the original resolution blurred image,

$$S(\mathbf{x}) = B_\sigma(\mathbf{x}) - P.\text{InterpolatedLevel}(L).$$

4. For each quad of pixels, $\{(i, j), (i + 1, j), (i, j + 1), (i + 1, j + 1)\}$, count the number of zero crossings along the four edges.
5. When there are exactly two zero crossings, compute their locations using (7.25) and store these edgel endpoints along with the midpoint in the edgel structure.
6. For each edgel, compute the local gradient by taking the horizontal and vertical differences between the values of S along the zero crossing edges.
7. Store the magnitude of this gradient as the edge strength and either its orientation or that of the segment joining the edgel endpoints as the edge orientation.
8. Add the edgel to a list of edgels or store it in a 2D array of edgels (addressed by pixel coordinates).

Ex 7.8: Edge linking and thresholding. Link up the edges computed in the previous exercise into chains and optionally perform thresholding with hysteresis.

The steps may include:

1. Store the edgels either in a 2D array (say, an integer image with indices into the edgel list) or pre-sort the edgel list first by (integer) x coordinates and then y coordinates, for faster neighbor finding.
2. Pick up an edgel from the list of unlinked edgels and find its neighbors in both directions until no neighbor is found or a closed contour is obtained. Flag edgels as linked as you visit them and push them onto your list of linked edgels.
3. (Optional) Perform hysteresis-based thresholding (Canny 1986). Use two thresholds “hi” and “lo” for the edge strength. A candidate edgel is considered an edge if either its strength is above the “hi” threshold or its strength is above the “lo” threshold and it is (recursively) connected to a previously detected edge.
4. (Optional) Link together contours that have small gaps but whose endpoints have similar orientations.

5. (Optional) Find junctions between adjacent contours, e.g., using some of the ideas (or references) from [Maire, Arbelaez *et al.* \(2008\)](#).

Ex 7.9: Contour matching. Convert a closed contour (linked edgel list) into its arc-length parameterization and use this to match object outlines.

The steps may include:

1. Walk along the contour and create a list of (x_i, y_i, s_i) triplets, using the arc-length formula

$$s_{i+1} = s_i + \|\mathbf{x}_{i+1} - \mathbf{x}_i\|. \quad (7.42)$$

2. Resample this list onto a regular set of (x_j, y_j, j) samples using linear interpolation of each segment.
3. Compute the average values of x and y , i.e., $\bar{x}$ and $\bar{y}$ and subtract them from your sampled curve points.
4. Resample the original (x_i, y_i, s_i) piecewise-linear function onto a length-independent set of samples, say $j \in [0, 1023]$. (Using a length which is a power of two makes subsequent Fourier transforms more convenient.)
5. Compute the Fourier transform of the curve, treating each (x, y) pair as a complex number.
6. To compare two curves, fit a linear equation to the phase difference between the two curves. (Careful: phase wraps around at 360° . Also, you may wish to weight samples by their Fourier spectrum magnitude—see Section 9.1.2.)
7. (Optional) Prove that the constant phase component corresponds to the temporal shift in s , while the linear component corresponds to rotation.

Of course, feel free to try any other curve descriptor and matching technique from the computer vision literature ([Tek and Kimia 2003](#); [Sebastian and Kimia 2005](#)).

Ex 7.10: Jigsaw puzzle solver—challenging. Write a program to automatically solve a jigsaw puzzle from a set of scanned puzzle pieces. Your software may include the following components:

1. Scan the pieces (either face up or face down) on a flatbed scanner with a distinctively colored background.
2. (Optional) Scan in the box top to use as a low-resolution reference image.

3. Use color-based thresholding to isolate the pieces.
4. Extract the contour of each piece using edge finding and linking.
5. (Optional) Re-represent each contour using an arc-length or some other re-parameterization. Break up the contours into meaningful matchable pieces. (Is this hard?)
6. (Optional) Associate color values with each contour to help in the matching.
7. (Optional) Match pieces to the reference image using some rotationally invariant feature descriptors.
8. Solve a global optimization or (backtracking) search problem to snap pieces together and place them in the correct location relative to the reference image.
9. Test your algorithm on a succession of more difficult puzzles and compare your results with those of others.

For some additional ideas, have a look at [Cho, Avidan, and Freeman \(2010\)](#).

Ex 7.11: Successive approximation line detector. Implement a line simplification algorithm (Section 7.4.1) ([Ramer 1972](#); [Douglas and Peucker 1973](#)) to convert a hand-drawn curve (or linked edge image) into a small set of polylines.

(Optional) Re-render this curve using either an approximating or interpolating spline or Bezier curve ([Szeliski and Ito 1986](#); [Bartels, Beatty, and Barsky 1987](#); [Farin 2002](#)).

Ex 7.12: Line fitting uncertainty. Estimate the uncertainty (covariance) in your line fit using uncertainty analysis.

1. After determining which edgels belong to the line segment (using either successive approximation or Hough transform), re-fit the line segment using total least squares ([Van Huffel and Vandewalle 1991](#); [Van Huffel and Lemmerling 2002](#)), i.e., find the mean or centroid of the edgels and then use eigenvalue analysis to find the dominant orientation.
2. Compute the perpendicular errors (deviations) to the line and robustly estimate the variance of the fitting noise using an estimator such as MAD (Appendix B.3).
3. (Optional) re-fit the line parameters by throwing away outliers or using a robust norm or influence function.
4. Estimate the error in the perpendicular location of the line segment and its orientation.

Ex 7.13: Vanishing points. Compute the vanishing points in an image using one of the techniques described in Section 7.4.3 and optionally refine the original line equations associated with each vanishing point. Your results can be used later to track a target or reconstruct architecture (Section 13.6.1).

Ex 7.14: Vanishing point uncertainty. Perform an uncertainty analysis on your estimated vanishing points. You will need to decide how to represent your vanishing point, e.g., homogeneous coordinates on a sphere, to handle vanishing points near infinity.

See the discussion of Bingham distributions by Collins and Weiss (1990) for some ideas.

Ex 7.15: Region segmentation. Implement one of the region segmentation algorithms described in this chapter. Some popular segmentation algorithms include:

- k-means (Section 5.2.2);
- mixtures of Gaussians (Section 5.2.2);
- mean shift (Section 7.5.2);
- normalized cuts (Section 7.5.3);
- similarity graph-based segmentation (Section 7.5.1);
- binary Markov random fields solved using graph cuts (Section 4.3.2).

Apply your region segmentation to a video sequence and use it to track moving regions from frame to frame.

Alternatively, test out your segmentation algorithm on the Berkeley segmentation database (Martin, Fowlkes *et al.* 2001).